

Exercise Solutions

Optimization in Machine Learning

Solution versions only - generated for review.

Exercise 5: Univariate Optimization	p. 2
Exercise 6: Multivariate Optimization 1	p. 18
Exercise 7: Multivariate Optimization 2	p. 41
Exercise 8: Multivariate Optimization 3 (SGD)	p. 56
Exercise 9: Multivariate Optimization 4 (Newton-Raphson + Gauss-Newton)	p. 66
Exercise 10: Linear Programming (Sparse Quantile Regression)	p. 79
Exercise 11: Nonlinear Programming (Lagrange + KKT + SVM)	p. 93
Exercise 13: Bayesian Optimization	p. 108

Exercise 5: Univariate Optimization

Optimization in Machine Learning

Reproducibility

Produced with Python 3.10 and R 4.4.1. To reproduce, install the pinned package versions below.

Python

```
pip install numpy==2.0.2 matplotlib==3.10.8
```

R

```
install.packages("remotes")  
remotes::install_version("ggplot2", "3.5.1")
```

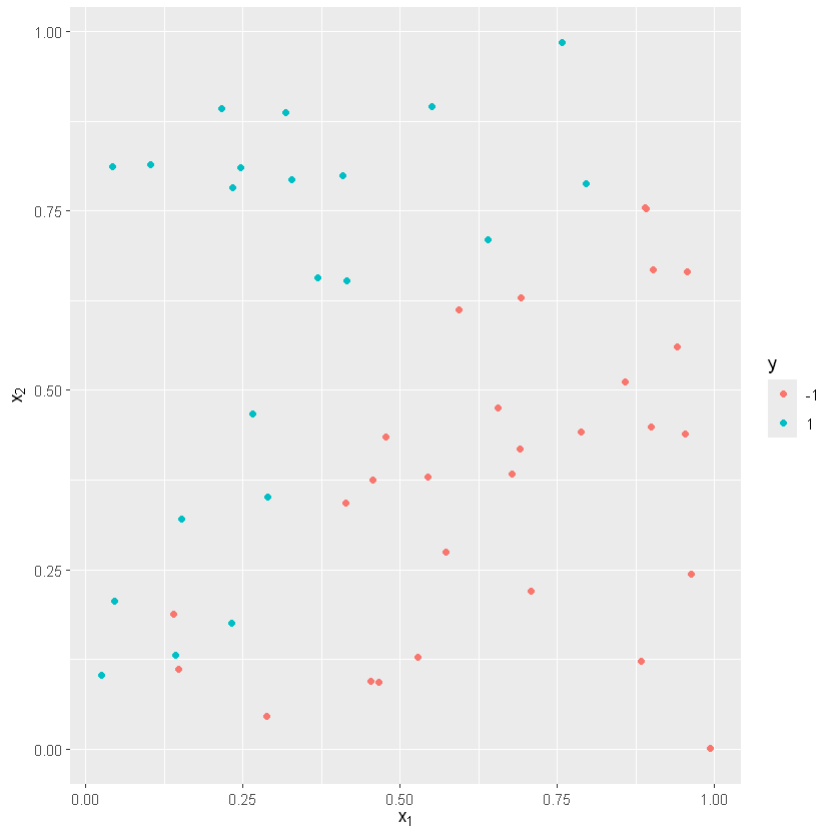
Setup

You are given the following data situation:

R

```
library(ggplot2)  
  
set.seed(123)  
  
X <- matrix(runif(100), ncol = 2)  
y <- -((X %*% c(-1, 1) + rnorm(50, 0, 0.1) < 0) * 2 - 1)  
df <- as.data.frame(X)  
df$type <- as.character(y)
```

```
ggplot(df) +
  geom_point(aes(x = V1, y = V2, color = type)) +
  xlab(expression(x[1])) +
  ylab(expression(x[2])) +
  labs(color = "y")
```



Python

```
import numpy as np
import matplotlib.pyplot as plt

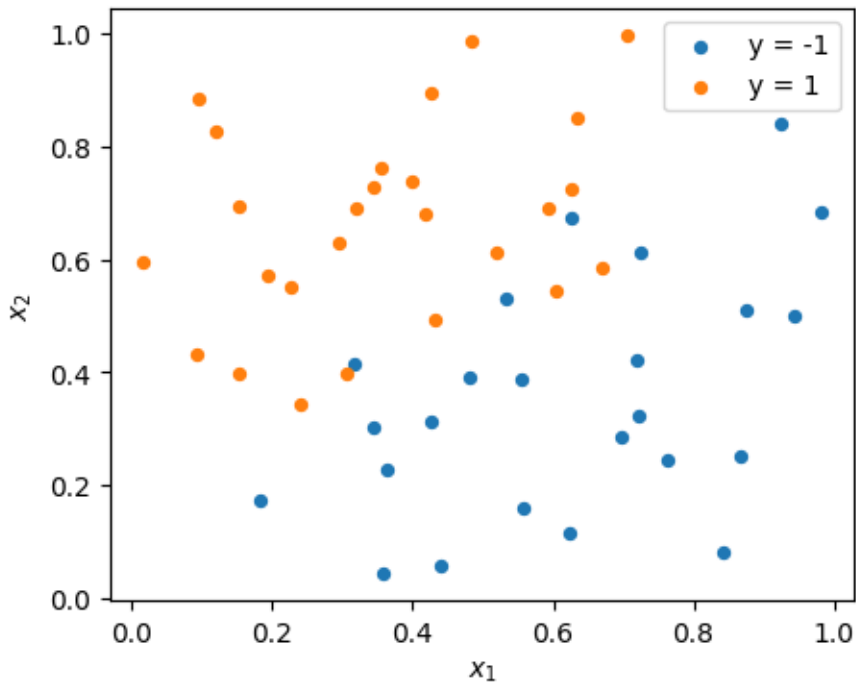
np.random.seed(123)

X = np.random.uniform(size=(50, 2))
lin = X @ np.array([-1.0, 1.0]) + np.random.normal(0, 0.1, 50)
y = -((lin < 0).astype(int) * 2 - 1)
```

```

fig, ax = plt.subplots(figsize=(5, 4))
for label in (-1, 1):
    mask = y == label
    ax.scatter(X[mask, 0], X[mask, 1], label=f"y = {label}", s=18)
ax.set_xlabel(r"$x_1$")
ax.set_ylabel(r"$x_2$")
ax.legend()
plt.show()

```



i Note

R vs. Python parity. R's `set.seed(123)` and Python's `np.random.seed(123)` use *different* RNGs. The data above (and consequently the optima below) differ between languages. Both implementations are correct; only the underlying data differs. When comparing R and Python, look at *algorithmic behavior* (convergence, bracket shrinking, zig-zag pattern) rather than expecting numerically identical θ values.

In the following we want to estimate a linear SVM without intercept and with $\lambda = 1$. We assume we know that $\theta_2 = 2$.

(a) Convexity of a 1D slice

Show that if $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ is convex then $g_c : \mathbb{R} \rightarrow \mathbb{R}$, $x \mapsto f(x, c)$ for any $c \in \mathbb{R}$ is convex.

Solution.

Since f is convex, for arbitrary $\mathbf{x}, \mathbf{y} \in \mathbb{R}^2$ and $t \in [0, 1]$:

$$f(\mathbf{x} + t(\mathbf{y} - \mathbf{x})) \leq f(\mathbf{x}) + t(f(\mathbf{y}) - f(\mathbf{x})).$$

Apply this to $\mathbf{x}_c = (x, c)^\top$ and $\mathbf{y}_c = (y, c)^\top$ with $x, y \in \mathbb{R}$ and fixed $c \in \mathbb{R}$. The right-hand sides equal $g_c(x)$ and $g_c(y)$, and the left-hand side equals $g_c(x + t(y - x))$, giving

$$g_c(x + t(y - x)) \leq g_c(x) + t(g_c(y) - g_c(x)).$$

Hence g_c is convex.

Geometric intuition. Fixing one coordinate restricts f to the affine line $\{(x, c) : x \in \mathbb{R}\}$. Convexity is preserved under any affine restriction.

Why we care. Every 1D solver in this exercise (golden-section, Brent's, alternating univariate minimization) only works on convex 1D functions. This proof is what licenses applying them to one parameter at a time of a *multivariate* convex SVM objective.

(b) Why the non-geometric primal SVM

Explain why the non-geometric primal linear SVM formulation should be used rather than the geometric one if we want to find θ_1 via the golden-ratio algorithm.

Note: We use this algorithm for educational purposes; in practice more advanced algorithms are typically used.

Solution.

Recall the two equivalent SVM primal formulations (related by $C = \frac{1}{2\lambda}$):

Geometric primal:

$$\min_{\theta, \theta_0, \zeta} \quad \frac{1}{2} \|\theta\|^2 + C \sum_{i=1}^n \zeta_i \quad \text{s.t.} \quad y^{(i)}(\theta^\top \mathbf{x}^{(i)} + \theta_0) \geq 1 - \zeta_i, \quad \zeta_i \geq 0.$$

Non-geometric primal:

$$\min_{\theta} \quad \lambda \|\theta\|^2 + \sum_{i=1}^n \max(0, 1 - y^{(i)} \theta^\top \mathbf{x}^{(i)}).$$

Golden-section and Brent's method are *unconstrained* 1D solvers — they have no mechanism for the geometric primal's $2n$ inequality constraints, and restricting to one parameter (per (a)) does not remove them. The non-geometric primal is unconstrained and convex in θ ; by (a) its restriction to θ_1 (with θ_2 fixed) stays convex, so a 1D solver applies directly.

(c) Golden-section search

Find θ_1 via the golden-section algorithm. Implement the algorithm yourself. Use absolute error 0.01 as termination criterion and $[-3, 3]$ as the starting interval.

Solution.

R

```
# SVM objective (no intercept, lambda = 1)
f <- function(theta) {
  theta %*% theta +
    sum(sapply(1 - y * (X %*% theta), function(x) max(x, 0)))
}

# Restrict to theta_1 with theta_2 fixed at 2
ft1 <- function(theta_1) f(c(theta_1, 2))

# phi = (sqrt(5) - 1) / 2 is the golden ratio reciprocal (~0.618).
# Picking the inner candidates at this fraction makes each iteration shrink
# the bracket by exactly (1-phi) ~= 0.382 AND lets us reuse one of the two
# previous evaluation points the next round - so only ONE new function call
# per iteration is needed.
phi <- (sqrt(5) - 1) / 2

gr <- function(f, lx = -3, rx = 3, abs_error = 0.01) {
  xbest_old <- Inf
  iters <- 0
  dist <- rx - lx
  cx <- c(lx + (1 - phi) * dist, rx - (1 - phi) * dist)

  while (TRUE) {
```

```

    iters <- iters + 1
    fcx1 <- f(cx[1])
    fcx2 <- f(cx[2])

    # Keep the side containing the better candidate; replace the discarded
    # candidate with its golden-section reflection so the SAME outer point
    # plus ONE new point cover the new (smaller) bracket.
    if (fcx1 < fcx2) {
        fbest <- fcx1; xbest <- cx[1]
        rx <- cx[2]
        cx[2] <- cx[2] - (cx[1] - lx)
    } else {
        fbest <- fcx2; xbest <- cx[2]
        lx <- cx[1]
        cx[1] <- cx[1] + (rx - cx[2])
    }
    cx <- sort(cx)

    if (abs(xbest_old - xbest) < abs_error) {
        return(list(xbest = xbest, fbest = fbest, iters = iters))
    }
    xbest_old <- xbest
}
}

res_gr <- gr(ft1)
cat(sprintf("golden-section: theta_1* = %.4f, f = %.4f, iters = %d\n",
            res_gr$xbest, res_gr$fbest, res_gr$iters))

```

golden-section: theta_1* = -2.4590, f = 29.9129, iters = 5

Python

```

def f(theta):
    """SVM objective: ||theta||^2 + sum hinge(1 - y_i (X_i . theta))."""
    theta = np.asarray(theta, dtype=float)
    return float(theta @ theta + np.maximum(1 - y * (X @ theta), 0).sum())

def ft1(theta_1):

```

```

    return f(np.array([theta_1, 2.0]))

# phi = (sqrt(5) - 1) / 2 is the golden ratio reciprocal (~0.618).
# Picking the inner candidates at this fraction shrinks the bracket by exactly
# (1 - phi) ~= 0.382 each iteration AND lets us reuse one of the two previous
# evaluation points - so only ONE new function call per iteration is needed.
phi = (np.sqrt(5) - 1) / 2

def gr(f, lx=-3.0, rx=3.0, abs_error=0.01):
    xbest_old = np.inf
    iters = 0
    dist = rx - lx
    cx = [lx + (1 - phi) * dist, rx - (1 - phi) * dist]
    while True:
        iters += 1
        fcx1 = f(cx[0])
        fcx2 = f(cx[1])
        # Keep the side containing the better candidate; reflect the discarded
        # side's outer bound through golden ratio so we reuse one prior eval.
        if fcx1 < fcx2:
            fbest, xbest = fcx1, cx[0]
            rx = cx[1]
            cx[1] = cx[1] - (cx[0] - lx)
        else:
            fbest, xbest = fcx2, cx[1]
            lx = cx[0]
            cx[0] = cx[0] + (rx - cx[1])
        cx = sorted(cx)
        if abs(xbest_old - xbest) < abs_error:
            return {"xbest": xbest, "fbest": fbest, "iters": iters}
        xbest_old = xbest

res_gr = gr(ft1)
print(f"golden-section: theta_1* = {res_gr['xbest']:.4f}, "
      f"f = {res_gr['fbest']:.4f}, iters = {res_gr['iters']}")

```

golden-section: theta_1* = -2.1246, f = 31.8233, iters = 4

(d) Parabola interpolation through three points

Given three points $(x_1, y_1), (x_2, y_2), (x_3, y_3)$ show that the parameters $a, b, c \in \mathbb{R}$ of the interpolating parabola $p(x) = ax^2 + bx + c$ can be found via

$$\begin{pmatrix} a \\ b \\ c \end{pmatrix} = \begin{pmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ x_3^2 & x_3 & 1 \end{pmatrix}^{-1} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix}.$$

Solution.

Three equations:

$$ax_i^2 + bx_i + c = y_i \quad \text{for } i = 1, 2, 3.$$

Stacking them in matrix form gives $\Lambda(a, b, c)^\top = (y_1, y_2, y_3)^\top$ with Λ as in the problem statement. Λ is the **Vandermonde matrix** of (x_1, x_2, x_3) ; its determinant is $(x_2 - x_1)(x_3 - x_1)(x_3 - x_2)$, which is nonzero iff the three x_i are pairwise distinct.

Failure mode worth knowing. In Brent's method (next part), if two of the bracket points coalesce, Λ becomes singular and parabolic interpolation breaks. Production Brent implementations guard against this by falling back to a golden-section step whenever the parabolic candidate is invalid.

(e) Brent's method

Find θ_1 via Brent's method. Implement a simplified version (only check whether the proposed point is in the current interval) of the algorithm. Use absolute error 0.01 as termination criterion and $[-3, 3]$ as the starting interval. For the first step, use a golden-ratio step.

Brent's idea. When the function looks parabolic near the minimum, fit a parabola through three candidate points (lx, rx, cx) and jump to its vertex (one quadratic-interpolation step from (d)). When the parabola gives a bad point (vertex outside the current bracket, or matrix near-singular), fall back to a golden-section step. Result: near-quadratic convergence on smooth functions, golden's safety on rough ones.

R

```

# One golden-section step (used as fallback inside brent).
gr_step <- function(f, lx, rx) {
  dist <- rx - lx
  cxs <- c(lx + (1 - phi) * dist, rx - (1 - phi) * dist)
  fcx <- sapply(cxs, f)
  if (fcx[1] < fcx[2]) {
    fbest <- fcx[1]; cx <- cxs[1]; rx <- cxs[2]
  } else {
    fbest <- fcx[2]; cx <- cxs[2]; lx <- cxs[1]
  }
  c(lx, rx, cx, fbest)
}

brent <- function(f, lx = -3, rx = 3, abs_error = 0.01) {
  xbest_old <- Inf
  iters <- 0; parabola_steps <- 0; golden_steps <- 0
  first_step <- TRUE
  cx <- NA_real_

  while (TRUE) {
    iters <- iters + 1

    if (first_step || cx <= lx || cx >= rx) {
      # First iteration OR previous candidate outside bracket: golden step
      res <- gr_step(f, lx, rx)
      lx <- res[1]; rx <- res[2]; cx <- res[3]; xbest <- cx; fbest <- res[4]
      golden_steps <- golden_steps + 1
      first_step <- FALSE
    } else {
      # Quadratic interpolation: parabola through (lx, rx, cx), jump to vertex
      xs <- c(lx, rx, cx)
      fxs <- sapply(xs, f)
      params <- solve(t(sapply(xs, function(x) c(x^2, x, 1)))), fxs)
      cx_new <- -params[2] / (2 * params[1])

      if (cx_new < rx & cx_new > lx) {
        cxs <- sort(c(cx, cx_new))
        fcx <- sapply(cxs, f)
        if (fcx[1] < fcx[2]) {
          fbest <- fcx[1]; cx <- cxs[1]; rx <- cxs[2]
        } else {
          fbest <- fcx[2]; cx <- cxs[2]; lx <- cxs[1]
        }
      }
    }
  }
}

```

```

    }
    xbest <- cx
    parabola_steps <- parabola_steps + 1
  } else {
    # Parabola vertex outside bracket: golden fallback
    res <- gr_step(f, lx, rx)
    lx <- res[1]; rx <- res[2]; cx <- res[3]; xbest <- cx; fbest <- res[4]
    golden_steps <- golden_steps + 1
  }
}

if (abs(xbest - xbest_old) < abs_error) {
  return(list(xbest = xbest, fbest = fbest, iters = iters,
             parabola_steps = parabola_steps, golden_steps = golden_steps))
}
xbest_old <- xbest
}
}

res_br <- brent(ft1)
cat(sprintf(paste0(
  "Brent:          theta_1* = %.4f, f = %.4f, ",
  "iters = %d (parabola=%d, golden=%d)\n"),
  res_br$xbest, res_br$fbest, res_br$iters,
  res_br$parabola_steps, res_br$golden_steps))
cat(sprintf("golden-section: iters = %d (from c)\n", res_gr$iters))

```

```

Brent:          theta_1* = -2.4095, f = 29.9033, iters = 5 (parabola=4, golden=1)
golden-section: iters = 5 (from c)

```

Python

```

def gr_step(f, lx, rx):
    """One golden-section step. Uses the parameter f (not a hardcoded ft1)."""
    dist = rx - lx
    cxs = [lx + (1 - phi) * dist, rx - (1 - phi) * dist]
    fcx = [f(c) for c in cxs]
    if fcx[0] < fcx[1]:
        fbest, cx, rx = fcx[0], cxs[0], cxs[1]
    else:

```

```

        fbest, cx, lx = fcx[1], cxs[1], cxs[0]
    return lx, rx, cx, fbest

def brent(f, lx=-3.0, rx=3.0, abs_error=0.01):
    xbest_old = np.inf
    iters = parabola_steps = golden_steps = 0
    first_step = True
    cx = None
    fbest = None
    while True:
        iters += 1

        if first_step or cx <= lx or cx >= rx:
            # First iter or previous candidate outside bracket: golden step
            lx, rx, cx, fbest = gr_step(f, lx, rx)
            xbest = cx
            golden_steps += 1
            first_step = False
        else:
            # Quadratic interpolation: parabola through (lx, rx, cx) -> vertex
            xs = [lx, rx, cx]
            fxs = [f(x) for x in xs]
            A = np.array([[x ** 2, x, 1.0] for x in xs])
            params = np.linalg.solve(A, fxs)
            cx_new = -params[1] / (2 * params[0])
            if lx < cx_new < rx:
                cxs = sorted([cx, cx_new])
                fcx = [f(c) for c in cxs]
                if fcx[0] < fcx[1]:
                    fbest, cx, rx = fcx[0], cxs[0], cxs[1]
                else:
                    fbest, cx, lx = fcx[1], cxs[1], cxs[0]
                xbest = cx
                parabola_steps += 1
            else:
                # Parabola vertex outside bracket: golden fallback
                lx, rx, cx, fbest = gr_step(f, lx, rx)
                xbest = cx
                golden_steps += 1

    if abs(xbest - xbest_old) < abs_error:

```

```

        return {
            "xbest": xbest, "fbest": fbest, "iters": iters,
            "parabola_steps": parabola_steps, "golden_steps": golden_steps,
        }
        xbest_old = xbest

res_br = brent(ft1)
print(f"Brent:          theta_1* = {res_br['xbest']:.4f}, "
      f"f = {res_br['fbest']:.4f}, iters = {res_br['iters']} "
      f"(parabola={res_br['parabola_steps']}, golden={res_br['golden_steps']})")
print(f"golden-section: iters = {res_br['iters']} (from c)")

```

```

Brent:          theta_1* = -1.8411, f = 31.7194, iters = 3 (parabola=2, golden=1)
golden-section: iters = 4 (from c)

```

Cost trade-off. Golden-section does no linear algebra per step but takes more steps. Brent solves a 3×3 system per parabolic step (~ 10 flops) and converges in fewer iterations. On smooth objectives the speedup is large; on the SVM hinge loss above (kink at every margin violation) the speedup is modest because parabolic fits are often poor. The iteration counts printed above make this trade-off concrete.

(f) Alternating univariate minimization

Now assume we do not know θ_2 . Initial guess is $\theta_2 = 0$. Alternately minimize w.r.t. either θ_1 or θ_2 via the golden-section method (starting interval reset to $[-3, 3]$ each time) while holding the other parameter fixed. Switch to the other parameter when absolute error is smaller than 0.01. Repeat this procedure 10 times.

We record the running (θ_1, θ_2) in a list named `trace` so we can plot the optimization path in (g).

R

```

t1 <- 0; t2 <- 0
trace <- matrix(c(t1, t2), nrow = 1)

for (i in 0:9) {
  if (i %% 2 == 0) {

```

```

    res <- gr(function(t) f(c(t, t2)))
    t1 <- res$xbest
  } else {
    res <- gr(function(t) f(c(t1, t)))
    t2 <- res$xbest
  }
  trace <- rbind(trace, c(t1, t2))
  cat(sprintf("iter %2d: theta = (%.4f, %.4f), f = %.4f\n",
              i, t1, t2, f(c(t1, t2))))
}

```

```

iter 0: theta = (-1.5836, 0.0000), f = 37.8786
iter 1: theta = (-1.5836, 1.5836), f = 32.4903
iter 2: theta = (-2.1246, 1.5836), f = 29.7429
iter 3: theta = (-2.1246, 1.5836), f = 29.7429
iter 4: theta = (-2.1246, 1.5836), f = 29.7429
iter 5: theta = (-2.1246, 1.5836), f = 29.7429
iter 6: theta = (-2.1246, 1.5836), f = 29.7429
iter 7: theta = (-2.1246, 1.5836), f = 29.7429
iter 8: theta = (-2.1246, 1.5836), f = 29.7429
iter 9: theta = (-2.1246, 1.5836), f = 29.7429

```

Python

```

t1, t2 = 0.0, 0.0
trace = [(t1, t2)]

for i in range(10):
    if i % 2 == 0:
        res = gr(lambda t: f(np.array([t, t2])))
        t1 = res["xbest"]
    else:
        res = gr(lambda t: f(np.array([t1, t])))
        t2 = res["xbest"]
    trace.append((t1, t2))
    print(f"iter {i:2d}: theta = ({t1:+.4f}, {t2:+.4f}), "
          f"f = {f(np.array([t1, t2])):.4f}")

```

```

iter 0: theta = (-0.7082, +0.0000), f = 46.6452

```

```

iter 1: theta = (-0.7082, +1.5836), f = 36.2619
iter 2: theta = (-1.5836, +1.5836), f = 33.1057
iter 3: theta = (-1.5836, +2.1246), f = 32.3456
iter 4: theta = (-2.1246, +2.1246), f = 31.5999
iter 5: theta = (-2.1246, +2.1246), f = 31.5999
iter 6: theta = (-2.1246, +2.1246), f = 31.5999
iter 7: theta = (-2.1246, +2.1246), f = 31.5999
iter 8: theta = (-2.1246, +2.1246), f = 31.5999
iter 9: theta = (-2.1246, +2.1246), f = 31.5999

```

(g) Optimization trace

How does the optimization trace of (f) look in parameter space?

Solution.

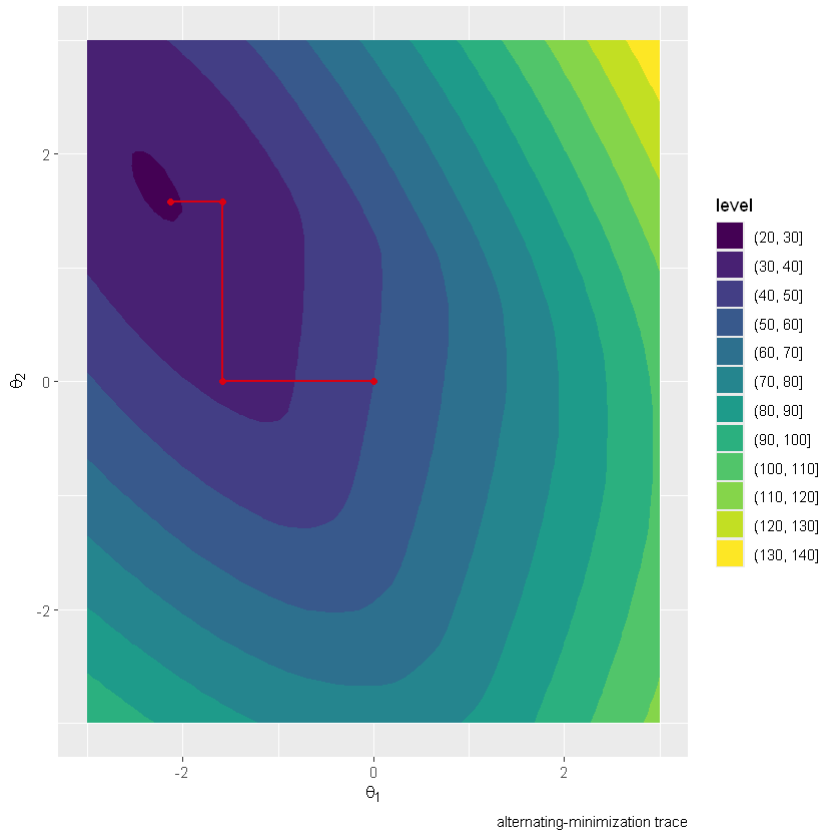
R

```

x <- seq(-3, 3, by = 0.1)
xx <- expand.grid(X1 = x, X2 = x)
fxx <- apply(xx, 1, f)
df_plot <- data.frame(xx = xx, fxx = fxx)
trace_df <- as.data.frame(trace)
colnames(trace_df) <- c("V1", "V2")

ggplot() +
  geom_contour_filled(data = df_plot, aes(x = xx.X1, y = xx.X2, z = fxx)) +
  geom_path(data = trace_df, aes(x = V1, y = V2),
            color = "#D90012", linewidth = 0.6) +
  geom_point(data = trace_df, aes(x = V1, y = V2),
             color = "#D90012", size = 1.5) +
  xlab(expression(theta[1])) +
  ylab(expression(theta[2])) +
  labs(caption = "alternating-minimization trace")

```



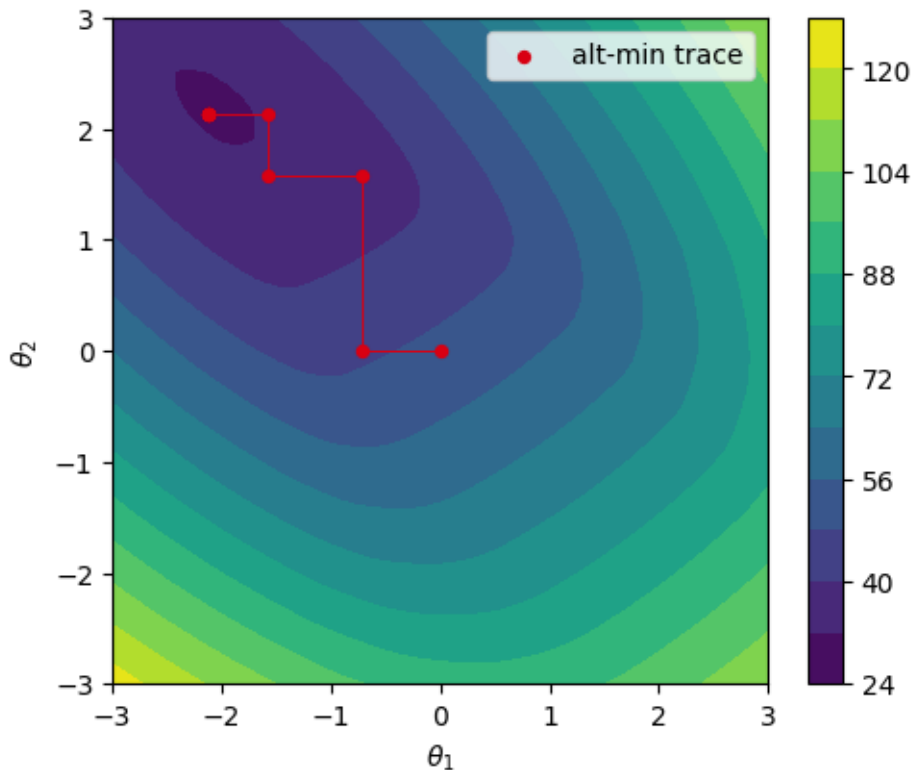
Python

```
grid = np.arange(-3, 3 + 0.1, 0.1)
T1, T2 = np.meshgrid(grid, grid)
Z = np.array([[f(np.array([t1_, t2_])) for t1_ in grid] for t2_ in grid])

trace_arr = np.array(trace)

fig, ax = plt.subplots(figsize=(5.5, 4.5))
cs = ax.contourf(T1, T2, Z, levels=14)
ax.plot(trace_arr[:, 0], trace_arr[:, 1], color="#D90012", linewidth=0.7)
ax.scatter(trace_arr[:, 0], trace_arr[:, 1], color="#D90012", s=18,
           label="alt-min trace")
ax.set_xlabel(r"$\theta_1$")
ax.set_ylabel(r"$\theta_2$")
ax.legend()
```

```
fig.colorbar(cs)
plt.show()
```



The trace is an axis-aligned zig-zag. Each segment is parallel to one coordinate axis because the alternating procedure updates one parameter at a time, with the other held fixed. It **converges well before the 10 iterations finish**: once the updates stop reducing the loss, the remaining iterates land on the same point and stack on top of each other, so the path shows only a handful of visible segments rather than ten.

Why this matters. This problem is well-conditioned, so coordinate-wise descent converges quickly; the slowness shows on *poorly-conditioned* problems (elongated, rotated objective contours), where each axis-aligned step makes only tiny progress along the long axis of the level sets. That is why optimizing one coordinate at a time is rarely competitive in higher dimensions, and it motivates the *gradient-based* methods of the next chapter, which move along the steepest-descent direction instead of along the coordinate axes.

Exercise 6: Multivariate Optimization 1

Optimization in Machine Learning

Reproducibility

Produced with Python 3.10 and R 4.4.1. To reproduce, install the pinned package versions below.

Python

```
pip install numpy==2.0.2 matplotlib==3.10.8
```

R

```
install.packages("remotes")
remotes::install_version("ggplot2", "3.5.1")
remotes::install_version("gridExtra", "2.3")
```

Setup

You are given the following data situation:

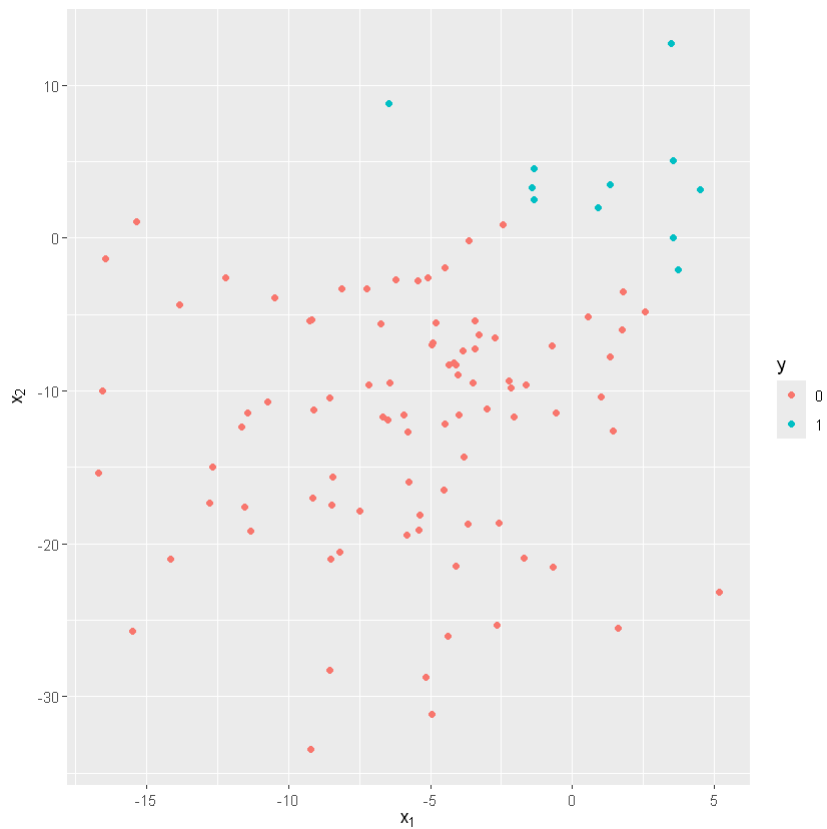
R

```
library(ggplot2)
library(gridExtra)

set.seed(314)
n <- 100
X <- cbind(rnorm(n, -5, 5), rnorm(n, -10, 10))
z <- 2 * X[, 1] + 3 * X[, 2]
```

```
pr <- 1 / (1 + exp(-z))
y <- as.integer(pr > 0.5)
df <- data.frame(X = X, y = y)

ggplot(df) +
  geom_point(aes(x = X.1, y = X.2, color = as.factor(y))) +
  xlab(expression(x[1])) +
  ylab(expression(x[2])) +
  labs(colour = "y")
```



Python

```
import numpy as np
import matplotlib.pyplot as plt

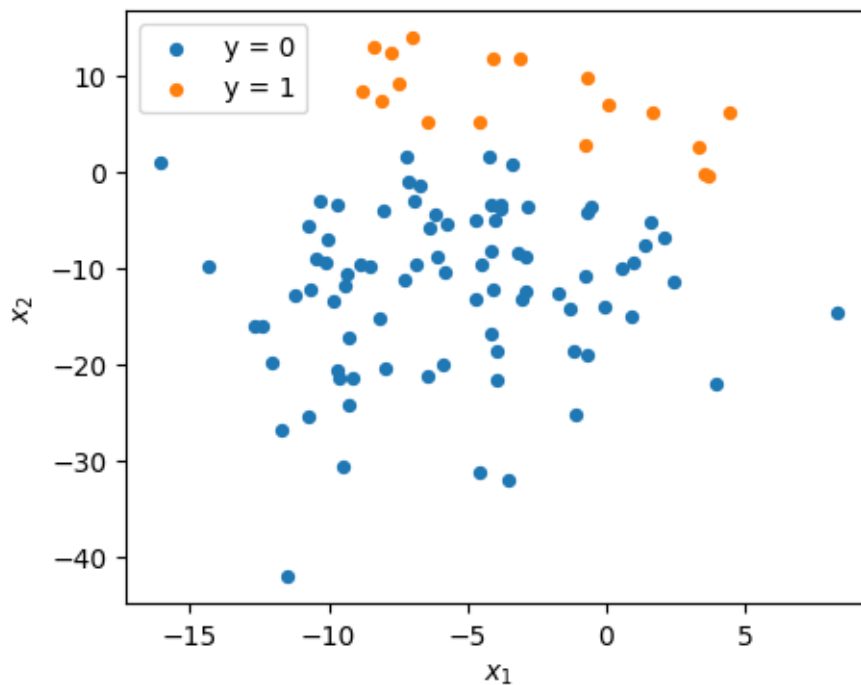
np.random.seed(314)
```

```

n = 100
X = np.column_stack([np.random.normal(-5, 5, n), np.random.normal(-10, 10, n)])
z = 2 * X[:, 0] + 3 * X[:, 1]
pr = 1 / (1 + np.exp(-z))
y = (pr > 0.5).astype(int)

fig, ax = plt.subplots(figsize=(5, 4))
for label in (0, 1):
    mask = y == label
    ax.scatter(X[mask, 0], X[mask, 1], label=f"y = {label}", s=18)
ax.set_xlabel(r"$x_1$"); ax.set_ylabel(r"$x_2$")
ax.legend()
plt.show()

```



i Note

R vs. Python parity. R's `set.seed(314)` and Python's `np.random.seed(314)` use *different* RNGs. The data above (and consequently the GD trajectories below) differ between languages. Both implementations are correct; only the underlying data differs. Compare *algorithmic behavior* (divergence, convergence, backtracking adaptation) rather than expecting numerically identical θ values.

In the following we want to estimate a logistic regression *without intercept* via gradient descent.

Note: We use vanilla GD for educational purposes; in practice more sophisticated algorithms are typically preferred.

(a) Complete separation makes the empirical risk unbounded below

The data above is **completely separable**: the two classes can be perfectly classified with a linear classifier. Show that in this situation, if $\tilde{\theta}$ perfectly classifies the data then

$$\mathcal{R}_{\text{emp}}(\tilde{\theta}) > \mathcal{R}_{\text{emp}}(\alpha\tilde{\theta}) \quad \text{for all } \alpha > 1.$$

Implication: the per-observation log-loss can be driven arbitrarily low by scaling θ , so the empirical risk has no finite minimum.

Solution.

We start with

$$\mathcal{R}_{\text{emp}}(\tilde{\theta}) = \sum_{i=1}^n \log\left(1 + \exp(\tilde{\theta}^\top \mathbf{x}^{(i)})\right) - y^{(i)} \tilde{\theta}^\top \mathbf{x}^{(i)} = \sum_{i=1}^n \begin{cases} \log\left(1 + \exp(\tilde{\theta}^\top \mathbf{x}^{(i)})\right) & \text{if } y^{(i)} = 0, \\ \log\left(1 + \exp(\tilde{\theta}^\top \mathbf{x}^{(i)})\right) - \tilde{\theta}^\top \mathbf{x}^{(i)} & \text{if } y^{(i)} = 1. \end{cases}$$

Since $\tilde{\theta}$ perfectly classifies the data, we know that

$$\begin{cases} \tilde{\theta}^\top \mathbf{x}^{(i)} < 0 & \text{if } y^{(i)} = 0, \\ \tilde{\theta}^\top \mathbf{x}^{(i)} \geq 0 & \text{if } y^{(i)} = 1. \end{cases}$$

Hence, we can focus on the functions $g(z) = \log(1 + \exp(-z))$ and $h(z) = \log(1 + \exp(z)) - z$ for $z > 0$ and study their monotonicity. (The $y = 1$ term is $h(z)$ with $z \geq 0$ directly; the $y = 0$ term $\log(1 + \exp(z))$ with $z < 0$ is $g(-z)$ with $-z > 0$. As functions g and h coincide — $\log(1 + \exp(z)) - z = \log(1 + \exp(-z))$ — so the two names just track which label case we are in.) We compute

$$g'(z) = - \underbrace{\frac{\exp(-z)}{1 + \exp(-z)}}_{>0} < 0, \quad h'(z) = \underbrace{\frac{\exp(z)}{1 + \exp(z)}}_{<1} - 1 < 0.$$

Therefore, both g and h are strictly monotonically decreasing. It follows that $\mathcal{R}_{\text{emp}}(\tilde{\theta}) > \mathcal{R}_{\text{emp}}(\alpha\tilde{\theta})$ for $\alpha > 1$.

(b) Visualize the empirical risk

Visualize $\mathcal{R}_{\text{emp}}$ on $[-1, 4] \times [-1, 4]$.

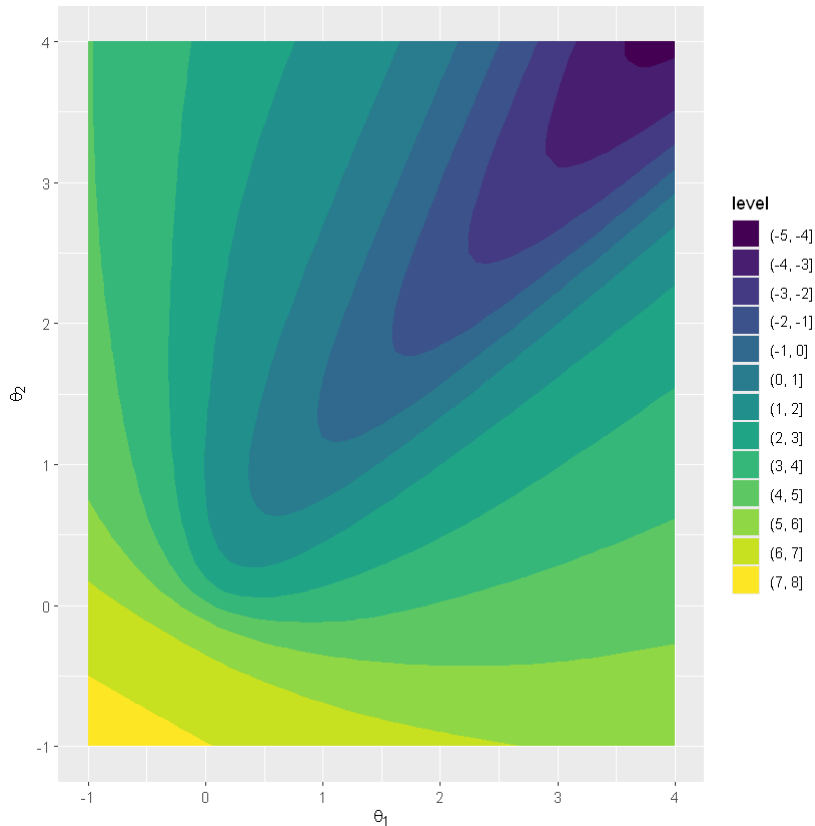
Solution.

R

```
# Empirical risk (lambda = 0 -> unregularized)
f <- function(theta, lambda) {
  z <- X %*% theta
  lambda * (theta %*% theta) + sum(-y * z + log(1 + exp(z)))
}

x <- seq(-1, 4, by = 0.1)
xx <- expand.grid(X1 = x, X2 = x)
# log-scale (as in (f)) to reveal the level-set shape across the wide range
fxx <- log(apply(xx, 1, function(t) f(t, 0)))
df_grid <- data.frame(xx, fxx = fxx)

ggplot(df_grid, aes(x = X1, y = X2, z = fxx)) +
  geom_contour_filled() +
  xlab(expression(theta[1])) +
  ylab(expression(theta[2]))
```



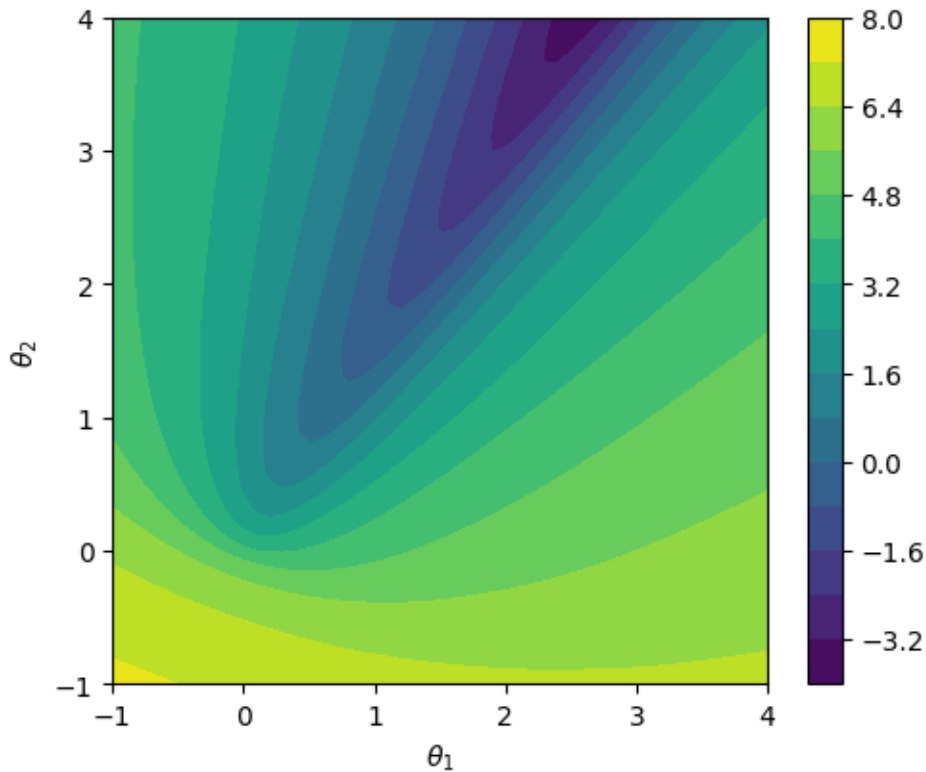
Python

```
def f(theta, lam):
    """L2-regularized logistic empirical risk (lam = 0 -> unregularized)."""
    theta = np.asarray(theta, dtype=float)
    z = X @ theta
    return float(lam * (theta @ theta) + np.sum(-y * z + np.log(1 + np.exp(z))))

grid = np.arange(-1, 4 + 0.1, 0.1)
T1, T2 = np.meshgrid(grid, grid)
# log-scale (as in (f)) to reveal the level-set shape across the wide range
Z = np.log(np.array([[f(np.array([t1, t2]), 0) for t1 in grid] for t2 in grid]))

fig, ax = plt.subplots(figsize=(5.5, 4.5))
cs = ax.contourf(T1, T2, Z, levels=14)
ax.set_xlabel(r"$\theta_1$"); ax.set_ylabel(r"$\theta_2$")
```

```
fig.colorbar(cs)
plt.show()
```



The level sets of $\mathcal{R}_{\text{emp}}$ open toward the upper-right: the risk decreases without bound as θ moves along that direction, consistent with (a).

(c) Gradient of the empirical risk

Find $\nabla_{\theta} \mathcal{R}_{\text{emp}}$ for arbitrary θ .

Solution.

Differentiating term-by-term:

$$\nabla_{\theta} \mathcal{R}_{\text{emp}} = \sum_{i=1}^n \left(\sigma(\theta^{\top} \mathbf{x}^{(i)}) - y^{(i)} \right) \mathbf{x}^{(i)}, \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Equivalently in vector form: $\mathbf{X}^{\top}(\sigma(\mathbf{X}\theta) - \mathbf{y})$ where σ is applied element-wise. This is the standard logistic-regression gradient.

Note on regularization. When the L2-regularized objective is $\lambda\|\theta\|^2 + \mathcal{R}_{\text{emp}}(\theta)$, the regularizer contributes an extra $2\lambda\theta$ to the gradient — important for parts (e), (g).

(d) Vanilla gradient descent

Solve the logistic regression via gradient descent. Use step size $\alpha = 0.01$, starting point $\theta^{[0]} = (0, 0)^\top$, and train for 500 steps. Repeat with $\alpha = 0.02$. Explain what you observe.

Hint: recall (a).

Solution.

R

```
# Gradient of the (regularized) logistic empirical risk.
# Returns a 1x2 row vector. NOTE: the regularizer contributes 2*lambda*theta.
df_t <- function(theta, lambda) {
  2 * lambda * t(theta) - (t(y) %*% X) +
  t(1 / (1 + exp(-X %*% theta))) %*% X
}

gd_step <- function(theta, alpha, lambda) {
  theta - alpha * df_t(theta, lambda)[1, ]
}

# Run GD for n_iters; show trace, ||theta||, R_emp; print + return final state.
plot_fun <- function(step_fn, lambda, n_iters = 500, label = "") {
  theta <- c(0, 0)
  thetas <- matrix(NA, n_iters, 2)
  norms <- numeric(n_iters)
  fs <- numeric(n_iters)
  for (i in 1:n_iters) {
    theta <- step_fn(theta)      # one GD-style update
    thetas[i, ] <- theta
    norms[i] <- sqrt(sum(theta^2))
    fs[i] <- f(theta, lambda)
  }
  df_trace <- data.frame(thetas, t = 1:n_iters)
  trace_plot <- ggplot(df_trace, aes(x = X1, y = X2, colour = t)) +
    geom_point(size = 0.7) +
    xlab(expression(theta[1])) + ylab(expression(theta[2]))
}
```

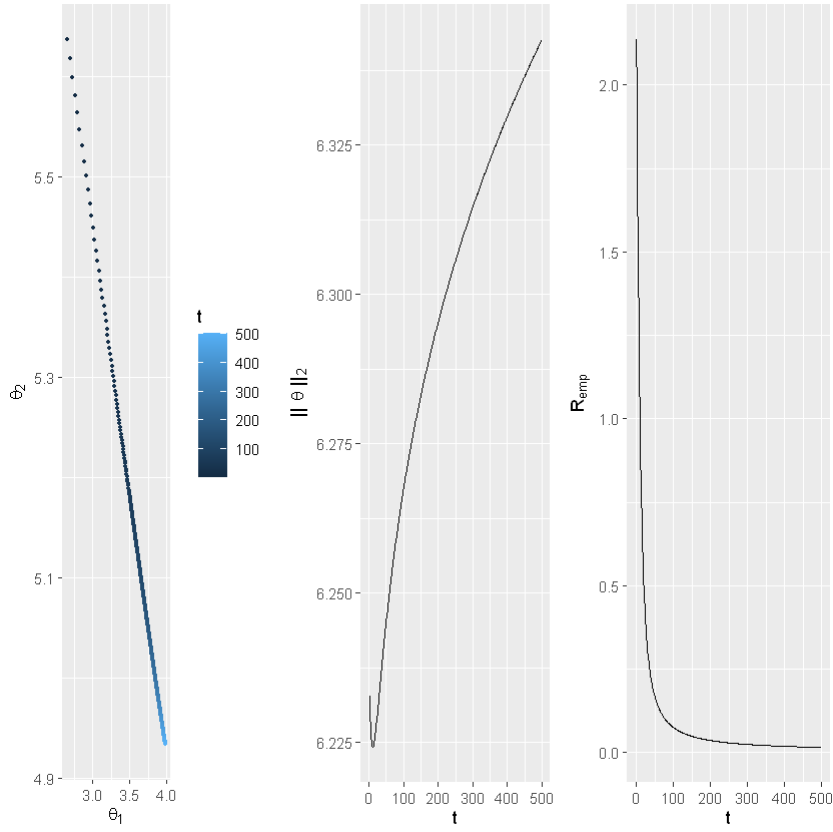
```

norm_plot <- ggplot(data.frame(t = 1:n_iters, n = norms), aes(t, n)) +
  geom_line() + ylab(expression("||" ~ theta ~ "||"[2]))
remp_plot <- ggplot(data.frame(t = 1:n_iters, fs = fs), aes(t, fs)) +
  geom_line() + ylab(expression(R[emp]))
grid.arrange(trace_plot, norm_plot, remp_plot, ncol = 3)
cat(sprintf(paste0(
  "%-22s final theta = (%7.4f, %7.4f), ",
  "||theta|| = %7.3f, R_emp = %.3f\n"),
  label, thetas[n_iters, 1], thetas[n_iters, 2],
  norms[n_iters], fs[n_iters]))
invisible(list(thetas = thetas, norms = norms, fs = fs))
}

## alpha = 0.01, lambda = 0 (vanilla GD, unregularized)
plot_fun(function(theta) gd_step(theta, 0.01, 0), lambda = 0,
  label = "vanilla a=0.01")
## alpha = 0.02, lambda = 0
plot_fun(function(theta) gd_step(theta, 0.02, 0), lambda = 0,
  label = "vanilla a=0.02")

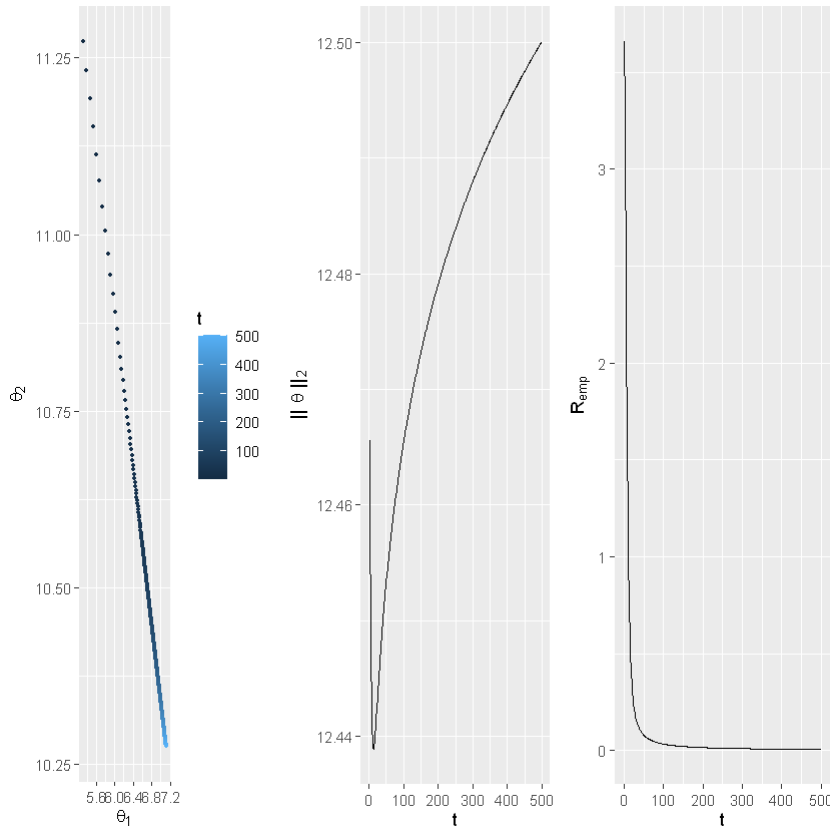
```

vanilla a=0.01 final theta = (3.9867, 4.9332), ||theta|| = 6.343, R_emp = 0.015



vanilla $a=0.02$

final $\theta = (7.1181, 10.2754)$, $\|\theta\|_2 = 12.500$, $R_{\text{emp}} = 0.006$



Python

```
def grad(theta, lam):
    """Gradient of the L2-regularized logistic risk (note 2*lam*theta)."""
    theta = np.asarray(theta, dtype=float)
    sigma = 1 / (1 + np.exp(-(X @ theta)))
    return 2 * lam * theta + (sigma - y) @ X

def gd_step(theta, alpha, lam):
    return theta - alpha * grad(theta, lam)

def plot_fun(step_fn, lam, n_iters=500, theta0=np.zeros(2), label=""):
    """Run GD for n_iters; plot trace, ||theta||, R_emp; print final state."""
    theta = theta0.copy()
```

```

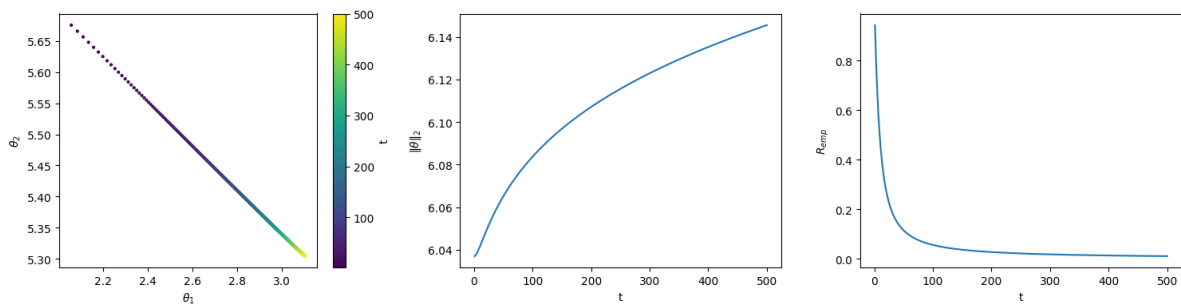
thetas = np.empty((n_iters, 2))
norms = np.empty(n_iters)
fs = np.empty(n_iters)
for i in range(n_iters):
    theta = step_fn(theta)
    thetas[i] = theta
    norms[i] = np.linalg.norm(theta)
    fs[i] = f(theta, lam)

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
sc = axes[0].scatter(thetas[:, 0], thetas[:, 1],
                    c=np.arange(1, n_iters + 1), s=4, cmap="viridis")
axes[0].set_xlabel(r"$\theta_1$"); axes[0].set_ylabel(r"$\theta_2$")
fig.colorbar(sc, ax=axes[0], label="t")
axes[1].plot(np.arange(1, n_iters + 1), norms)
axes[1].set_xlabel("t"); axes[1].set_ylabel(r"$||\theta||_2$")
axes[2].plot(np.arange(1, n_iters + 1), fs)
axes[2].set_xlabel("t"); axes[2].set_ylabel(r"$R_{\text{emp}}$")
plt.tight_layout()
plt.show()

print(f"{label:22} final theta = ({theta[0]:7.4f}, {theta[1]:7.4f}), "
      f"$||\theta||_2$ = {norms[-1]:7.3f}, R_emp = {fs[-1]:.3f}")

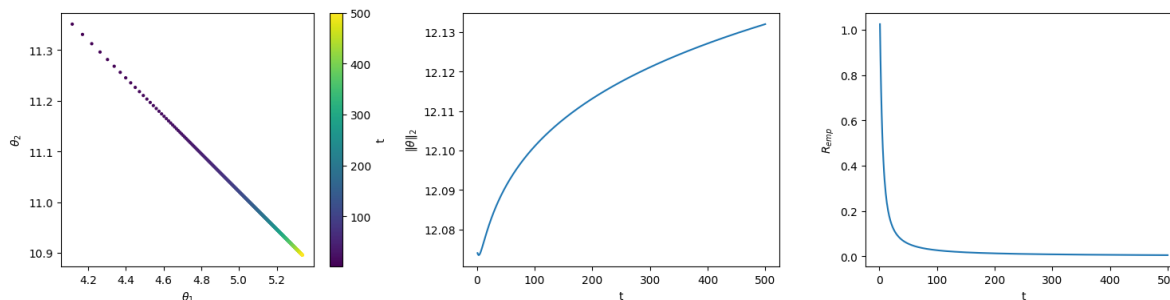
## alpha = 0.01, lambda = 0 (vanilla GD, unregularized)
plot_fun(lambda theta: gd_step(theta, 0.01, 0), lam=0, label="vanilla a=0.01")
## alpha = 0.02, lambda = 0
plot_fun(lambda theta: gd_step(theta, 0.02, 0), lam=0, label="vanilla a=0.02")

```



vanilla a=0.01

final theta = (3.1035, 5.3043), $||\theta||_2$ = 6.146, R_{emp} = 0.011



vanilla $\alpha=0.02$ final $\theta = (5.3362, 10.8954)$, $\|\theta\| = 12.132$, $R_{\text{emp}} = 0.005$

Observation. Both step sizes drive the iterates outward indefinitely while the loss falls monotonically toward zero. The $\|\theta\|$ curves show a *steep early climb* followed by progressively slower growth. The iterates never plateau; they just creep outward more slowly as t grows.

The two step sizes differ mainly in the **terminal norm**: at iteration 500, $\|\theta\|$ at $\alpha = 0.02$ is roughly twice that at $\alpha = 0.01$ — the per-iteration motion scales with α , accumulated over a fixed number of steps.

Theoretically GD does not converge here because $\mathcal{R}_{\text{emp}}$ has no minimum (a). Empirically: the $\|\theta\|$ curve never plateaus, even when its rate of growth becomes very slow.

(e) Add L2 regularization

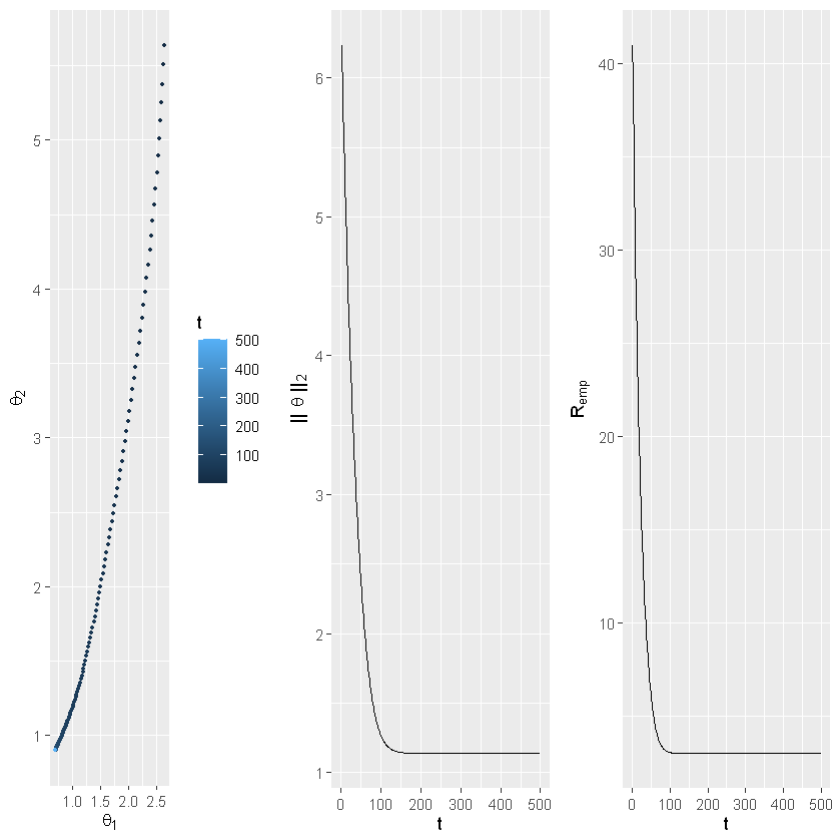
Repeat (d), but minimize $\lambda\|\theta\|^2 + \mathcal{R}_{\text{emp}}(\theta)$ with $\lambda = 1$. What changes?

Solution.

R

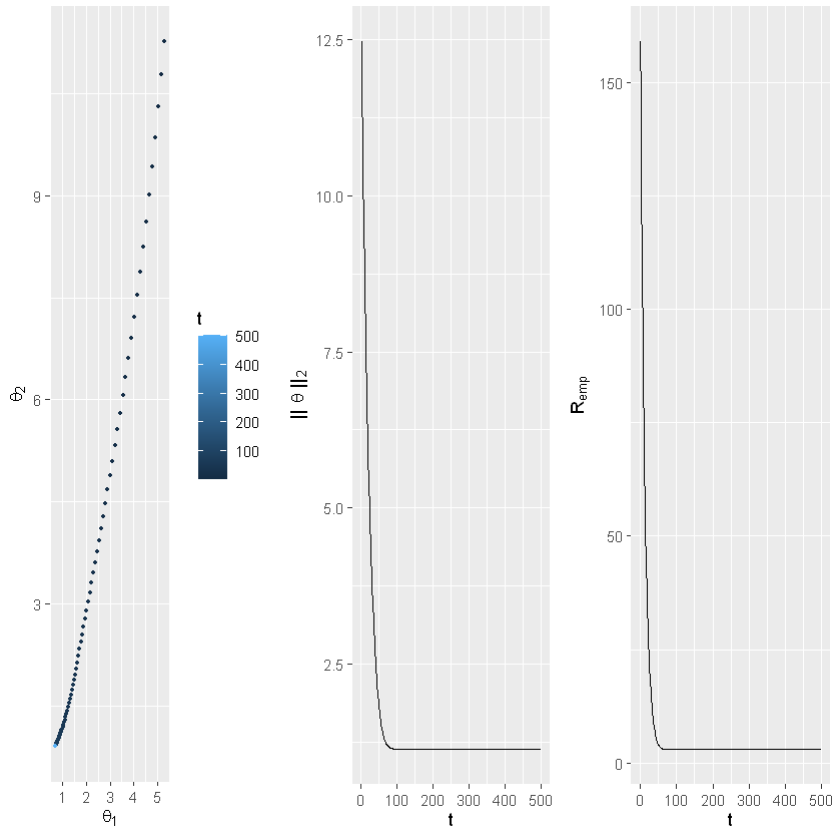
```
## lambda = 1, alpha = 0.01
plot_fun(function(theta) gd_step(theta, 0.01, 1), lambda = 1,
         label = "reg a=0.01")
## lambda = 1, alpha = 0.02
plot_fun(function(theta) gd_step(theta, 0.02, 1), lambda = 1,
         label = "reg a=0.02")
```

reg $\alpha=0.01$ final $\theta = (0.6985, 0.8986)$, $\|\theta\| = 1.138$, $R_{\text{emp}} = 2.985$



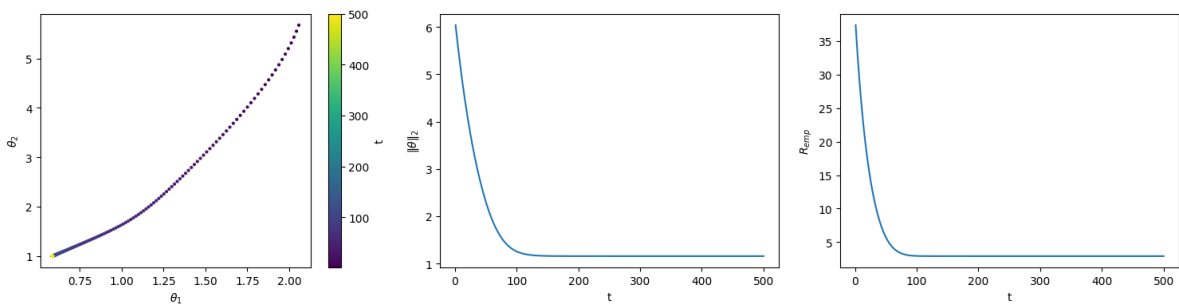
reg a=0.02

final theta = (0.6985, 0.8986), $\|\theta\|_2 = 1.138$, $R_{\text{emp}} = 2.985$

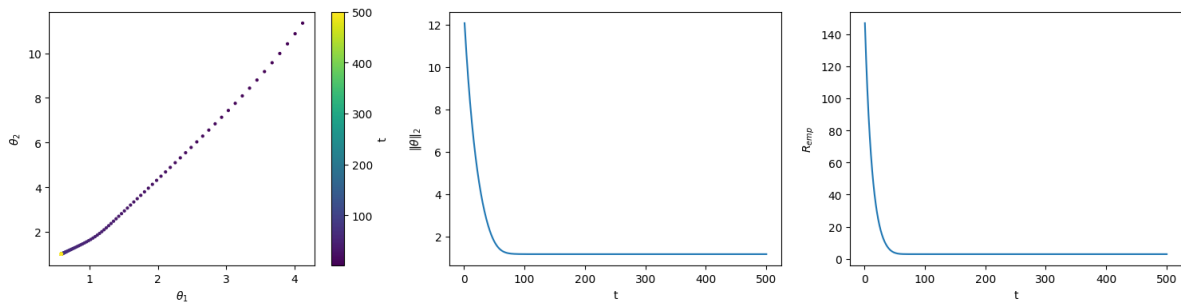


Python

```
## lambda = 1, alpha = 0.01
plot_fun(lambda theta: gd_step(theta, 0.01, 1), lam=1, label="reg a=0.01 ")
## lambda = 1, alpha = 0.02
plot_fun(lambda theta: gd_step(theta, 0.02, 1), lam=1, label="reg a=0.02 ")
```



reg a=0.01 final theta = (0.5860, 0.9966), ||theta|| = 1.156, R_emp = 2.907



reg a=0.02 final theta = (0.5860, 0.9966), ||theta|| = 1.156, R_emp = 2.907

What changes. The regularizer $\lambda\|\theta\|^2$ gives the objective a finite minimum. Instead of the iterates growing without bound as in (d), the $\|\theta\|$ curves now rise and then settle to a plateau — the signature of convergence — and both step sizes converge to the same minimum.

(f) Visualize the regularized empirical risk

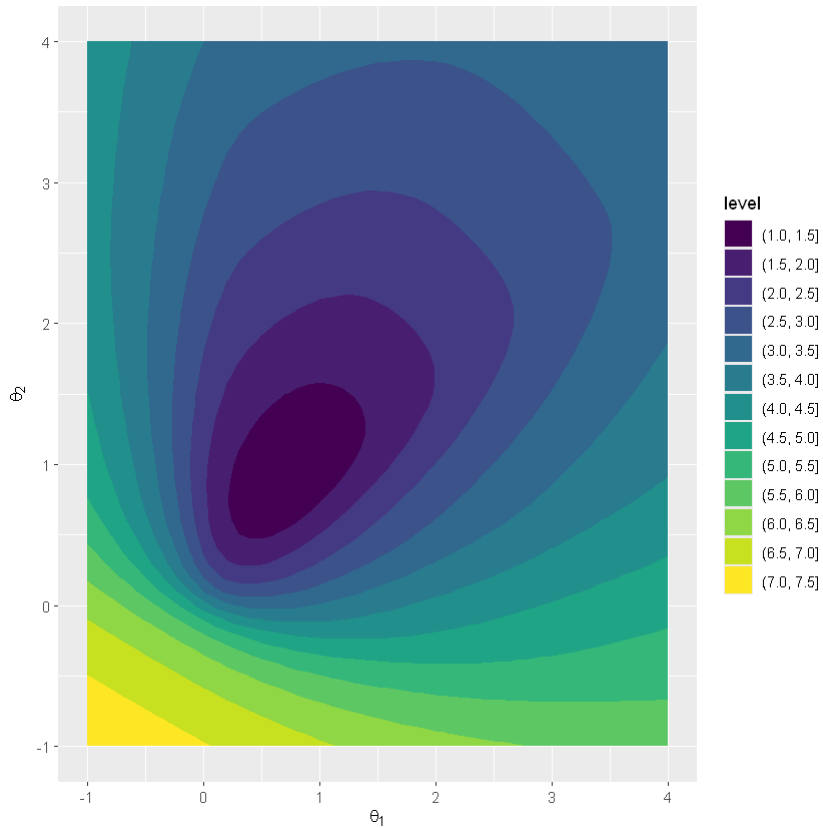
Visualize the regularized empirical risk $\lambda\|\theta\|^2 + \mathcal{R}_{\text{emp}}(\theta)$ on $[-1, 4] \times [-1, 4]$ with $\lambda = 1$.

Solution.

R

```
fxx_reg <- log(apply(xx, 1, function(t) f(t, 1)))
df_grid_reg <- data.frame(xx, fxx = fxx_reg)

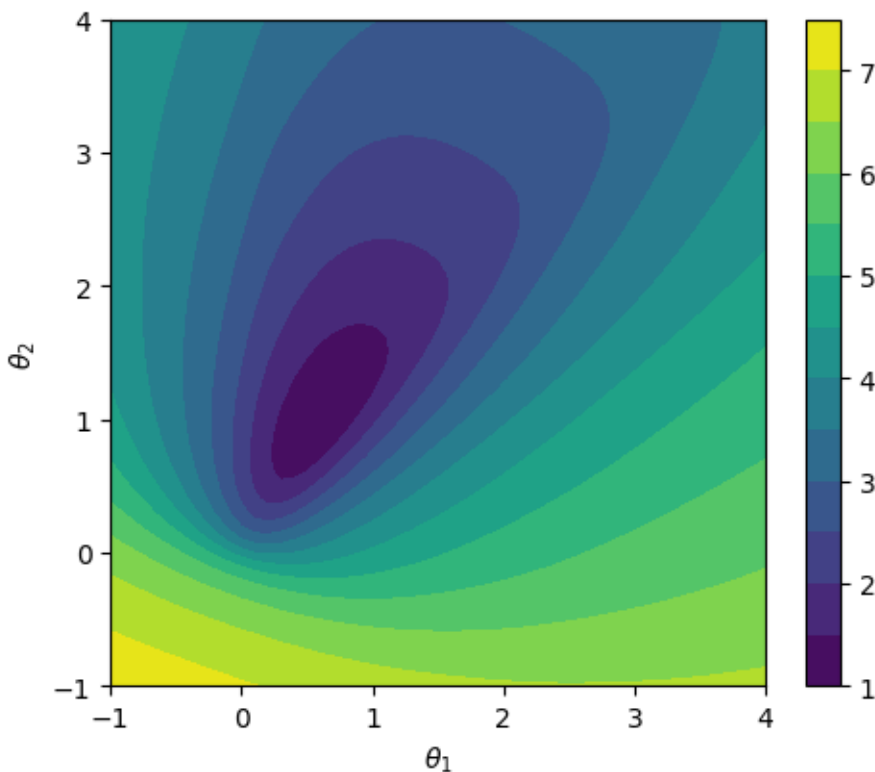
ggplot(df_grid_reg, aes(x = X1, y = X2, z = fxx)) +
  geom_contour_filled() +
  xlab(expression(theta[1])) +
  ylab(expression(theta[2]))
```



Python

```
Z_reg = np.array([[np.log(f(np.array([t1, t2])), 1)) for t1 in grid]
                  for t2 in grid])

fig, ax = plt.subplots(figsize=(5.5, 4.5))
cs = ax.contourf(T1, T2, Z_reg, levels=14)
ax.set_xlabel(r"$\theta_1$"); ax.set_ylabel(r"$\theta_2$")
fig.colorbar(cs)
plt.show()
```



Compare to (b): the regularized landscape has closed level sets around a finite minimum — a **bowl**, not a slide. This is what makes GD converge in (e). The minimum’s location is biased toward the origin compared to the unregularized maximum-likelihood direction, by an amount depending on λ .

(g) Gradient descent with backtracking line search

Repeat (e) but with **backtracking line search**: at each step, propose $\theta_{\text{prop}} = \theta - \alpha \nabla f(\theta)$ and accept it if it satisfies the Armijo condition

$$f(\theta_{\text{prop}}) \leq f(\theta) - \gamma \alpha \|\nabla f(\theta)\|^2,$$

otherwise shrink $\alpha \leftarrow \tau \alpha$ and try again. Use $\gamma = 0.9$, $\tau = 0.5$.

Note: the trace below visualizes from iteration $t = 1$ (after one backtracking step), not from the starting point $\theta^{[0]} = (0, 0)^\top$.

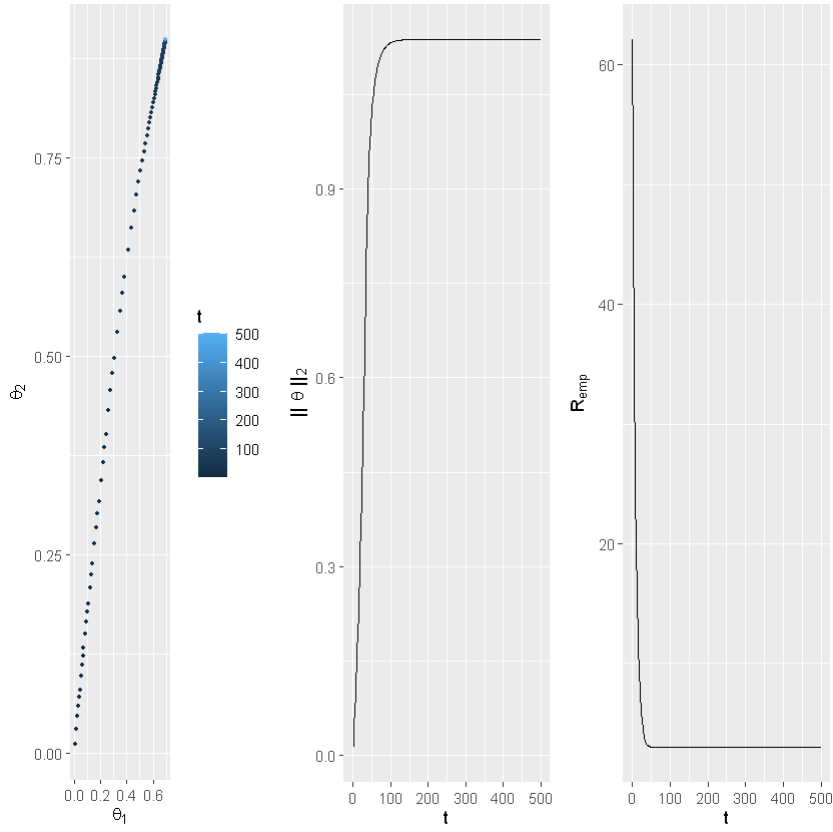
Solution.

R

```
gd_backtracking_step <- function(theta, alpha, gamma, tau, lambda) {
  f_theta <- f(theta, lambda)
  d_theta <- df_t(theta, lambda)[1, ]
  for (i in 1:1000) {
    theta_prop <- theta - alpha * d_theta
    if (f(theta_prop, lambda) <= f_theta - gamma * alpha * sum(d_theta^2)) {
      return(theta_prop)
    }
    alpha <- tau * alpha
  }
  stop("backtracking failed to find a descent step after 1000 shrinks; ",
       "check curvature / starting point / gamma=", gamma)
}

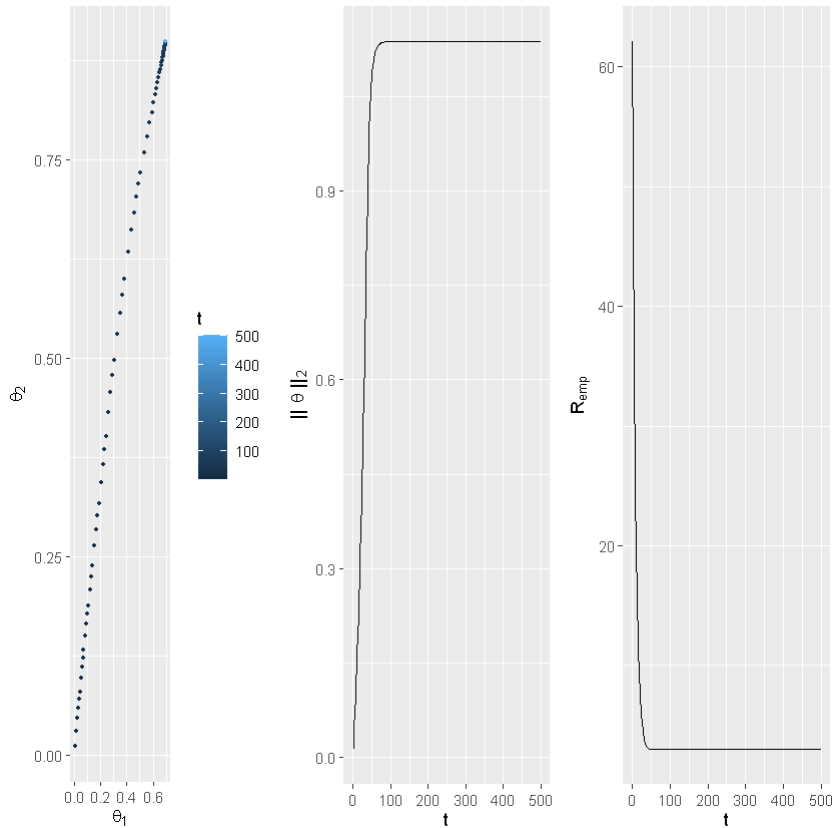
## lambda = 1, alpha_init = 0.01
result_bt <- plot_fun(
  function(theta) gd_backtracking_step(theta, 0.01, 0.9, 0.5, 1),
  lambda = 1, label = "backtrack a=0.01")
## lambda = 1, alpha_init = 0.02
plot_fun(
  function(theta) gd_backtracking_step(theta, 0.02, 0.9, 0.5, 1),
  lambda = 1, label = "backtrack a=0.02")
```

backtrack a=0.01 final theta = (0.6985, 0.8986), ||theta|| = 1.138, R_emp = 2.985



backtrack a=0.02

final theta = (0.6985, 0.8986), $\| \theta \| = 1.138$, $R_{\text{emp}} = 2.985$



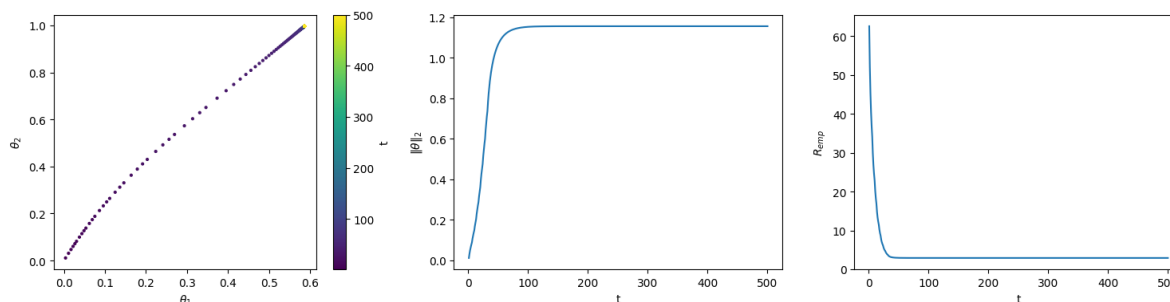
Python

```
def gd_backtracking_step(theta, alpha, gamma, tau, lam, max_iter=1000):
    """Single GD step with Armijo backtracking. Shrinks alpha until accepted."""
    f_theta = f(theta, lam)
    d_theta = grad(theta, lam)
    grad_sq = d_theta @ d_theta
    for _ in range(max_iter):
        theta_prop = theta - alpha * d_theta
        if f(theta_prop, lam) <= f_theta - gamma * alpha * grad_sq:
            return theta_prop
        alpha *= tau
    raise RuntimeError(
        f"backtracking failed to find a descent step after {max_iter} shrinks; "
        f"check curvature / starting point / gamma={gamma}")
```

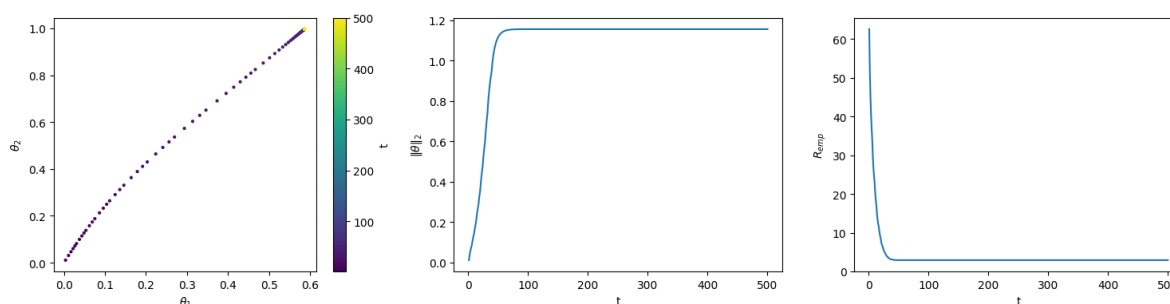
```

## lambda = 1, alpha_init = 0.01
plot_fun(
    lambda theta: gd_backtracking_step(theta, 0.01, 0.9, 0.5, 1),
    lam=1, label="backtrack a=0.01")
## lambda = 1, alpha_init = 0.02
plot_fun(
    lambda theta: gd_backtracking_step(theta, 0.02, 0.9, 0.5, 1),
    lam=1, label="backtrack a=0.02")

```



backtrack a=0.01 final theta = (0.5860, 0.9966), $\|\text{theta}\| = 1.156$, $R_{\text{emp}} = 2.907$



backtrack a=0.02 final theta = (0.5860, 0.9966), $\|\text{theta}\| = 1.156$, $R_{\text{emp}} = 2.907$

Observation. Backtracking adapts α per iteration: the step is automatically shortened when a proposed update would not produce a sufficient decrease. The loss curve is monotone non-increasing by construction (Armijo condition), and both initial α values converge to the same minimum found in (e).

Where this leaves us. This exercise traces a ladder of fixes:

- Vanilla GD on separable data $\rightarrow$ diverges (a, d).

- L2 regularization $\rightarrow$ finite minimum (e).
- Backtracking $\rightarrow$ adaptive step that does not depend on the user picking the right α up front (g).

The next stage is **direction**: even with a well-chosen step, the local-gradient direction is suboptimal on poorly-conditioned problems — when the level sets are elongated, the negative gradient points mostly across the valley rather than along it, so the iterates zig-zag and progress slowly. Exercise 7 introduces *momentum* to break that limitation.

Exercise 7: Multivariate Optimization 2

Optimization in Machine Learning

Reproducibility

Produced with Python 3.10 and R 4.4.1. To reproduce, install the pinned package versions below.

Python

```
pip install numpy==2.0.2 matplotlib==3.10.8
```

R

```
install.packages("remotes")  
remotes::install_version("ggplot2", "3.5.1")
```

Setup

A radial basis function (RBF) network has been fitted to a black-box function, giving the model

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}, \mathbf{x} \mapsto \sum_{i=1}^2 w_i \cdot \rho(\|\mathbf{x} - \mathbf{c}_i\|_{S_i})$$

with $\mathbf{c}_1 = (-1.1, 1.1)^\top$, $\mathbf{c}_2 = (0.8, -0.8)^\top$, weights $w_1 = 1$, $w_2 = -1$, **biweight (quartic) kernel**

$$\rho : \mathbb{R} \rightarrow \mathbb{R}, u \mapsto \begin{cases} (1 - u^2)^2 & |u| < 1 \\ 0 & \text{otherwise} \end{cases}$$

and Mahalanobis distances $\|\cdot\|_{S_i}$ with covariance matrices $S_1 = \mathbf{I}$, $S_2 = \begin{pmatrix} 1.1 & -0.9 \\ -0.9 & 1.1 \end{pmatrix}$. The Mahalanobis distance is $\|\mathbf{x} - \mathbf{c}\|_S = \sqrt{(\mathbf{x} - \mathbf{c})^\top S^{-1}(\mathbf{x} - \mathbf{c})}$.

Note: We chose this kernel and distance for educational purposes; in practice, Gaussian kernels and Euclidean distances are more common.

(a) Surface plot

Plot f on the range $[-2, 2] \times [-2, 2]$.

Solution.

R

```
library(ggplot2)

c1 <- c(-1.1, 1.1)
c2 <- c(0.8, -0.8)
S2 <- matrix(c(1.1, -0.9, -0.9, 1.1), nrow = 2)
S2_inv <- solve(S2)

rho <- function(u) ifelse(abs(u) < 1, (1 - u^2)^2, 0)
dist1 <- function(x) sqrt(sum((x - c1)^2)) # Euclidean (S1 = I)
dist2 <- function(x) sqrt((x - c2) %*% S2_inv %*% (x - c2)) # Mahalanobis

# w_1 = +1, w_2 = -1
f <- function(x) rho(dist1(x)) - rho(dist2(x))

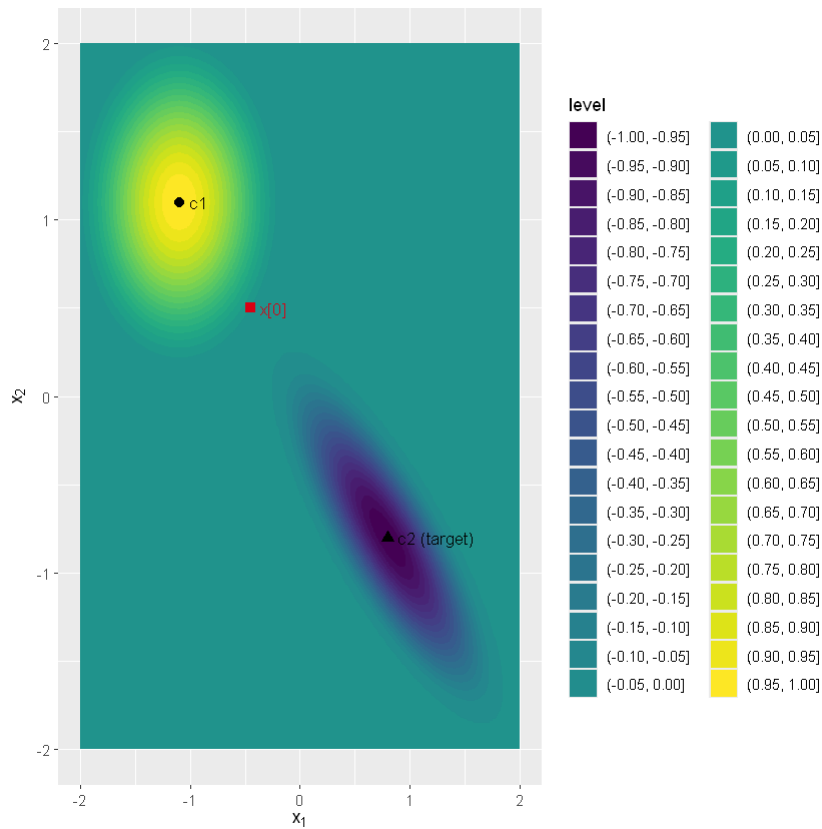
# Surface plot on [-2, 2] x [-2, 2]
x <- seq(-2, 2, by = 0.01)
xx <- expand.grid(X1 = x, X2 = x)
fxx <- apply(xx, 1, f)
df_grid <- data.frame(xx, fxx = fxx)

cont_plot <- ggplot(df_grid, aes(x = X1, y = X2, z = fxx)) +
  geom_contour_filled(binwidth = 0.05) +
  xlab(expression(x[1])) +
  ylab(expression(x[2])) +
  # landmarks: positive-bump center, negative-bump center (target), start point
  annotate("point", x = c1[1], y = c1[2], color = "black", size = 2.5) +
  annotate("text", x = c1[1], y = c1[2], label = " c1", hjust = 0,
```

```

        color = "black", size = 3.5) +
    annotate("point", x = c2[1], y = c2[2], color = "black",
           size = 2.5, shape = 17) +
    annotate("text", x = c2[1], y = c2[2], label = "  c2 (target)", hjust = 0,
           color = "black", size = 3.5) +
    annotate("point", x = -0.45, y = 0.5, color = "#D90012",
           size = 2.5, shape = 15) +
    annotate("text", x = -0.45, y = 0.5, label = "  x[0]", hjust = 0,
           color = "#D90012", size = 3.5)
cont_plot

```



Python

```

import numpy as np
import matplotlib.pyplot as plt

```

```

c1 = np.array([-1.1, 1.1])
c2 = np.array([0.8, -0.8])
S2 = np.array([[1.1, -0.9], [-0.9, 1.1]])
S2_inv = np.linalg.inv(S2)

def rho(u):
    """Biweight (quartic) kernel:  $(1 - u^2)^2$  for  $|u| < 1$ , else 0."""
    return (1 - u ** 2) ** 2 if abs(u) < 1 else 0.0

def dist1(x):
    return float(np.sqrt((x - c1) @ (x - c1))) # Euclidean (S1 = I)

def dist2(x):
    return float(np.sqrt((x - c2) @ S2_inv @ (x - c2))) # Mahalanobis

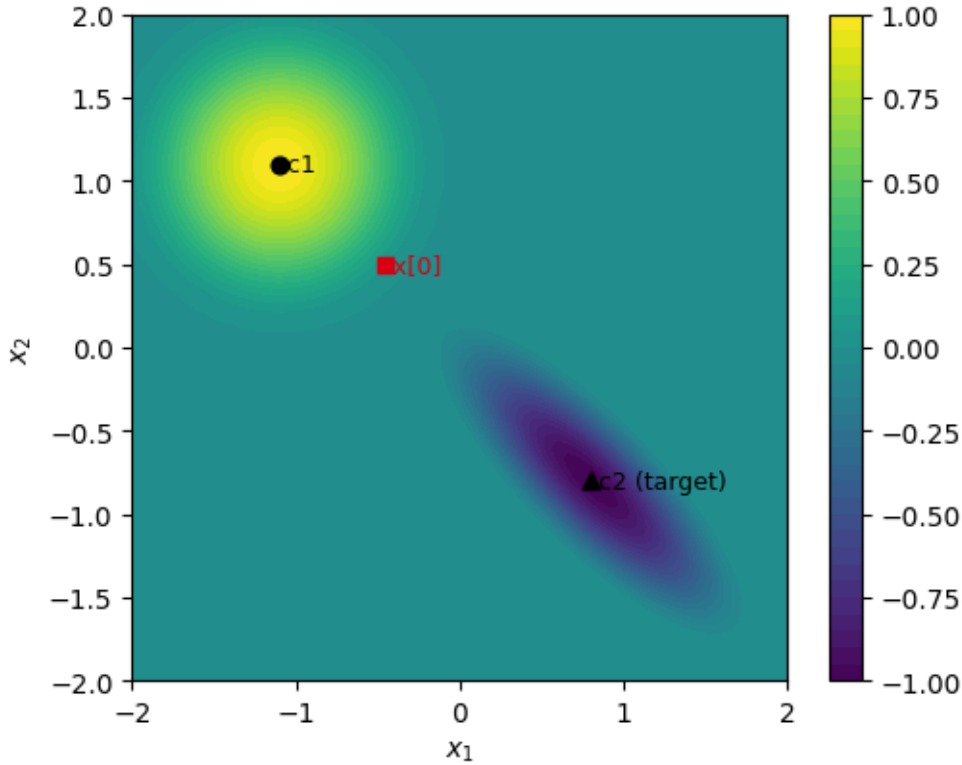
def f(x):
    """w_1 = +1, w_2 = -1."""
    x = np.asarray(x, dtype=float)
    return rho(dist1(x)) - rho(dist2(x))

# Surface plot on [-2, 2] x [-2, 2]
grid = np.arange(-2, 2 + 0.01, 0.01)
T1, T2 = np.meshgrid(grid, grid)
Z = np.array([[f(np.array([t1, t2])) for t1 in grid] for t2 in grid])

def plot_landscape(ax):
    """Render the contour with c_1, c_2, x[0] landmarks (reused by d, f)."""
    cs = ax.contourf(T1, T2, Z, levels=np.arange(-1.0, 1.0 + 0.05, 0.05))
    ax.scatter(*c1, color="black", s=40, marker="o", zorder=3)
    ax.annotate(" c1", c1, color="black", fontsize=9, va="center")
    ax.scatter(*c2, color="black", s=40, marker="^", zorder=3)
    ax.annotate(" c2 (target)", c2, color="black", fontsize=9, va="center")
    ax.scatter(-0.45, 0.5, color="#D90012", s=40, marker="s", zorder=3)
    ax.annotate(" x[0]", (-0.45, 0.5), color="#D90012", fontsize=9, va="center")
    ax.set_xlabel(r"$x_1$"); ax.set_ylabel(r"$x_2$")
    return cs

```

```
fig, ax = plt.subplots(figsize=(5.5, 4.5))
cs = plot_landscape(ax)
fig.colorbar(cs)
plt.show()
```



The plot shows two basins: a positive bump at $\mathbf{c}_1$ (where $f = +1$) and a negative dip at $\mathbf{c}_2$ (where $f = -1$). The starting point $\mathbf{x}^{[0]} = (-0.45, 0.5)$ lies inside $\mathbf{c}_1$'s support.

Outside both supports the function is identically zero — the **dead zone** (the uniform color filling most of the plot). This flat region is why vanilla GD gets stuck in (d): a constant function has zero gradient, so any step landing in the dead zone cannot move.

(b) Disjoint supports

Show that $\bigcap_{i=1}^2 \{\mathbf{x} \in \mathbb{R}^2 \mid \rho(\|\mathbf{x} - \mathbf{c}_i\|_{S_i}) \neq 0\} = \emptyset$.

Solution.

First we analyze $\rho(u)$ for $|u| < 1$:

$$(1 - u^2)^2 = 0 \iff (1 - u^2) = 0 \iff u^2 = 1,$$

so $\rho(u) \neq 0$ for $u^2 < 1$ and $\rho(u) = 0$ for $u^2 \geq 1$. We can check this condition for both squared distances around the centers $\mathbf{c}_1, \mathbf{c}_2$:

1. $\|\mathbf{x} - \mathbf{c}_1\|_{S_1}^2 < 1 \iff \|\mathbf{x} - \mathbf{c}_1\|_2^2 < 1$ (unit circle around $\mathbf{c}_1$).
2. $\|\mathbf{x} - \mathbf{c}_2\|_{S_2}^2 = (\mathbf{x} - \mathbf{c}_2)^\top \begin{pmatrix} 1.1 & -0.9 \\ -0.9 & 1.1 \end{pmatrix}^{-1} (\mathbf{x} - \mathbf{c}_2) < 1$ (ellipse around $\mathbf{c}_2$).

To find the smallest enclosing circle of the ellipse, use the eigendecomposition of S_2 :

$$\begin{aligned} \det(S_2 - \lambda \mathbf{I}) = 0 &\iff \det \begin{pmatrix} 1.1 - \lambda & -0.9 \\ -0.9 & 1.1 - \lambda \end{pmatrix} = 0 \\ &\iff \lambda^2 - 2.2\lambda + 0.4 = 0 \iff \lambda_1 = 2.0, \lambda_2 = 0.2. \end{aligned}$$

The eigenvalues of S_2^{-1} are $1/\lambda_1 = 0.5$ and $1/\lambda_2 = 5$. Rotating into the eigenbasis $\mathbf{y} = \mathbf{V}(\mathbf{x} - \mathbf{c}_2)$ (with $|\det \mathbf{V}| = 1$) diagonalizes the quadratic form:

$$\|\mathbf{x} - \mathbf{c}_2\|_{S_2}^2 < 1 \iff 5y_1^2 + 0.5y_2^2 < 1.$$

This is an ellipse with semi-axes $1/\sqrt{5} \approx 0.45$ and $1/\sqrt{0.5} = \sqrt{2}$ (set one $y_j = 0$ and solve, e.g. $5y_1^2 = 1 \Rightarrow y_1 = 1/\sqrt{5}$). A circle centered at $\mathbf{c}_2$ enclosing it must reach the **larger** semi-axis, so radius $\sqrt{2}$ suffices.

Comparing the distance between centers with the sum of enclosing radii:

$$\|\mathbf{c}_2 - \mathbf{c}_1\|_2 = \sqrt{2 \cdot 1.9^2} \approx 2.69 > 1 + \sqrt{2} \approx 2.41,$$

so the circles cannot intersect, hence neither can the supports. Only $\rho(\|\mathbf{x} - \mathbf{c}_1\|_{S_1})$ or $\rho(\|\mathbf{x} - \mathbf{c}_2\|_{S_2})$ can be non-zero for a given $\mathbf{x} \in \mathbb{R}^2$.

(c) Global minimum, analytically

Find the global minimum of f analytically.

Hint: recall (b).

Solution.

By (b), the two RBF terms are independent — at any $\mathbf{x}$ at most one is nonzero. Combined with $\rho \geq 0$, $w_1 = +1$, and $w_2 = -1$:

- inside $\mathbf{c}_1$'s support: $f = +\rho(\cdot) \geq 0$
- inside $\mathbf{c}_2$'s support: $f = -\rho(\cdot) \leq 0$
- elsewhere: $f = 0$.

So the global minimum lives in $\mathbf{c}_2$'s support. Setting $\nabla_{\mathbf{x}}\rho(\|\mathbf{x} - \mathbf{c}_2\|_{S_2}) = \mathbf{0}$:

$$\nabla_{\mathbf{x}}\rho(\|\mathbf{x} - \mathbf{c}_2\|_{S_2}) = 2(1 - \|\mathbf{x} - \mathbf{c}_2\|_{S_2}^2) \cdot (-2)(\mathbf{x} - \mathbf{c}_2)^\top S_2^{-1} = \mathbf{0}.$$

Either $\|\mathbf{x} - \mathbf{c}_2\|_{S_2} = 1$ (the boundary, where $\rho = 0$) or $\mathbf{x} = \mathbf{c}_2$. Since $-\rho(\mathbf{0}) = -1 < 0 = -\rho(1)$, the global minimum is at $\mathbf{x} = \mathbf{c}_2$ with value $f(\mathbf{c}_2) = -1$.

(d) Two GD steps numerically

Write an R/Python script that performs two gradient descent steps starting at $\mathbf{x}^{[0]} = (-0.45, 0.5)^\top$ with step size $\alpha = 0.15$. What do you observe?

Solution.

R

```
# Gradient: leverages (b) - at any x at most ONE basin's support is active.
grad <- function(x) {
  d1_sq <- sum((x - c1)^2)
  d2_sq <- as.numeric((x - c2) %*% S2_inv %*% (x - c2))
  if (d1_sq < 1) {
    # Inside c_1 support: grad of rho(||x - c_1||)
    return(as.numeric(-4 * (1 - d1_sq) * (x - c1)))
  } else if (d2_sq < 1) {
    # Inside c_2 support; sign flipped because w_2 = -1
    return(as.numeric(4 * (1 - d2_sq) * (x - c2) %*% S2_inv))
  } else {
    return(c(0, 0)) # dead zone: gradient identically zero
  }
}
```

```

    }
}

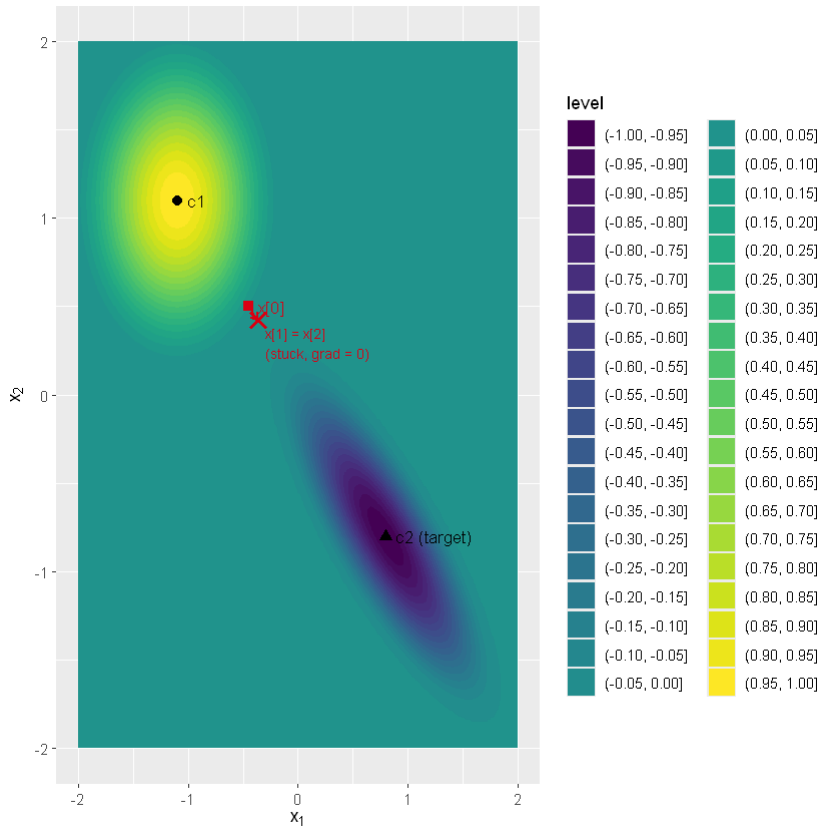
alpha <- 0.15
x0 <- c(-0.45, 0.5)
x1 <- x0 - alpha * grad(x0)
x2 <- x1 - alpha * grad(x1)

cat(sprintf("x[0] = (%+.4f, %+.4f)\n", x0[1], x0[2]))
cat(sprintf("x[1] = (%+.4f, %+.4f)\n", x1[1], x1[2]))
cat(sprintf("x[2] = (%+.4f, %+.4f)\n", x2[1], x2[2]))
cat(sprintf("grad at x[1] = (%+.4f, %+.4f) -- identically zero -> stuck\n",
            grad(x1)[1], grad(x1)[2]))

# Visualize: arrow x[0] -> x[1], then mark the frozen point x[1] = x[2].
cont_plot +
  annotate("segment", x = x0[1], y = x0[2], xend = x1[1], yend = x1[2],
          arrow = arrow(length = unit(0.18, "cm")),
          color = "#D90012", linewidth = 0.7) +
  annotate("point", x = x1[1], y = x1[2], color = "#D90012",
          size = 3, shape = 4, stroke = 1.5) +
  annotate("text", x = x1[1], y = x1[2],
          label = " x[1] = x[2]\n (stuck, grad = 0)",
          hjust = 0, vjust = 1.2, color = "#D90012", size = 3)

x[0] = (-0.4500, +0.5000)
x[1] = (-0.3652, +0.4217)
x[2] = (-0.3652, +0.4217)
grad at x[1] = (+0.0000, +0.0000) -- identically zero -> stuck

```



Python

```
def grad(x):
    """Gradient of f. By (b), at most one basin's support is active at any x."""
    x = np.asarray(x, dtype=float)
    d1_sq = (x - c1) @ (x - c1)
    d2_sq = (x - c2) @ S2_inv @ (x - c2)
    if d1_sq < 1:
        # Inside c_1 support: gradient of +rho(||x - c_1||)
        return -4 * (1 - d1_sq) * (x - c1)
    elif d2_sq < 1:
        # Inside c_2 support: gradient of -rho(||x - c_2||_S2); sign flipped
        return 4 * (1 - d2_sq) * (S2_inv @ (x - c2))
    else:
        return np.zeros(2) # dead zone: gradient identically zero
```

```

alpha = 0.15
x0 = np.array([-0.45, 0.5])
x1 = x0 - alpha * grad(x0)
x2 = x1 - alpha * grad(x1)

print(f"x[0] = ({x0[0]:+.4f}, {x0[1]:+.4f})")
print(f"x[1] = ({x1[0]:+.4f}, {x1[1]:+.4f})")
print(f"x[2] = ({x2[0]:+.4f}, {x2[1]:+.4f})")
g1 = grad(x1)
print(f"grad at x[1] = ({g1[0]:+.4f}, {g1[1]:+.4f})  "
      "-- identically zero -> stuck")

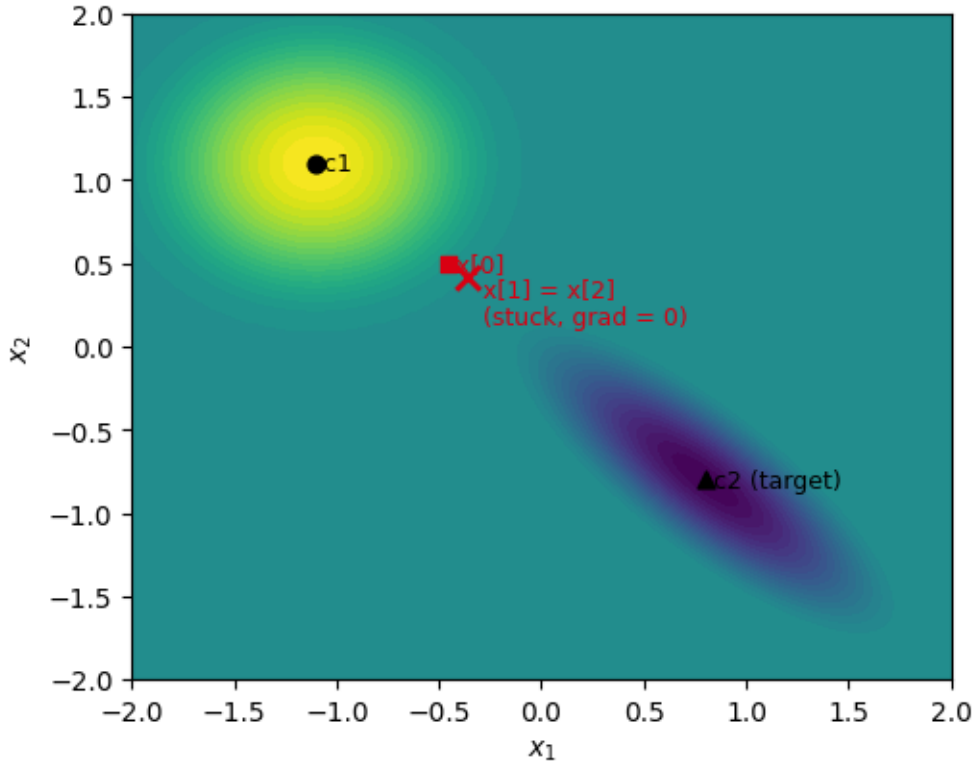
# Visualize the stuck-iterate situation.
fig, ax = plt.subplots(figsize=(5.5, 4.5))
plot_landscape(ax)
ax.annotate("", xy=x1, xytext=x0,
            arrowprops=dict(arrowstyle="->", color="#D90012", lw=1.2))
ax.scatter(*x1, color="#D90012", s=80, marker="x", linewidths=2, zorder=4)
ax.annotate("  x[1] = x[2]\n (stuck, grad = 0)", x1, color="#D90012",
            fontsize=9, va="top")
plt.show()

```

```

x[0] = (-0.4500, +0.5000)
x[1] = (-0.3652, +0.4217)
x[2] = (-0.3652, +0.4217)
grad at x[1] = (+0.0000, +0.0000)  -- identically zero -> stuck

```



Observation. After the first step the iterate $\mathbf{x}^{[1]}$ lies in the **dead zone** between the two basins. The gradient there is identically zero, so the second step cannot move: $\mathbf{x}^{[2]} = \mathbf{x}^{[1]}$ exactly.

(e) Two GD steps analytically

Confirm (d) with paper-and-pencil arithmetic. Same $\mathbf{x}^{[0]} = (-0.45, 0.5)^\top$, $\alpha = 0.15$.

Solution.

Start with $\mathbf{x}^{[0]} = (-0.45, 0.5)^\top$. Then

$$\|\mathbf{c}_1 - \mathbf{x}^{[0]}\|_2^2 = (-0.65)^2 + (0.6)^2 = 0.4225 + 0.36 = 0.7825 < 1,$$

so $\mathbf{x}^{[0]}$ is inside $\mathbf{c}_1$'s support. The gradient of $f = +\rho(\|\mathbf{x} - \mathbf{c}_1\|)$ here is

$$\nabla f(\mathbf{x}^{[0]}) = -4(1 - \|\mathbf{x}^{[0]} - \mathbf{c}_1\|_2^2) (\mathbf{x}^{[0]} - \mathbf{c}_1) = -4 \cdot 0.2175 \cdot (0.65, -0.6) \approx (-0.5655, 0.5220).$$

Step:

$$\mathbf{x}^{[1]} = \mathbf{x}^{[0]} - 0.15 \cdot \nabla f(\mathbf{x}^{[0]}) = (-0.3652, 0.4217)^\top.$$

Check supports at $\mathbf{x}^{[1]}$:

$$\|\mathbf{c}_1 - \mathbf{x}^{[1]}\|_2^2 \approx 1.0001 > 1, \quad \|\mathbf{c}_2 - \mathbf{x}^{[1]}\|_{S_2}^2 \approx 1.4323 > 1.$$

Both supports missed — $\mathbf{x}^{[1]}$ is in the dead zone, $\nabla f(\mathbf{x}^{[1]}) = \mathbf{0}$, so $\mathbf{x}^{[2]} = \mathbf{x}^{[1]}$.

(f) Gradient descent with momentum

Repeat (d) but with **gradient descent with momentum**. Use velocity initialization $\mathbf{v}^{[0]} = (0.4, -0.4)^\top$, momentum coefficient $\varphi = 0.5$, and run for 15 iterations with step size $\alpha = 0.15$. Find the global minimum.

Why momentum works here. Vanilla GD updates: $\mathbf{x}^{[t+1]} = \mathbf{x}^{[t]} - \alpha \nabla f(\mathbf{x}^{[t]})$. When $\nabla f = \mathbf{0}$ the iterate is frozen.

Momentum adds a velocity that persists across iterations:

$$\mathbf{v}^{[t+1]} = \varphi \mathbf{v}^{[t]} - \alpha \nabla f(\mathbf{x}^{[t]}), \quad \mathbf{x}^{[t+1]} = \mathbf{x}^{[t]} + \mathbf{v}^{[t+1]}.$$

When $\nabla f = \mathbf{0}$, the velocity decays geometrically ($\mathbf{v}^{[t+1]} = \varphi \mathbf{v}^{[t]}$) but does not vanish. With $\varphi = 0.5$ the iterate keeps moving across the dead zone, eventually landing inside $\mathbf{c}_2$'s support — at which point the gradient becomes nonzero and pulls the iterate toward $\mathbf{c}_2$, the true minimum.

Physical analogy. Think of a marble rolling on the surface f . Vanilla GD = marble with no inertia: it stops the instant the floor goes flat. Momentum GD = marble with mass: it carries through flat sections by virtue of its accumulated velocity, and only stops when actually trapped in a basin. The momentum coefficient $\varphi \in (0, 1)$ is friction — closer to 1 = less friction = more inertia.

R

```

alpha <- 0.15
phi <- 0.5
v <- c(0.4, -0.4)
x <- c(-0.45, 0.5)

xs <- x # stack iterates for the trajectory plot
for (i in 1:15) {
  v <- phi * v - alpha * grad(x)
  x <- x + v
  xs <- rbind(xs, x)
}
rownames(xs) <- NULL
trace_df <- data.frame(xs)

cat(sprintf(paste0(
  "x[15] = (%+.4f, %+.4f), f = %+.4f  ",
  "(target c_2 = (%+.4f, %+.4f), f = -1)\n"),
  x[1], x[2], f(x), c2[1], c2[2]))

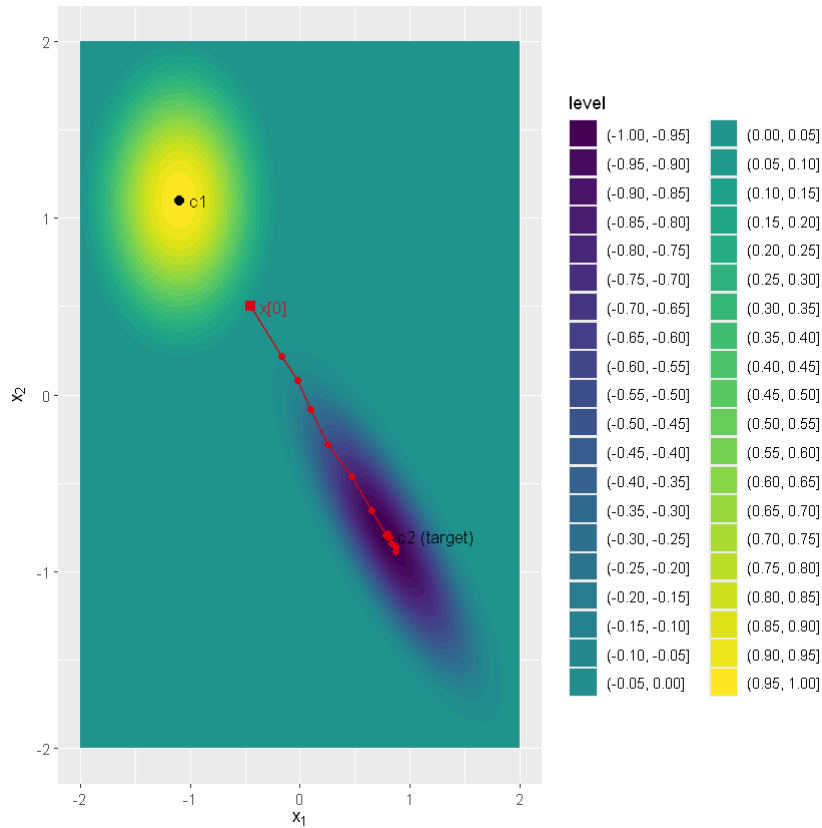
# Trajectory on the contour. cont_plot already has c1, c2, x[0] markers
# from (a); we just overlay the momentum path.
cont_plot +
  geom_path(data = trace_df, aes(x = X1, y = X2),
            color = "#D90012", linewidth = 0.6, inherit.aes = FALSE) +
  geom_point(data = trace_df, aes(x = X1, y = X2),
             color = "#D90012", size = 1.5, inherit.aes = FALSE)

```

```

x[15] = (+0.7831, -0.7951), f = -0.9990 (target c_2 = (+0.8000, -0.8000), f = -1)

```



Python

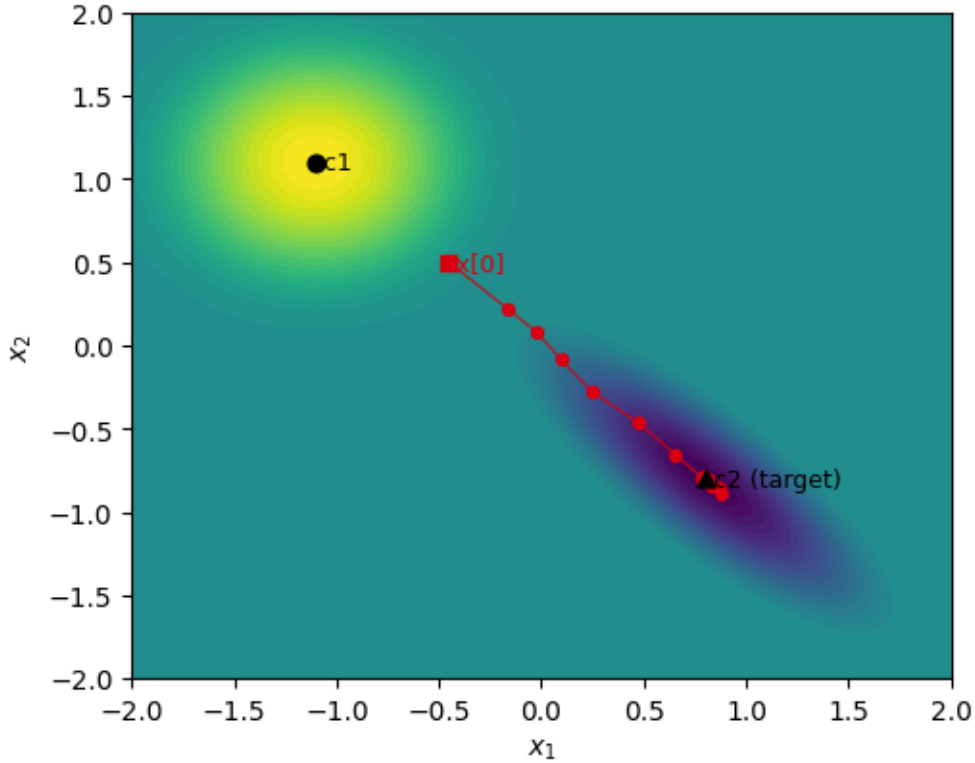
```
alpha = 0.15
phi = 0.5
v = np.array([0.4, -0.4])
x = np.array([-0.45, 0.5])

xs = [x.copy()]
for _ in range(15):
    v = phi * v - alpha * grad(x)
    x = x + v
    xs.append(x.copy())
trace = np.array(xs)

print(f"x[15] = ({x[0]:+.4f}, {x[1]:+.4f}), f = {f(x):+.4f} "
      f"(target c_2 = ({c2[0]:+.4f}, {c2[1]:+.4f}), f = -1)")
```

```
fig, ax = plt.subplots(figsize=(5.5, 4.5))
plot_landscape(ax)
ax.plot(trace[:, 0], trace[:, 1], color="#D90012", linewidth=0.7)
ax.scatter(trace[:, 0], trace[:, 1], color="#D90012", s=18)
plt.show()
```

$x[15] = (+0.7831, -0.7951)$, $f = -0.9990$ (target $c_2 = (+0.8000, -0.8000)$, $f = -1$)



Observation. The trajectory crosses the dead zone purely from inertia (since $\nabla f = \mathbf{0}$ there) and enters c_2 's support after a few iterations. From there the negative-weight bump produces a pull toward c_2 , and the remaining iterations refine to $f \approx -0.999$ — within 0.001 of the true global minimum at c_2 where $f = -1$.

Exercise 8: Multivariate Optimization 3 (SGD)

Optimization in Machine Learning

Reproducibility

Produced with Python 3.10 and R 4.4.1. To reproduce, install the pinned package versions below.

Python

```
pip install numpy==2.0.2 matplotlib==3.10.8
```

R

```
install.packages("remotes")
remotes::install_version("ggplot2", "3.5.1")
remotes::install_version("gridExtra", "2.3")
```

Consider ordinary linear least squares (without intercept):

$$\min_{\theta} \mathbb{E}_{\mathbf{x}, y} [(\theta^\top \mathbf{x} - y)^2]$$

with $\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \Sigma_{\mathbf{x}})$ and $y \mid \mathbf{x} \sim \mathcal{N}(\theta^{*\top} \mathbf{x}, \sigma^2)$. Throughout the coding parts we use the univariate setting $\Sigma_{\mathbf{x}} = 1/4$, $\sigma = 1/10$, $\theta^* = 1/2$. Each coding cell draws fresh samples from this generative model (fresh mini-batches in (c); streaming single samples in (e)).

Note

R vs. Python parity. R's `set.seed(123)` and Python's `np.random.seed(123)` use *different* RNGs. The data above (and consequently the SGD trajectories below) differ between languages. Both implementations are correct; only the underlying data differs. Compare *algorithmic behavior* (cloud shape, convergence pattern) rather than expecting

numerically identical θ values.

(a) Unbiasedness of the SGD gradient

Show that

$$\mathbb{E}_{\mathbf{x},y}[\nabla_{\theta}[(\theta^{\top}\mathbf{x} - y)^2]] = \nabla_{\theta}\mathbb{E}_{\mathbf{x},y}[(\theta^{\top}\mathbf{x} - y)^2].$$

Solution.

We compute both expressions and compare the results.

$$\begin{aligned}\mathbb{E}_{\mathbf{x},y}[\nabla_{\theta}[(\theta^{\top}\mathbf{x} - y)^2]] &= \mathbb{E}_{\mathbf{x},y}[2\mathbf{x}\mathbf{x}^{\top}\theta - 2\mathbf{x}y] \\ &= \mathbb{E}_{\mathbf{x}}\mathbb{E}_{y|\mathbf{x}}[2\mathbf{x}\mathbf{x}^{\top}\theta - 2\mathbf{x}y] \\ &= \mathbb{E}_{\mathbf{x}}[2\mathbf{x}\mathbf{x}^{\top}\theta - 2\mathbf{x}\mathbf{x}^{\top}\theta^*] \\ &= 2\Sigma_{\mathbf{x}}(\theta - \theta^*).\end{aligned}$$

$$\begin{aligned}\nabla_{\theta}\mathbb{E}_{\mathbf{x},y}[(\theta^{\top}\mathbf{x} - y)^2] &= \nabla_{\theta}\mathbb{E}_{\mathbf{x},y}[\theta^{\top}\mathbf{x}\mathbf{x}^{\top}\theta - 2\theta^{\top}\mathbf{x}y + y^2] \\ &= \nabla_{\theta}\mathbb{E}_{\mathbf{x}}[\theta^{\top}\mathbf{x}\mathbf{x}^{\top}\theta] - \nabla_{\theta}\mathbb{E}_{\mathbf{x},y}[2\theta^{\top}\mathbf{x}y] + \nabla_{\theta}\mathbb{E}_y[y^2] \\ &= 2\Sigma_{\mathbf{x}}\theta - 2\nabla_{\theta}\mathbb{E}_{\mathbf{x}}\mathbb{E}_{y|\mathbf{x}}[\theta^{\top}\mathbf{x}y] \\ &= 2\Sigma_{\mathbf{x}}\theta - 2\nabla_{\theta}\mathbb{E}_{\mathbf{x}}\mathbb{E}_{y|\mathbf{x}}[\theta^{\top}\mathbf{x}\mathbf{x}^{\top}\theta^*] \\ &= 2\Sigma_{\mathbf{x}}\theta - 2\nabla_{\theta}(\theta^{\top}\Sigma_{\mathbf{x}}\theta^*) \\ &= 2\Sigma_{\mathbf{x}}\theta - 2\Sigma_{\mathbf{x}}\theta^* \\ &= 2\Sigma_{\mathbf{x}}(\theta - \theta^*).\end{aligned}$$

(b) Implication for SGD

Interpret (a) in terms of stochastic gradient descent.

Solution.

We can estimate $\mathbb{E}_{\mathbf{x},y}[\nabla_{\theta}[(\theta^{\top}\mathbf{x} - y)^2]]$ without bias via SGD, since we have access to realizations of the gradients $\nabla_{\theta}[(\theta^{\top}\mathbf{x} - y)^2]$. From (a), it follows that this estimate is also an unbiased estimate of the gradient of our objective function $\nabla_{\theta}\mathbb{E}_{\mathbf{x},y}[(\theta^{\top}\mathbf{x} - y)^2]$. Hence, SGD can be successfully applied in this situation.

(c) Gradient variance simulation

In the univariate setting $\Sigma_x = 1/4$, $\sigma = 1/10$, $\theta^* = 1/2$, with data of size 10,000:

Plot the **confusion** (variance of the SGD gradient around the true full-batch gradient) for $\theta \in \{0, 0.05, 0.10, \dots, 1.00\}$. For each θ draw 200 independent batches and plot the per-batch confusion. Repeat for batch sizes $m = 100$ and $m = 1,000$.

Solution.

R

```
library(ggplot2)
library(gridExtra)

set.seed(123)
sigma_x <- 0.5 # Sigma_x = 1/4, so sd = 1/2
sigma_y <- 0.1
theta_star <- 0.5

# data of size n (drawn once; each confusion below re-samples a fresh batch)
n <- 10000
x <- sort(rnorm(n, sd = sigma_x))
y <- theta_star * x + rnorm(n, sd = sigma_y)

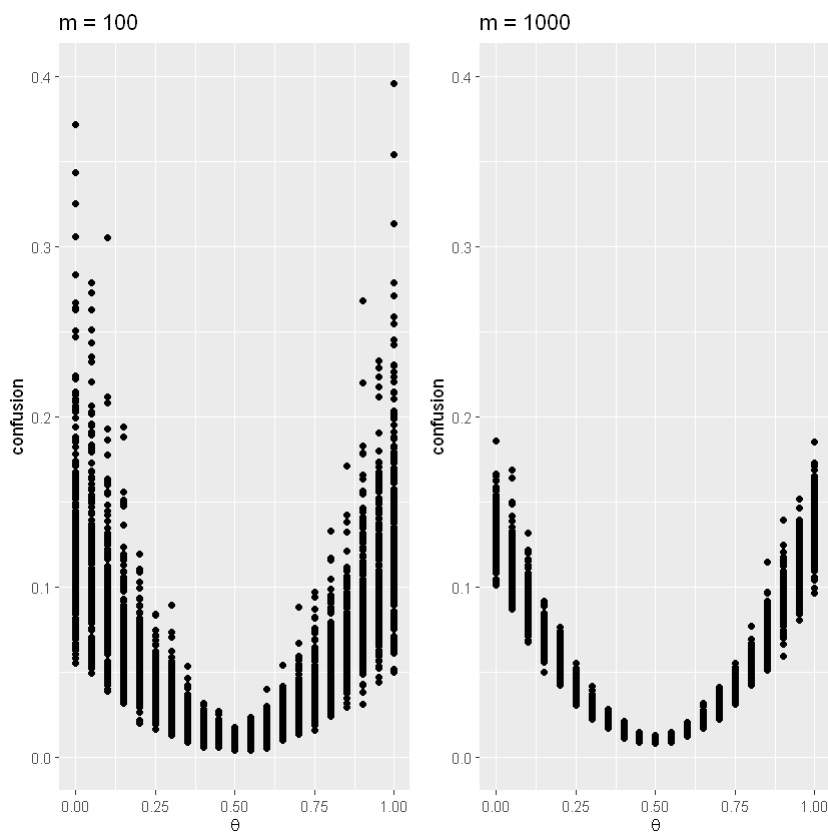
# confusion: mean squared deviation of the per-sample gradients from the
# population gradient 2 * sigma_x^2 * (theta - theta_star)
compute_conf <- function(theta, m) {
  x <- rnorm(m, sd = sigma_x)
  y <- theta_star * x + rnorm(m, sd = sigma_y)
  mean((2 * (x * x * theta - y * x) - 2 * sigma_x^2 * (theta - theta_star))^2)
}

thetas <- seq(0, 1, length.out = 21)
reps <- 200

run_grid <- function(m) {
  confs <- c()
  for (i in 1:reps)
    for (theta in thetas) confs <- c(confs, compute_conf(theta, m))
  data.frame(theta = rep(thetas, reps), conf = confs)
}
```

```
df_100 <- run_grid(100)
df_1000 <- run_grid(1000)

p1 <- ggplot(df_100, aes(theta, conf)) + geom_point() +
  ylim(0, 0.4) + xlab(expression(theta)) +
  ylab("confusion") + ggtitle("m = 100")
p2 <- ggplot(df_1000, aes(theta, conf)) + geom_point() +
  ylim(0, 0.4) + xlab(expression(theta)) +
  ylab("confusion") + ggtitle("m = 1000")
grid.arrange(p1, p2, ncol = 2)
```



Python

```
import numpy as np
import matplotlib.pyplot as plt
```

```

np.random.seed(123)
sigma_x = 0.5 # Sigma_x = 1/4, so sd = 1/2
sigma_y = 0.1
theta_star = 0.5

# data of size n (drawn once; each confusion below re-samples a fresh batch)
n = 10000
x = np.sort(np.random.normal(0, sigma_x, n))
y = theta_star * x + np.random.normal(0, sigma_y, n)

def compute_conf(theta, m):
    # confusion: mean squared deviation of the per-sample gradients from the
    # population gradient  $2 * \sigma_x^2 * (\theta - \theta_{star})$ 
    x = np.random.normal(0, sigma_x, m)
    y = theta_star * x + np.random.normal(0, sigma_y, m)
    per_sample = 2 * (x * x * theta - y * x)
    pop_grad = 2 * sigma_x ** 2 * (theta - theta_star)
    return float(np.mean((per_sample - pop_grad) ** 2))

thetas = np.linspace(0, 1, 21)
reps = 200

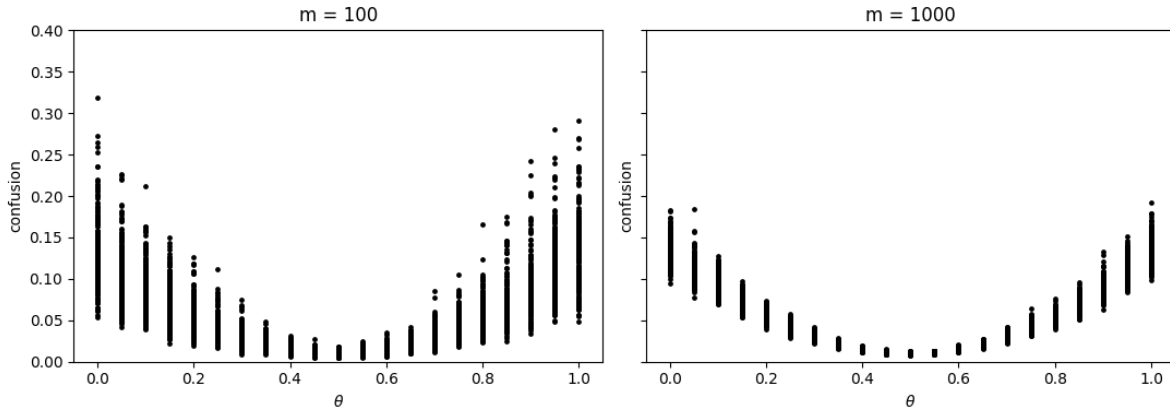
def run_grid(m):
    out = np.empty(reps * len(thetas))
    k = 0
    for _ in range(reps):
        for th in thetas:
            out[k] = compute_conf(th, m); k += 1
    return out

vals_100 = run_grid(100)
vals_1000 = run_grid(1000)

fig, axes = plt.subplots(1, 2, figsize=(11, 4), sharey=True)
for ax, vals, m_lbl in zip(axes, (vals_100, vals_1000), (100, 1000)):
    ax.scatter(np.tile(thetas, reps), vals, s=6, color="black")
    ax.set_ylim(0, 0.4)
    ax.set_xlabel(r"$\theta$"); ax.set_ylabel("confusion")

```

```
ax.set_title(f"m = {m_lbl}")
plt.tight_layout()
plt.show()
```



For each θ and batch size m we draw a fresh batch and measure the *confusion*: the mean squared deviation of the per-sample gradients $2(x_i^2\theta - y_ix_i)$ from the population gradient $g(\theta) = 2\sigma_x^2(\theta - \theta^*)$. This estimates the sample-to-sample variance of the gradient at θ , which the larger batch $m = 1000$ pins down more tightly than $m = 100$.

(d) Reading the simulation

What do you observe in (c)?

Solution.

Qualitatively, we observe for both settings that the mean and the variance of the confusion rise symmetrically around θ^* . As expected, the mean and the variance of the confusion are smaller for the larger batch size $m = 1000$ than for $m = 100$.

(e) SGD vs population GD

On the same setup, run SGD with **batch size 1**, step size $\alpha = 0.3$, starting at $\theta = 0$, for 20 iterations. Repeat the SGD run 200 times and overlay all trajectories. Compare with **population GD** (same step size, same start) using the analytical gradient $g(\theta) = 2\sigma_x^2(\theta - \theta^*)$.

Solution.

R

```
set.seed(123)
alpha <- 0.3
n_iters <- 20
n_runs <- 200
m <- 1

# SGD with batch size 1
sgd_traces <- matrix(NA, n_runs, n_iters)
for (j in 1:n_runs) {
  theta <- 0
  for (i in 1:n_iters) {
    x <- rnorm(m, sd = sigma_x)
    y <- theta_star * x + rnorm(m, sd = sigma_y)
    theta <- theta - alpha * mean(2 * (x * x * theta - y * x))
    sgd_traces[j, i] <- theta
  }
}

# Population GD using the analytical gradient
#  $2 * \sigma_x^2 * (\theta - \theta_{\text{star}})$ 
theta <- 0
gd_trace <- theta
for (i in 1:n_iters) {
  theta <- theta - alpha * 2 * sigma_x^2 * (theta - theta_star)
  gd_trace <- c(gd_trace, theta)
}

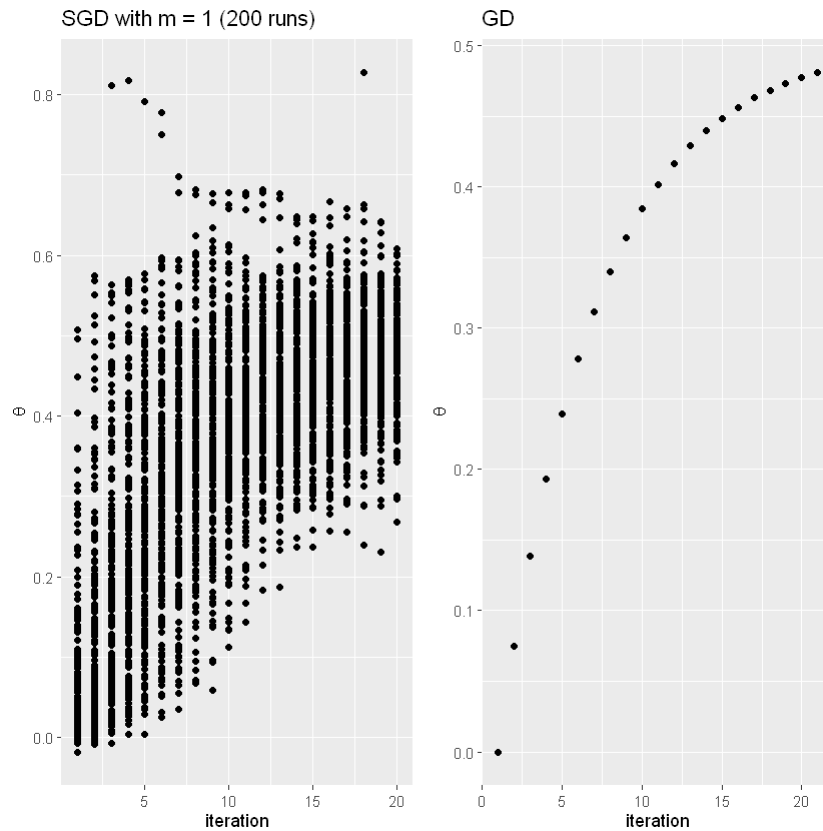
df_sgd <- data.frame(it = rep(1:n_iters, each = n_runs),
                     theta = as.numeric(sgd_traces))
df_gd <- data.frame(it = seq_along(gd_trace), theta = gd_trace)

p_sgd <- ggplot(df_sgd, aes(it, theta)) + geom_point() +
  ylab(expression(theta)) + xlab("iteration") +
  ggtitle("SGD with m = 1 (200 runs)")
p_gd <- ggplot(df_gd, aes(it, theta)) + geom_point() +
  ylab(expression(theta)) + xlab("iteration") + ggtitle("GD")
grid.arrange(p_sgd, p_gd, ncol = 2)

cat(sprintf(paste0(
  "After 20 iters:  SGD mean = %+.4f, sd = %.4f;  ",
```

```
"GD theta = %+4f  (theta* = %.4f)\n"),
    mean(sgd_traces[, n_iters]), sd(sgd_traces[, n_iters]),
    gd_trace[length(gd_trace)], theta_star))
```

After 20 iters: SGD mean = +0.4647, sd = 0.0626; GD theta = +0.4806 (theta* = 0.5000)



Python

```
np.random.seed(123)
alpha = 0.3
n_iters = 20
n_runs = 200
m = 1

# SGD with batch size 1
```

```

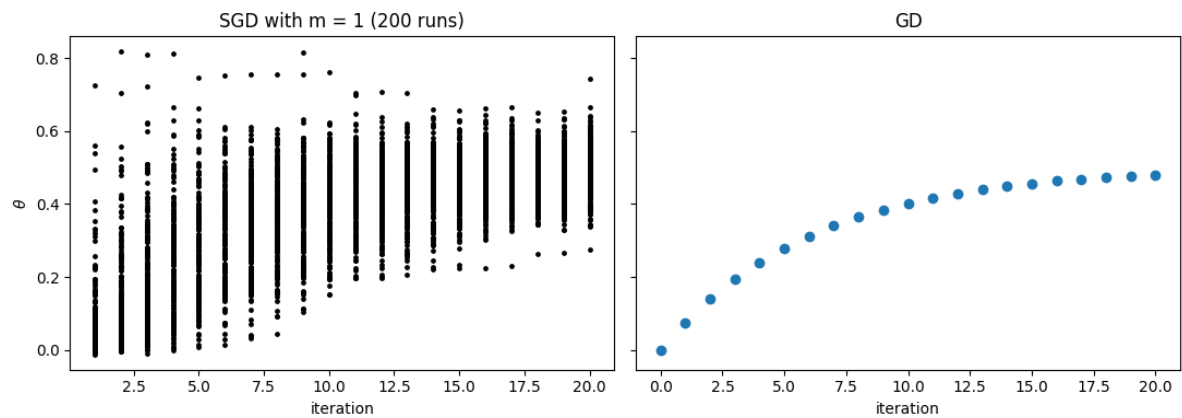
sgd_traces = np.empty((n_runs, n_iters))
for j in range(n_runs):
    theta = 0.0
    for i in range(n_iters):
        x = np.random.normal(0, sigma_x, m)
        y = theta_star * x + np.random.normal(0, sigma_y, m)
        theta = theta - alpha * np.mean(2 * (x * x * theta - y * x))
        sgd_traces[j, i] = theta

# Population GD using the analytical gradient
#  $2 * \sigma_x^2 * (\theta - \theta_{\text{star}})$ 
theta = 0.0
gd_trace = [theta]
for i in range(n_iters):
    theta = theta - alpha * 2 * sigma_x ** 2 * (theta - theta_star)
    gd_trace.append(theta)

fig, axes = plt.subplots(1, 2, figsize=(11, 4), sharey=True)
sgd_it = np.arange(1, n_iters + 1)
for j in range(n_runs):
    axes[0].scatter(sgd_it, sgd_traces[j], s=6, color="black")
axes[0].set_xlabel("iteration"); axes[0].set_ylabel(r"$\theta$")
axes[0].set_title("SGD with m = 1 (200 runs)")
axes[1].scatter(np.arange(len(gd_trace)), gd_trace)
axes[1].set_xlabel("iteration"); axes[1].set_title("GD")
plt.tight_layout()
plt.show()

print(f"After 20 iters:  SGD mean = {sgd_traces[:, -1].mean():+.4f}, "
      f"sd = {sgd_traces[:, -1].std():.4f};  "
      f"GD theta = {gd_trace[-1]:+.4f}  ( $\theta^* = \{\theta_{\text{star}}\}$ )")

```



After 20 iters: SGD mean = +0.4862, sd = 0.0688; GD theta = +0.4806 ($\theta^* = 0.5000$)

Exercise 9: Multivariate Optimization 4 (Newton-Raphson + Gauss-Newton)

Optimization in Machine Learning

Reproducibility

Produced with Python 3.10 and R 4.4.1. To reproduce, install the pinned package versions below.

Python

```
pip install numpy==2.0.2 matplotlib==3.10.8
```

R

```
install.packages("remotes")  
remotes::install_version("ggplot2", "3.5.1")  
remotes::install_version("gridExtra", "2.3")
```

Setup

We model the data with $\pi : \mathbb{R}^2 \rightarrow [0, 1]$ defined as

$$\pi(x_1, x_2) = \frac{1}{1 + \exp((1, x_1, x_2)^\top \theta)}$$

and minimize the **Brier loss**

$$\mathcal{R}_{\text{emp}} = \sum_{i=1}^n \|y^{(i)} - \pi(\mathbf{x}^{(i)})\|_2^2.$$

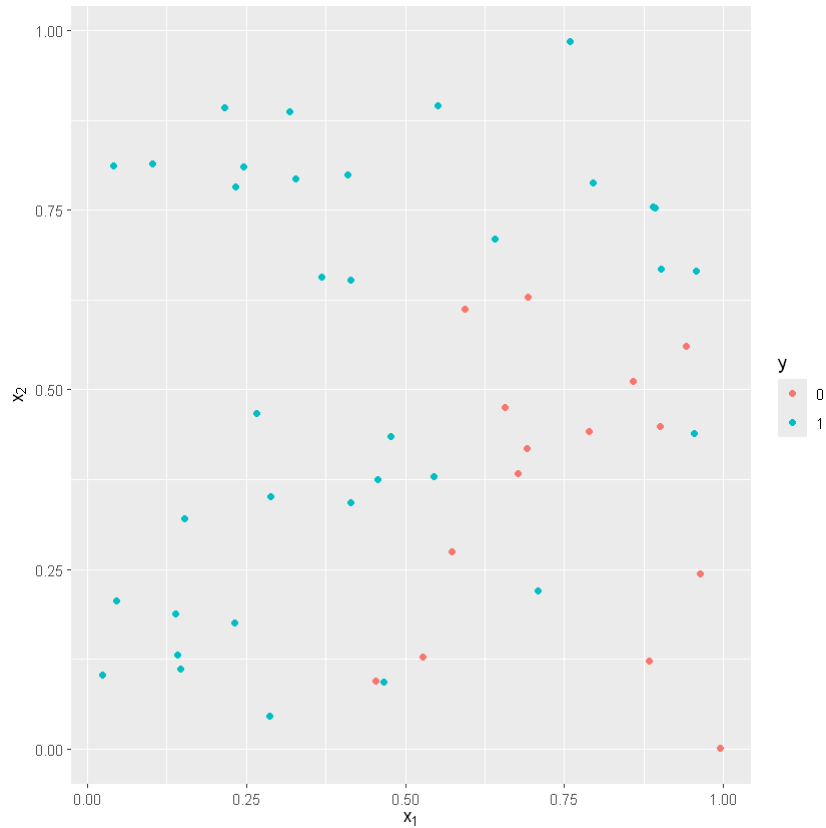
Note the **flipped sigmoid** convention: $\pi(x) \rightarrow 0$ as $\tilde{\mathbf{x}}^\top \theta \rightarrow +\infty$ (the opposite of the standard logistic). This is a quirk of this exercise's parameterization — keep it in mind when reading gradient signs below.

R

```
library(ggplot2)

set.seed(123)
n <- 50
X <- matrix(runif(2 * n), ncol = 2)
X_model <- cbind(1, X) # n x 3 design
y <- -((X_model %*% c(0.3, -1, 1) + rnorm(n, 0, 0.3) < 0) - 1)
df <- as.data.frame(X); df$type <- as.character(y)

ggplot(df) +
  geom_point(aes(x = V1, y = V2, color = type)) +
  xlab(expression(x[1])) + ylab(expression(x[2])) +
  labs(color = "y")
```



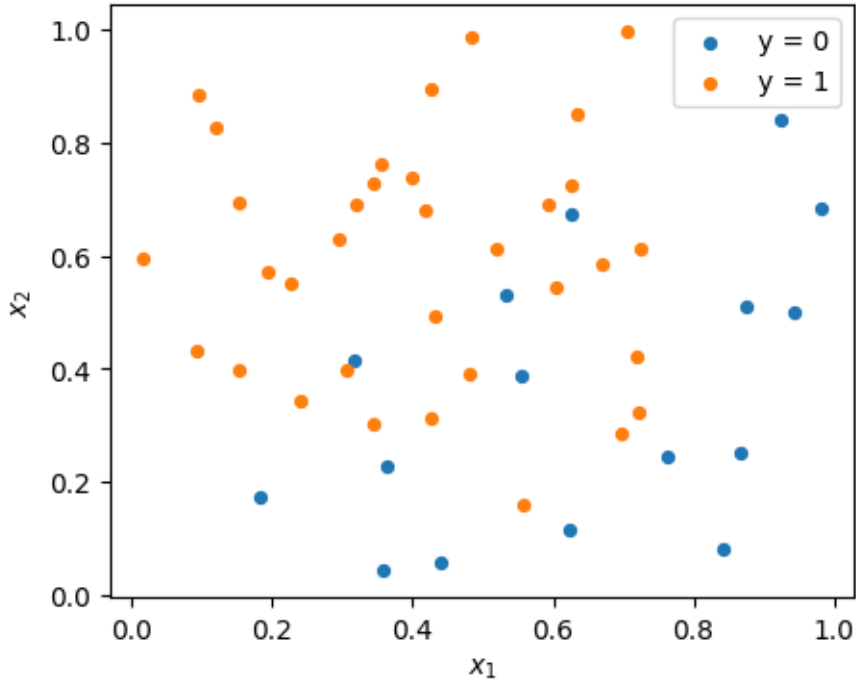
Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(123)
n = 50
X = np.random.uniform(size=(n, 2))
X_model = np.column_stack([np.ones(n), X]) # n x 3 design
lin = X_model @ np.array([0.3, -1.0, 1.0]) + np.random.normal(0, 0.3, n)
y = -((lin < 0).astype(int) - 1)
y = y.astype(float)

fig, ax = plt.subplots(figsize=(5, 4))
for label in (0, 1):
    mask = y == label
    ax.scatter(X[mask, 0], X[mask, 1], label=f"y = {label}", s=18)
```

```
ax.set_xlabel(r"$x_1$"); ax.set_ylabel(r"$x_2$")
ax.legend()
plt.show()
```



i Note

R vs. Python parity. R's `set.seed(123)` and Python's `np.random.seed(123)` use *different* RNGs and call them in different orders, so the simulated data differ between languages. NR and GN are deterministic given the data, so the *trajectories* (as printed below) will not match between languages — but the *qualitative behavior* (NR converging quadratically, GN converging more slowly) should match.

(a) Gradient of the Brier loss

Show that

$$\nabla_{\theta} \mathcal{R}_{\text{emp}} = \sum_{i=1}^n 2 \cdot \frac{y^{(i)} \exp(f^{(i)}) - (\exp(-f^{(i)}) + 1)^{-1}}{(\exp(f^{(i)}) + 1)^2} \cdot \tilde{\mathbf{x}}^{(i)}$$

where $\tilde{\mathbf{x}}^{(i)} = (1, x_1^{(i)}, x_2^{(i)})^\top$ and $f^{(i)} = \tilde{\mathbf{x}}^{(i)\top} \theta$.

Solution.

Define $s : \mathbb{R} \rightarrow \mathbb{R}, f \mapsto \frac{1}{1+\exp(f)}$.

$$\begin{aligned}
\nabla_{\theta} \mathcal{R}_{\text{emp}} &= \nabla_{\theta} \sum_{i=1}^n \|y^{(i)} - f(\mathbf{x}^{(i)})\|_2^2 = \sum_{i=1}^n \frac{d}{df} \|y^{(i)} - s(f^{(i)})\|_2^2 \cdot \nabla_{\theta} f^{(i)} \\
&= \sum_{i=1}^n 2 \frac{y^{(i)}(\exp(f^{(i)}) + 1) - 1}{\exp(f^{(i)}) + 1} \cdot \frac{\exp(f^{(i)})}{(\exp(f^{(i)}) + 1)^2} \tilde{\mathbf{x}}^{(i)} \\
&= \sum_{i=1}^n 2 \frac{y^{(i)}(\exp(f^{(i)}) + 1) - 1}{\exp(f^{(i)}) + 1} \cdot \frac{\frac{\exp(f^{(i)})}{\exp(f^{(i)}) + 1}}{\exp(f^{(i)}) + 1} \tilde{\mathbf{x}}^{(i)} \\
&= \sum_{i=1}^n 2 \frac{y^{(i)}(\exp(f^{(i)})) - \frac{\exp(f^{(i)})}{\exp(f^{(i)}) + 1}}{(\exp(f^{(i)}) + 1)^2} \tilde{\mathbf{x}}^{(i)} \\
&= \sum_{i=1}^n 2 \frac{y^{(i)}(\exp(f^{(i)})) - (\exp(-f^{(i)}) + 1)^{-1}}{(\exp(f^{(i)}) + 1)^2} \tilde{\mathbf{x}}^{(i)}
\end{aligned}$$

(b) Hessian of the Brier loss

Show that

$$\nabla_{\theta}^2 \mathcal{R}_{\text{emp}} = \sum_{i=1}^n 2 \cdot \frac{\exp(f^{(i)})(y^{(i)}(1 - \exp(2f^{(i)})) - 1 + 2\exp(f^{(i)}))}{(\exp(f^{(i)}) + 1)^4} \cdot \tilde{\mathbf{x}}^{(i)} \tilde{\mathbf{x}}^{(i)\top}.$$

Solution.

Introduce the shorthand $E := \exp(f^{(i)})$ (so $\exp(-f^{(i)}) = E^{-1}$) and drop the per-sample index i inside the sum ($y := y^{(i)}, \tilde{\mathbf{x}} := \tilde{\mathbf{x}}^{(i)}$). Differentiating the gradient from (a):

$$\begin{aligned}
\nabla_{\theta}^2 \mathcal{R}_{\text{emp}} &= \sum_{i=1}^n \frac{d}{df} 2 \frac{yE - (E^{-1} + 1)^{-1}}{(E + 1)^2} \tilde{\mathbf{x}} \nabla_{\theta} f^{\top} \\
&= \sum_{i=1}^n 2 \frac{(yE - (E^{-1} + 1)^{-2} E^{-1})(E + 1)^2 - (yE - (E^{-1} + 1)^{-1}) 2(E + 1)E}{(E + 1)^4} \tilde{\mathbf{x}} \tilde{\mathbf{x}}^{\top} \\
&= \sum_{i=1}^n 2 \frac{yE(E + 1)^2 - E - (2yE(E + 1) - 2E)E}{(E + 1)^4} \tilde{\mathbf{x}} \tilde{\mathbf{x}}^{\top} \\
&= \sum_{i=1}^n 2 \frac{E(y(E + 1)^2 - 1 - 2yE(E + 1) + 2E)}{(E + 1)^4} \tilde{\mathbf{x}} \tilde{\mathbf{x}}^{\top} \\
&= \sum_{i=1}^n 2 \frac{E(y(E^2 + 2E + 1 - 2E^2 - 2E) - 1 + 2E)}{(E + 1)^4} \tilde{\mathbf{x}} \tilde{\mathbf{x}}^{\top} \\
&= \sum_{i=1}^n 2 \frac{E(y(1 - E^2) - 1 + 2E)}{(E + 1)^4} \tilde{\mathbf{x}} \tilde{\mathbf{x}}^{\top}
\end{aligned}$$

(c) Non-convexity

Show that $\mathcal{R}_{\text{emp}}$ is not convex in general.

Solution.

Assume, e.g., there is only one observation with $y^{(1)} = 0$ then

$$\nabla_{\theta}^2 \mathcal{R}_{\text{emp}} = \frac{2 \exp(f^{(1)})(2 \exp(f^{(1)}) - 1)}{(\exp(f^{(1)}) + 1)^4} \underbrace{\tilde{\mathbf{x}}^{(1)} \tilde{\mathbf{x}}^{(1)\top}}_{\text{p.s.d.}}.$$

If a p.s.d. matrix is multiplied with a negative number it becomes a n.s.d. matrix, i.e., $\nabla_{\theta}^2 \mathcal{R}_{\text{emp}}$ is n.s.d. if $2 \exp(f^{(1)}) < 1 \iff f^{(1)} < \ln(0.5)$. This condition trivially holds, e.g., if $\theta = (\ln(0.5) - 1, 0, 0)^{\top}$.

(d) Newton-Raphson

Implement Newton-Raphson to minimize $\mathcal{R}_{\text{emp}}$. Run for 30 iterations starting at $\theta^{[0]} = \mathbf{0}$. Plot $\mathcal{R}_{\text{emp}}$ and the parameter trajectory.

Solution.

R

```
# Brier loss + gradient + Hessian using named intermediates from solution (b):
# E = exp(f), D = E + 1, s = 1/D = pi(x).
remp <- function(theta) {
  E <- exp(X_model %>% theta)
  sum((y - 1 / (E + 1))^2)
}

grad_remp <- function(theta) {
  E <- exp(X_model %>% theta)
  inner <- 2 * (y * E - E / (E + 1)) / (E + 1)^2      # per-sample d_R/d_f
  c(t(inner) %>% X_model)                             # chain rule x df/dtheta
}

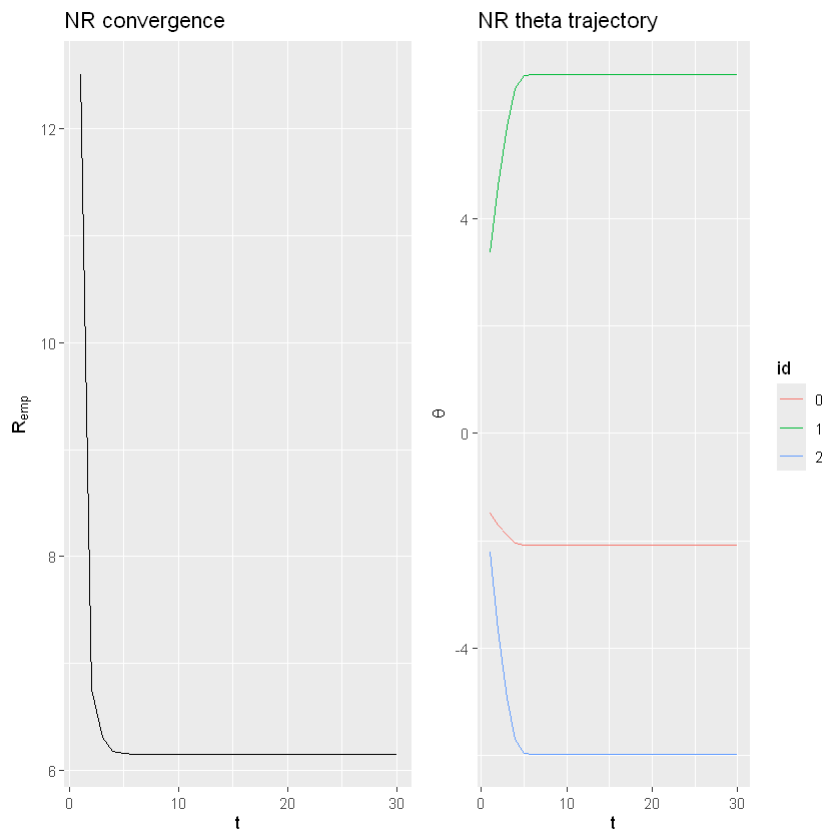
hess_remp <- function(theta) {
  E <- exp(X_model %>% theta)
  diag_factor <- as.numeric(2 * E * (y * (1 - E^2) - 1 + 2 * E) / (E + 1)^4)
  t(X_model) %>% (diag_factor * X_model)
}

# Newton-Raphson loop
theta <- c(0, 0, 0)
remps_nr <- numeric(30)
thetas_nr <- matrix(NA, 30, 3)
for (i in 1:30) {
  remps_nr[i] <- remp(theta)
  theta <- theta + solve(hess_remp(theta), -grad_remp(theta))
  thetas_nr[i, ] <- theta
}
cat(sprintf("NR final theta = (%+.4f, %+.4f, %+.4f), R_emp = %.4f\n",
            theta[1], theta[2], theta[3], remp(theta)))

# Diagnostic plots
library(gridExtra)
p1 <- ggplot(data.frame(t = 1:30, R = remps_nr), aes(t, R)) + geom_line() +
  ylab(expression(R[emp])) + ggtitle("NR convergence")
p2 <- ggplot(data.frame(t = rep(1:30, 3),
                        theta = c(thetas_nr),
                        id = factor(rep(0:2, each = 30)))) +
  geom_line(aes(t, theta, color = id)) +
  ylab(expression(theta)) + ggtitle("NR theta trajectory")
```

```
grid.arrange(p1, p2, ncol = 2)
```

NR final theta = (-2.0871, +6.6674, -5.9675), R_emp = 6.1561



Python

```
# Brier loss + gradient + Hessian using named intermediates from solution (b):  
#  $E = \exp(f)$ ,  $D = E + 1$ ,  $s = 1/D = \pi(x)$ .  
def remp(theta):  
    E = np.exp(X_model @ theta)  
    return float(np.sum((y - 1 / (E + 1)) ** 2))  
  
def grad_remp(theta):  
    E = np.exp(X_model @ theta)
```

```

inner = 2 * (y * E - E / (E + 1)) / (E + 1) ** 2      # per-sample d_R/d_f
return X_model.T @ inner

def hess_remp(theta):
    E = np.exp(X_model @ theta)
    diag_factor = 2 * E * (y * (1 - E ** 2) - 1 + 2 * E) / (E + 1) ** 4
    return X_model.T @ (diag_factor[:, None] * X_model)

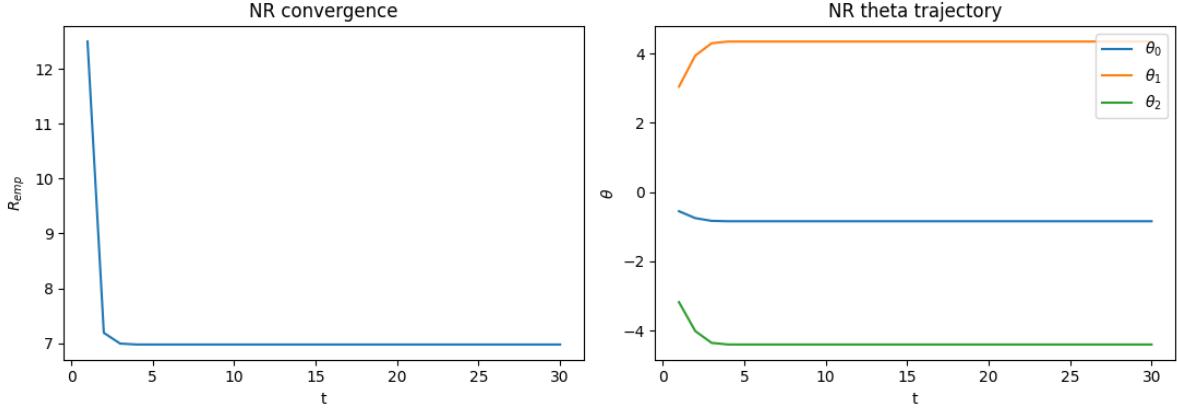
theta = np.zeros(3)
remps_nr = np.empty(30)
thetas_nr = np.empty((30, 3))
for i in range(30):
    remps_nr[i] = remp(theta)
    theta = theta + np.linalg.solve(hess_remp(theta), -grad_remp(theta))
    thetas_nr[i] = theta

print(f"NR final theta = ({theta[0]:+.4f}, {theta[1]:+.4f}, {theta[2]:+.4f}), "
      f"R_emp = {remp(theta):.4f}")

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].plot(np.arange(1, 31), remps_nr)
axes[0].set_xlabel("t"); axes[0].set_ylabel(r"$R_{\text{emp}}$")
axes[0].set_title("NR convergence")
for j in range(3):
    axes[1].plot(np.arange(1, 31), thetas_nr[:, j], label=fr"$\theta_{\text{j}}$")
axes[1].set_xlabel("t"); axes[1].set_ylabel(r"$\theta$"); axes[1].legend()
axes[1].set_title("NR theta trajectory")
plt.tight_layout()
plt.show()

```

NR final theta = (-0.8428, +4.3518, -4.4067), R_emp = 6.9744



For Newton-Raphson, we need to solve in each update step

$$\nabla_{\theta}^2 \mathcal{R}_{\text{emp}} \mathbf{d} = -\nabla_{\theta} \mathcal{R}_{\text{emp}}$$

for the descend direction $\mathbf{d}$.

(e) Gauss-Newton

Explain why Gauss-Newton is applicable here, and implement it (30 iterations, $\theta^{[0]} = \mathbf{0}$). Compare its convergence to Newton-Raphson from (d).

Solution.

In this case, we can apply Gauss-Newton since $\mathcal{R}_{\text{emp}}$ is the squared sum of the residuals

$$\mathbf{r} = (y^{(1)} - \pi(\mathbf{x}^{(1)}), \dots, y^{(n)} - \pi(\mathbf{x}^{(n)}))^{\top}.$$

Here, for the update step we need to compute

$$\nabla_{\theta} \mathbf{r} = \begin{pmatrix} \frac{\exp(f^{(1)})}{(1+\exp(f^{(1)}))^2} \tilde{\mathbf{x}}^{(1)\top} \\ \vdots \\ \frac{\exp(f^{(n)})}{(1+\exp(f^{(n)}))^2} \tilde{\mathbf{x}}^{(n)\top} \end{pmatrix}$$

For Gauss-Newton, we solve in each update step

$$(\nabla_{\theta} \mathbf{r}^{\top} \nabla_{\theta} \mathbf{r}) \cdot \mathbf{d} = -\nabla_{\theta} \mathbf{r}^{\top} \cdot \mathbf{r}$$

for the descend direction $\mathbf{d}$.

R

```

options(repr.plot.width = 11, repr.plot.height = 4)
# Gauss-Newton: replace Hessian with J^T J where J is the residual Jacobian.
# residual r_i = y_i - pi(x_i),
# dr_i/dtheta = -pi'(f) * x_i = (E / (E+1)^2) * x_i.
theta <- c(0, 0, 0)
remps_gn <- numeric(30)
thetas_gn <- matrix(NA, 30, 3)
for (i in 1:30) {
  remps_gn[i] <- remp(theta)
  E <- exp(X_model %*% theta)
  r <- y - 1 / (E + 1) # residuals
  J <- as.numeric(E / (E + 1)^2) * X_model # residual Jacobian
  theta <- theta + solve(t(J) %*% J, -t(J) %*% r)
  thetas_gn[i, ] <- theta
}
cat(sprintf("GN final theta = (%+.4f, %+.4f, %+.4f), R_emp = %.4f\n",
            theta[1], theta[2], theta[3], remp(theta)))

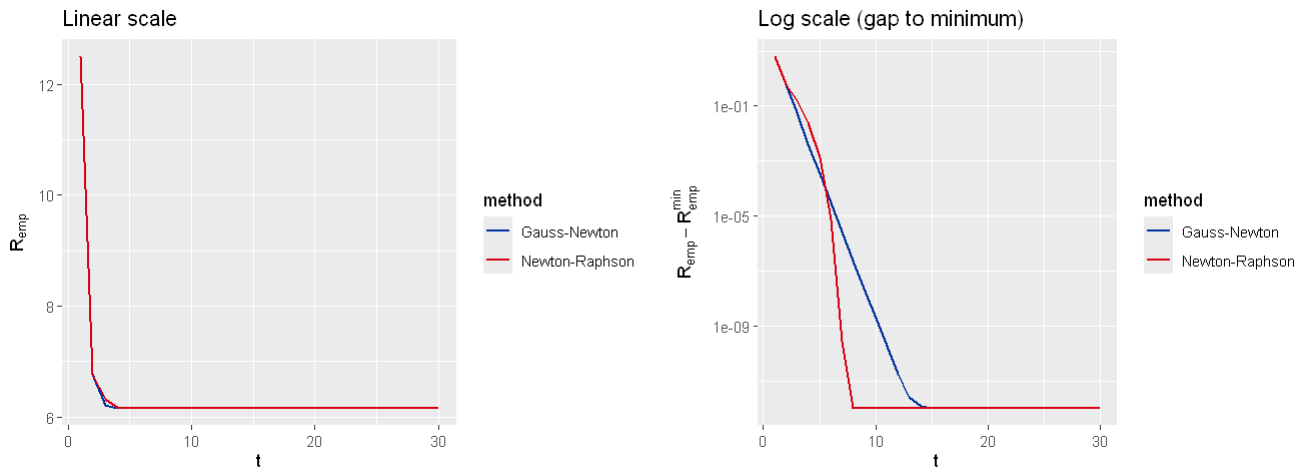
# Compare NR vs GN convergence. Linear axis (left) hides the convergence-rate
# distinction once both are close to the minimum; log-axis on R_emp - R_emp_min
# (right) exposes it.
R_min <- min(remps_nr, remps_gn) - 1e-12
df_compare <- rbind(
  data.frame(t = 1:30, R = remps_nr, gap = remps_nr - R_min,
             method = "Newton-Raphson"),
  data.frame(t = 1:30, R = remps_gn, gap = remps_gn - R_min,
             method = "Gauss-Newton")
)

p_lin <- ggplot(df_compare, aes(t, R, color = method)) +
  geom_line(linewidth = 0.7) +
  scale_color_manual(values = c("Newton-Raphson" = "#D90012",
                                "Gauss-Newton" = "#0033A0")) +
  ylab(expression(R[emp])) + ggtitle("Linear scale")
p_log <- ggplot(df_compare, aes(t, gap, color = method)) +
  geom_line(linewidth = 0.7) +
  scale_y_log10() +
  scale_color_manual(values = c("Newton-Raphson" = "#D90012",
                                "Gauss-Newton" = "#0033A0")) +
  ylab(expression(R[emp] - R[emp]^{min})) +

```

```
ggtitle("Log scale (gap to minimum)")
grid.arrange(p_lin, p_log, ncol = 2)
```

GN final theta = (-2.0871, +6.6674, -5.9675), R_emp = 6.1561



Python

```
# Gauss-Newton: replace Hessian with  $J^T J$ . Residual  $r_i = y_i - \pi(x_i)$ ,
#  $dr_i/d\theta = -\pi'(f) * x_i = (E / (E+1)^2) * x_i$ .
theta = np.zeros(3)
remps_gn = np.empty(30)
thetas_gn = np.empty((30, 3))
for i in range(30):
    remps_gn[i] = remp(theta)
    E = np.exp(X_model @ theta)
    r = y - 1 / (E + 1) # residuals
    J = (E / (E + 1) ** 2)[:, None] * X_model # residual Jacobian
    theta = theta + np.linalg.solve(J.T @ J, -J.T @ r)
    thetas_gn[i] = theta

print(f"GN final theta = ({theta[0]:+.4f}, {theta[1]:+.4f}, {theta[2]:+.4f}), "
      f"R_emp = {remp(theta):.4f}")

# Compare NR vs GN convergence. Linear axis (left) hides the convergence-rate
# distinction once both are close to the minimum; log-axis on R_emp - R_emp_min
```

```

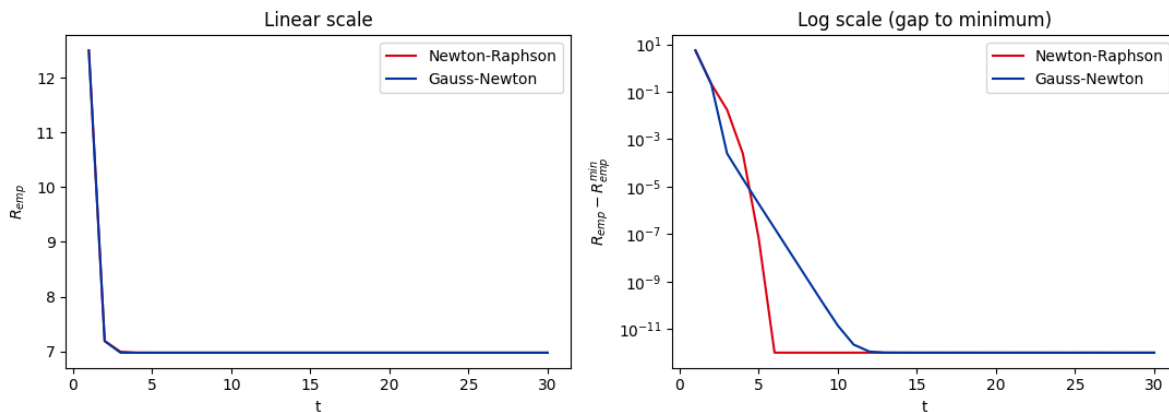
# (right) exposes it.
R_min = min(remps_nr.min(), remps_gn.min()) - 1e-12
iters = np.arange(1, 31)

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].plot(iters, remps_nr, color="#D90012", label="Newton-Raphson")
axes[0].plot(iters, remps_gn, color="#0033A0", label="Gauss-Newton")
axes[0].set_xlabel("t"); axes[0].set_ylabel(r" $R_{\text{emp}}$ ")
axes[0].set_title("Linear scale"); axes[0].legend()

axes[1].semilogy(iters, remps_nr - R_min, color="#D90012",
                 label="Newton-Raphson")
axes[1].semilogy(iters, remps_gn - R_min, color="#0033A0", label="Gauss-Newton")
axes[1].set_xlabel("t"); axes[1].set_ylabel(r" $R_{\text{emp}} - R_{\text{emp}}^{\text{min}}$ ")
axes[1].set_title("Log scale (gap to minimum)"); axes[1].legend()
plt.tight_layout()
plt.show()

```

GN final theta = (-0.8428, +4.3518, -4.4067), $R_{\text{emp}} = 6.9744$



Exercise 10: Linear Programming (Sparse Quantile Regression)

Optimization in Machine Learning

Reproducibility

Produced with Python 3.10 and R 4.4.1. To reproduce, install the pinned package versions below.

Python

```
pip install numpy==2.0.2 matplotlib==3.10.8 scipy==1.15.3
```

R

```
install.packages("remotes")
remotes::install_version("ggplot2", "3.5.1")
remotes::install_version("linprog", "0.9.6")
```

Setup

We have $n = 50$ univariate observations with skewed (gamma) errors:

$$x_i \sim \text{Uniform}(0, 1), \quad y_i = 2x_i + \text{Gamma}(\text{shape} = 1).$$

The skewed noise motivates *quantile* regression instead of OLS — we want to fit the conditional quantile of $y \mid x$, not the conditional mean.

Multivariate sparse quantile regression is

$$\min_{\beta \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n \rho_{\tau}(y^{(i)} - \beta_0 - \beta^{\top} \mathbf{x}^{(i)}) \quad \text{s.t.} \quad \|\beta\|_1 \leq t$$

with $\tau \in (0, 1)$ and the **pinball loss**

$$\rho_{\tau}(s) = \begin{cases} \tau \cdot s & s > 0 \\ -(1 - \tau) \cdot s & s \leq 0 \end{cases}$$

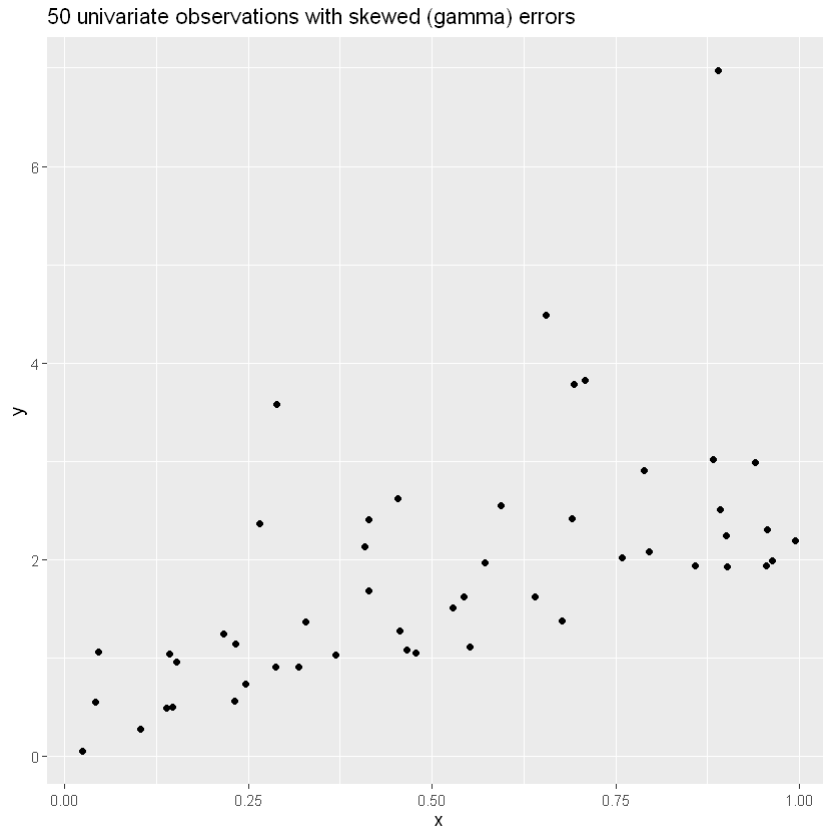
(asymmetric residual weighting; for $\tau = 0.5$ this is just $\frac{1}{2}|s|$). We treat the univariate ($p = 1$) case below.

R

```
library(ggplot2)

set.seed(123)
n <- 50
x <- runif(n)
y <- 2 * x + rgamma(n, shape = 1)

ggplot(data.frame(x = x, y = y), aes(x, y)) + geom_point() +
  ggtitle("50 univariate observations with skewed (gamma) errors")
```



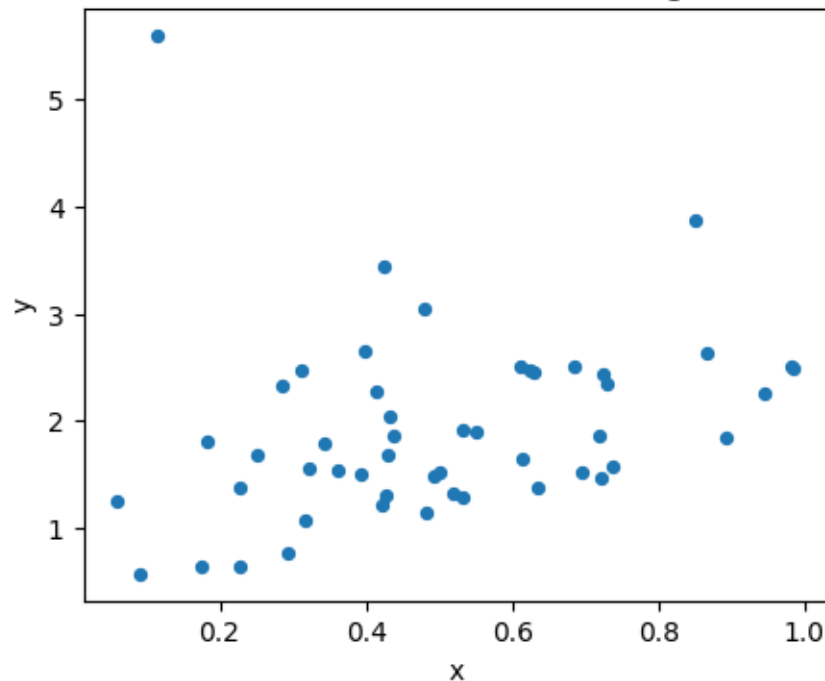
Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(123)
n = 50
x = np.random.uniform(size=n)
y = 2 * x + np.random.gamma(shape=1, size=n)

fig, ax = plt.subplots(figsize=(5, 4))
ax.scatter(x, y, s=18)
ax.set_xlabel("x"); ax.set_ylabel("y")
ax.set_title("50 univariate observations with skewed (gamma) errors")
plt.show()
```

50 univariate observations with skewed (gamma) errors



i Note

R vs. Python parity. R's `set.seed(123)` and Python's `np.random.seed(123)` use *different* RNGs, and `runif` / `rgamma` differ from `np.random.uniform` / `np.random.gamma` even given the same logical seed. The simulated data therefore differs between languages, so the LP solutions $\hat{\beta}$ will differ as well. Both implementations are correct; only the data differs. *Within* a language, the LP is deterministic — the same standard-form input gives identical $\hat{\beta}$ from `linprog` / `linprog`.

(a) Standard form of the LP

Find the standard LP form of the one-dimensional sparse quantile regression problem.

Hint: an unconstrained variable x can be decomposed into two non-negative variables $x = x^+ - x^-$, with $|x| = x^+ + x^-$. (At the optimum at most one of x^+, x^- is nonzero.)

Solution.

Univariate sparse quantile regression:

$$\min_{(\beta_0, \beta_1) \in \mathbb{R}^2} \frac{1}{n} \sum_{i=1}^n \rho_\tau(y^{(i)} - \beta_0 - \beta_1 x^{(i)}) \text{ s.t. } |\beta_1| \leq t$$

1. Decompose unconstrained parameters into positive and negative part: $\beta_i = \beta_i^+ - \beta_i^- \quad i = 0, 1$
2. Transform absolute value: $|\beta_1| \leq t \iff \beta_1^+ + \beta_1^- \leq t$
3. We can write $\rho_\tau(\underbrace{y^{(i)} - \beta_0 - \beta_1 x^{(i)}}_{=: r^{(i)}}) = \tau \cdot r^{(i)} \mathbb{1}_{\{r^{(i)} > 0\}} - (1 - \tau) \cdot r^{(i)} \mathbb{1}_{\{r^{(i)} \leq 0\}} = \tau \cdot r^{(i)+} + (1 - \tau) \cdot r^{(i)-}$
4. Transform equality of the residuals into two inequalities: $r^{(i)+} - r^{(i)-} = y^{(i)} - \beta_0 - \beta_1 x^{(i)} \iff r^{(i)+} - r^{(i)-} \leq y^{(i)} - \beta_0 - \beta_1 x^{(i)}$ and $-r^{(i)+} + r^{(i)-} \leq -y^{(i)} + \beta_0 + \beta_1 x^{(i)}$

With this we get the standard form:

$$\max_{\mathbf{z} \in \mathbb{R}^{4+2n}} \mathbf{c}^\top \mathbf{z}$$

s.t.

$$\mathbf{A}\mathbf{z} \leq \mathbf{b}, \quad \mathbf{z} \geq \mathbf{0}$$

with

$$\mathbf{z} = \begin{pmatrix} \beta_0^+ \\ \beta_0^- \\ \beta_1^+ \\ \beta_1^- \\ r^{(1)+} \\ \vdots \\ r^{(n)+} \\ r^{(1)-} \\ \vdots \\ r^{(n)-} \end{pmatrix}, \quad \mathbf{c} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ -\tau \\ \vdots \\ -\tau \\ -(1-\tau) \\ \vdots \\ -(1-\tau) \end{pmatrix}, \quad \mathbf{A} = \begin{pmatrix} 0 & 0 & 1 & 1 & 0 & \dots & 0 \\ 1_n & -1_n & \mathbf{x} & -\mathbf{x} & \mathbf{I}_n & & -\mathbf{I}_n \\ -1_n & 1_n & -\mathbf{x} & \mathbf{x} & -\mathbf{I}_n & & \mathbf{I}_n \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} t \\ \mathbf{y} \\ -\mathbf{y} \end{pmatrix}$$

(b) Visualise the empirical risk and feasible region

Plot $\mathcal{R}_{\text{emp}}(\beta_0, \beta_1)$ for $(\beta_0, \beta_1) \in [-3, 3]^2$ with $\tau = 0.4$. Mark the feasible region $|\beta_1| \leq t = 1.7$.

Solution.

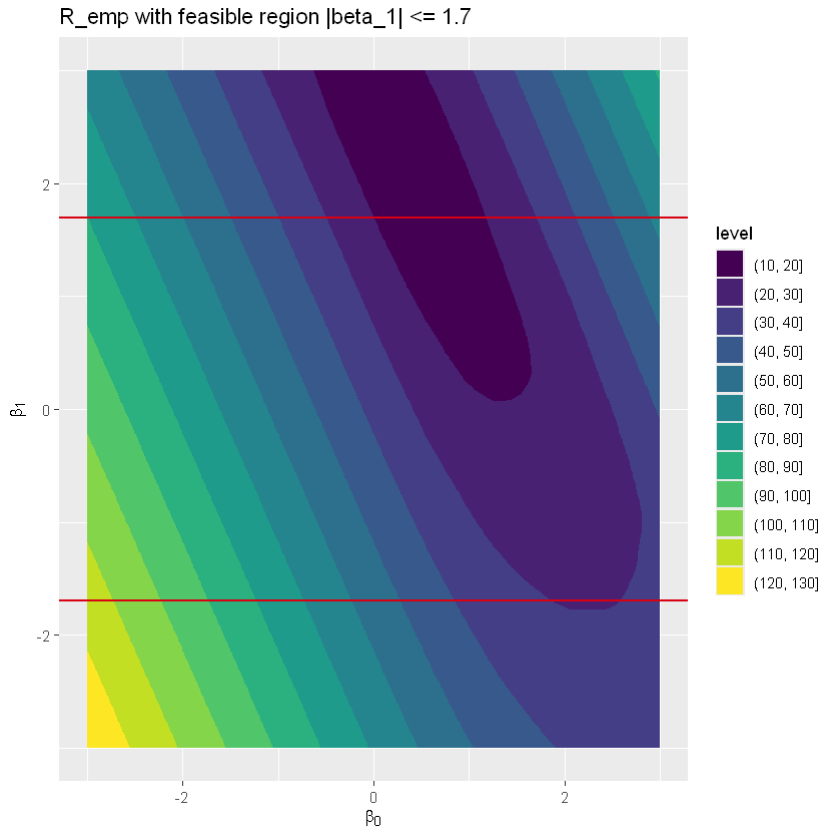
R

```
tau <- 0.4
tval <- 1.7

# Pinball loss
remp <- function(beta) {
  r <- y - beta[1] - beta[2] * x
  sum(ifelse(r > 0, tau * r, -(1 - tau) * r))
}

b_grid <- seq(-3, 3, by = 0.05)
bb <- expand.grid(X1 = b_grid, X2 = b_grid)
fbb <- apply(bb, 1, remp)
df_grid <- data.frame(bb, fbb = fbb)

remp_plot <- ggplot(df_grid, aes(X1, X2, z = fbb)) +
  geom_contour_filled() +
  geom_hline(yintercept = c(-tval, tval), color = "#D90012", linewidth = 0.7) +
  xlab(expression(beta[0])) + ylab(expression(beta[1])) +
  ggtitle("R_emp with feasible region  $|\beta_1| \leq 1.7$ ")
remp_plot
```



Python

```
tau = 0.4
tval = 1.7

def remp(beta):
    """Pinball loss summed over the data."""
    r = y - beta[0] - beta[1] * x
    return float(np.sum(np.where(r > 0, tau * r, -(1 - tau) * r)))

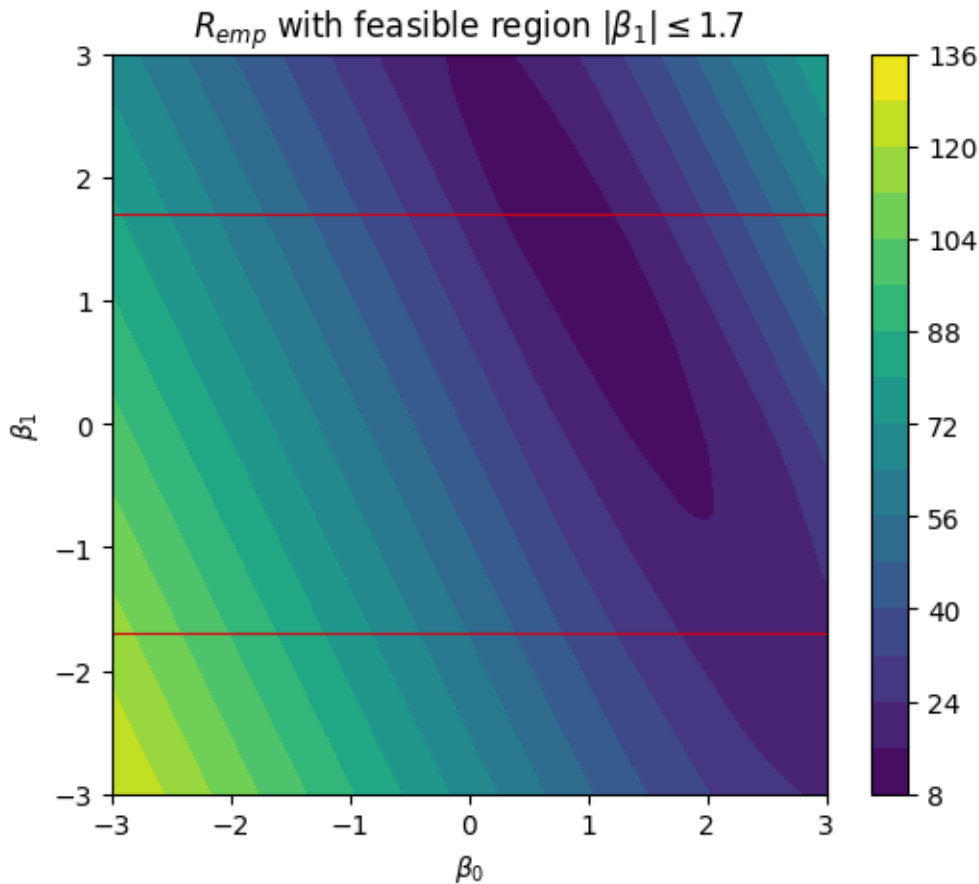
b_grid = np.arange(-3, 3 + 0.05, 0.05)
B0, B1 = np.meshgrid(b_grid, b_grid)
Z = np.array([[remp(np.array([b0, b1])) for b0 in b_grid] for b1 in b_grid])

fig, ax = plt.subplots(figsize=(6, 5))
```

```

cs = ax.contourf(B0, B1, Z, levels=14)
ax.axhline( tval, color="#D90012", linewidth=0.8)
ax.axhline(-tval, color="#D90012", linewidth=0.8)
ax.set_xlabel(r"$\beta_0$"); ax.set_ylabel(r"$\beta_1$")
ax.set_title(r"$R_{\text{emp}}$ with feasible region $|\beta_1| \leq 1.7$")
fig.colorbar(cs)
plt.show()

```



Two horizontal red lines at $\beta_1 = \pm t$ delimit the feasible region. The contour shows the level sets of $\mathcal{R}_{\text{emp}}$. The optimum lives on or inside the strip.

(c) Solve the LP

Use R's `linprog::solveLP` (or Python's `scipy.optimize.linprog`) to solve the sparse 40 % quantile regression with $t = 1.7$.

Solution.

R

```
library(linprog)

# Build the standard form (a):
#   z = (beta0+, beta0-, beta1+, beta1-, r1+, ..., rn+, r1-, ..., rn-)
#   c, A, b as in the prose above.
Amat <- c(0, 0, 1, 1, rep(0, 2 * n)) # budget
# r+ - r- <= y - beta_0 - beta_1 x
Amat <- rbind(Amat, cbind(1, -1, x, -x, diag(n), -diag(n)))
Amat <- rbind(Amat, cbind(-1, 1, -x, x, -diag(n), diag(n))) # mirrored
bvec <- c(tval, y, -y)
cvec <- c(0, 0, 0, 0, rep(tau, n), rep(1 - tau, n)) # positive c, minimize

res <- solveLP(cvec, bvec, Amat, maximum = FALSE, lpSolve = TRUE)

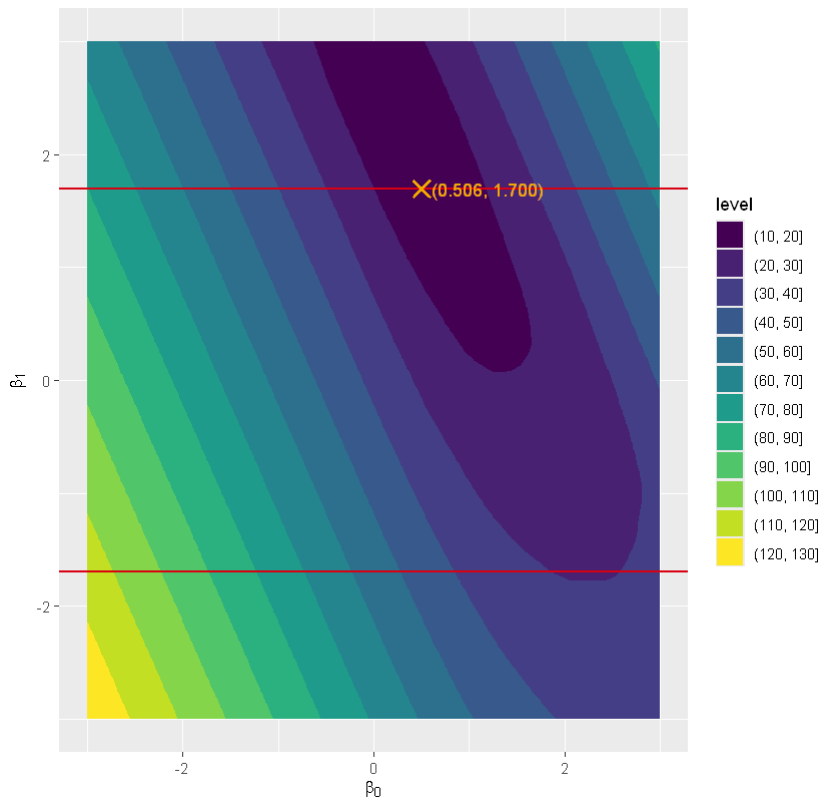
beta0 <- res$solution[1] - res$solution[2]
beta1 <- res$solution[3] - res$solution[4]
constraint_active <- abs(abs(beta1) - tval) < 1e-6

cat(sprintf("beta_hat = (%+.4f, %+.4f), R_emp* = %.4f\n",
            beta0, beta1, remp(c(beta0, beta1))))
cat(sprintf("budget constraint |beta_1| <= %.2f is %s (|beta_1*| = %.4f)\n",
            tval, if (constraint_active) "ACTIVE" else "slack", abs(beta1)))

# Contour from (b), now with the LP optimum marked in parameter space.
remp_plot +
  annotate("point", x = beta0, y = beta1, color = "#F2A800",
          size = 3.5, shape = 4, stroke = 1.5) +
  annotate("text", x = beta0, y = beta1,
          label = sprintf(" (%.3f, %.3f)", beta0, beta1),
          hjust = 0, color = "#F2A800") +
  ggtitle(sprintf("R_emp with feasible region; orange X = LP optimum (%s)",
                  if (constraint_active) "constraint active" else "interior"))
```

```
beta_hat = (+0.5062, +1.7000), R_emp* = 13.7880
budget constraint |beta_1| <= 1.70 is ACTIVE (|beta_1*| = 1.7000)
```

R_emp with feasible region; orange X = LP optimum (constraint active)



Python

```
from scipy.optimize import linprog

# Build the standard form (a):
#   z = (beta0+, beta0-, beta1+, beta1-, r1+, ..., rn+, r1-, ..., rn-)
#   (length 4 + 2n)
#   minimize c^T z subject to Az <= b, z >= 0.
I_n = np.eye(n)
row_budget = np.concatenate([[0, 0, 1, 1], np.zeros(2 * n)])
row_pos    = np.column_stack([np.ones(n), -np.ones(n), x, -x, I_n, -I_n])
row_neg    = np.column_stack([-np.ones(n), np.ones(n), -x, x, -I_n, I_n])
A = np.vstack([row_budget, row_pos, row_neg])
b = np.concatenate([[tval], y, -y])
c = np.concatenate([[0, 0, 0, 0], np.full(n, tau), np.full(n, 1 - tau)])

res = linprog(c, A_ub=A, b_ub=b, bounds=[(0, None)] * (4 + 2 * n),
```

```

        method="highs")

beta0 = res.x[0] - res.x[1]
beta1 = res.x[2] - res.x[3]
constraint_active = abs(abs(beta1) - tval) < 1e-6

print(f"beta_hat = ({beta0:+.4f}, {beta1:+.4f}), "
      f"R_emp* = {remp(np.array([beta0, beta1])):.4f}")
print(f"budget constraint |beta_1| <= {tval:.2f} is "
      f"{'ACTIVE' if constraint_active else 'slack'} "
      f"(|beta_1*| = {abs(beta1):.4f})")
print(f"linprog status: {res.message}, iters: {res.nit}")

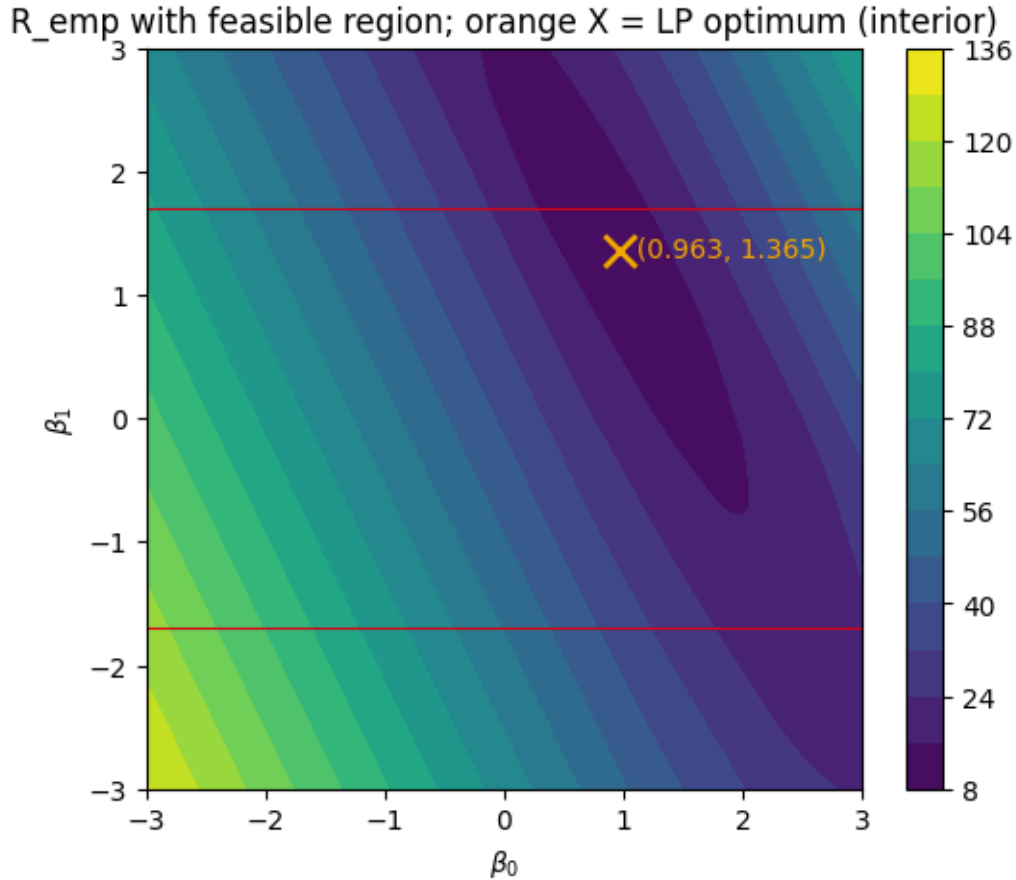
# Contour from (b), now with the LP optimum marked in parameter space.
fig, ax = plt.subplots(figsize=(6, 5))
cs = ax.contourf(B0, B1, Z, levels=14)
ax.axhline( tval, color="#D90012", linewidth=0.8)
ax.axhline(-tval, color="#D90012", linewidth=0.8)
ax.scatter(beta0, beta1, color="#F2A800", marker="x", s=140, linewidths=2)
ax.annotate(f" ({beta0:.3f}, {beta1:.3f})", (beta0, beta1), color="#F2A800",
           fontsize=10, va="center")
ax.set_xlabel(r"$\beta_0$"); ax.set_ylabel(r"$\beta_1$")
ax.set_title(f"R_emp with feasible region; orange X = LP optimum "
            f"({'constraint active' if constraint_active else 'interior'})")
fig.colorbar(cs)
plt.show()

```

```

beta_hat = (+0.9633, +1.3647), R_emp* = 12.6711
budget constraint |beta_1| <= 1.70 is slack (|beta_1*| = 1.3647)
linprog status: Optimization terminated successfully. (HiGHS Status 7: Optimal), iters: 50

```



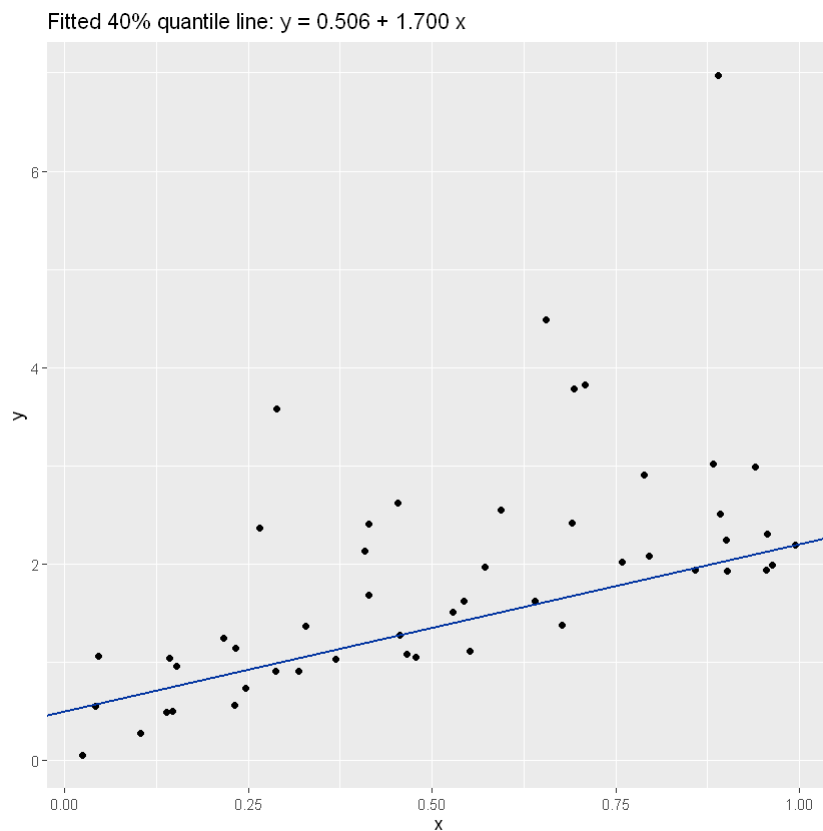
We construct $\mathbf{c}, \mathbf{A}, \mathbf{b}$ following (a), feed to the solver, and recover $\hat{\beta}_j = z_j^+ - z_j^-$. The fitted line $y = \hat{\beta}_0 + \hat{\beta}_1 x$ goes through the data; with $\tau = 0.4$ it lies *below* the $\tau = 0.5$ median fit (and well below the OLS fit, which the gamma-skewed errors would push high).

Whether the sparsity budget $|\beta_1| \leq t$ binds depends on the data. If the unconstrained quantile slope would exceed t , the optimum is clamped to the boundary $\beta_1 = \pm t$ (constraint *active*), so the plotted optimum lands exactly on the red line; otherwise the optimum lies strictly inside the feasible strip (constraint *slack*), below the line. Because R and Python draw different samples, the two panels here illustrate the two regimes — the printout states which one holds in each case.

R

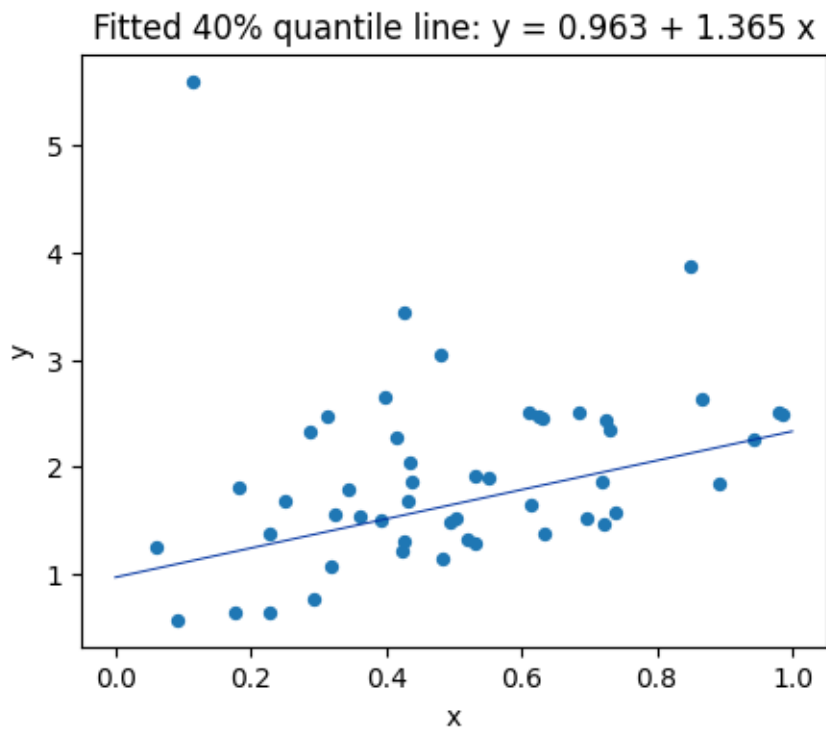
```
ggplot(data.frame(x = x, y = y), aes(x, y)) +
  geom_point() +
  geom_abline(intercept = beta0, slope = beta1,
```

```
color = "#0033A0", linewidth = 0.7) +
ggtitle(sprintf("Fitted 40%% quantile line: y = %.3f + %.3f x", beta0, beta1))
```



Python

```
fig, ax = plt.subplots(figsize=(5, 4))
ax.scatter(x, y, s=18)
x_range = np.array([0.0, 1.0])
ax.plot(x_range, beta0 + beta1 * x_range, color="#0033A0", linewidth=0.7)
ax.set_xlabel("x"); ax.set_ylabel("y")
ax.set_title(f"Fitted 40% quantile line: y = {beta0:.3f} + {beta1:.3f} x")
plt.show()
```



(d) Dual formulation

State the dual of the LP from (a).

Solution.

From the lecture, we know that the dual problem must be

$$\max_{\alpha \in \mathbb{R}^{2n+1}} -\alpha^\top \mathbf{b}$$

s.t.

$$-\alpha^\top \mathbf{A} \leq -\mathbf{c}^\top$$

$$\alpha \geq \mathbf{0}$$

Exercise 11: Nonlinear Programming (Lagrange + KKT + SVM)

Optimization in Machine Learning

Reproducibility

Produced with Python 3.10 and R 4.4.1. To reproduce, install the pinned package versions below.

Python

```
pip install numpy==2.0.2 matplotlib==3.10.8 cvxpy==1.7.5
```

R

```
install.packages("remotes")
remotes::install_version("CVXR", "1.8.2")
remotes::install_version("ggplot2", "3.5.1")
remotes::install_version("mlr3", "1.6.0")
```

Problem 1: Lagrange multipliers

Solve the constrained optimization problem

$$\min_{(x,y) \in \mathbb{R}^2} x + 2y \quad \text{s.t.} \quad x^2 + 4y^2 = 4$$

by Lagrange multipliers.

Solution.

We know that at a minimum (x^*, y^*) the contour lines of the objective function f and the equality constraint g touch and thus it needs to hold that

$$\nabla f(x^*, y^*) = \lambda \nabla g(x^*, y^*)$$

for some Lagrange multiplier $\lambda \in \mathbb{R}$.

Hence, we get the two equalities

$$1 = \lambda 2x^*$$

$$2 = \lambda 8y^*$$

yielding $x^* = 2y^*$.

The third equality is the equality constraint. It gives

$$\begin{aligned} x^{*2} + 4y^{*2} &= 4 \\ \Leftrightarrow 4y^{*2} + 4y^{*2} &= 4 \\ \Leftrightarrow 8y^{*2} &= 4 \\ \Leftrightarrow y^{*2} &= \frac{1}{2}. \end{aligned}$$

Therefore, we have two candidate solutions

$$(x_1^*, y_1^*) = (\sqrt{2}, 1/\sqrt{2}) \quad \text{or} \quad (x_2^*, y_2^*) = (-\sqrt{2}, -1/\sqrt{2}).$$

Since $f(x_1^*, y_1^*) > f(x_2^*, y_2^*)$, we get the solution

$$(x^*, y^*) = (-\sqrt{2}, -1/\sqrt{2}).$$

Problem 2: Nonlinear SVM

We have 200 binary observations from a *moons* distribution (two interleaved half-circles) — not linearly separable. The standard SVM trick: lift to a higher-dimensional feature space via $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^l$ and learn a linear separator there. The primal soft-margin SVM in feature space is

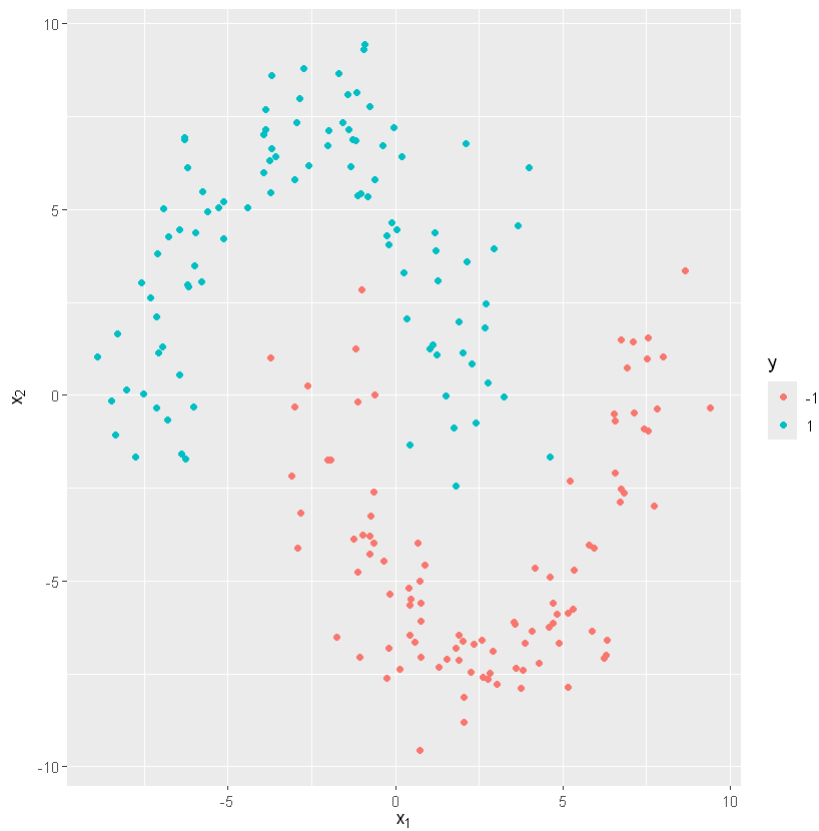
$$\min_{\theta, \theta_0, \zeta} \frac{1}{2} \|\theta\|^2 + C \sum_{i=1}^n \zeta^{(i)} \quad \text{s.t.} \quad y^{(i)} (\langle \theta, \phi(\mathbf{x}^{(i)}) \rangle + \theta_0) \geq 1 - \zeta^{(i)}, \quad \zeta^{(i)} \geq 0 \quad \forall i.$$

R

```
library(ggplot2)
library(CVXR)

# 200 nonlinearly separable observations from a moons distribution.
moon_data <- read.csv("data/moons.csv")
n <- nrow(moon_data)
moon_data$y_dec <- as.factor(moon_data$y)

ggplot(moon_data, aes(x = x1, y = x2, color = y_dec)) +
  geom_point(size = 1.5) +
  xlab(expression(x[1])) + ylab(expression(x[2])) +
  labs(color = expression(y))
```

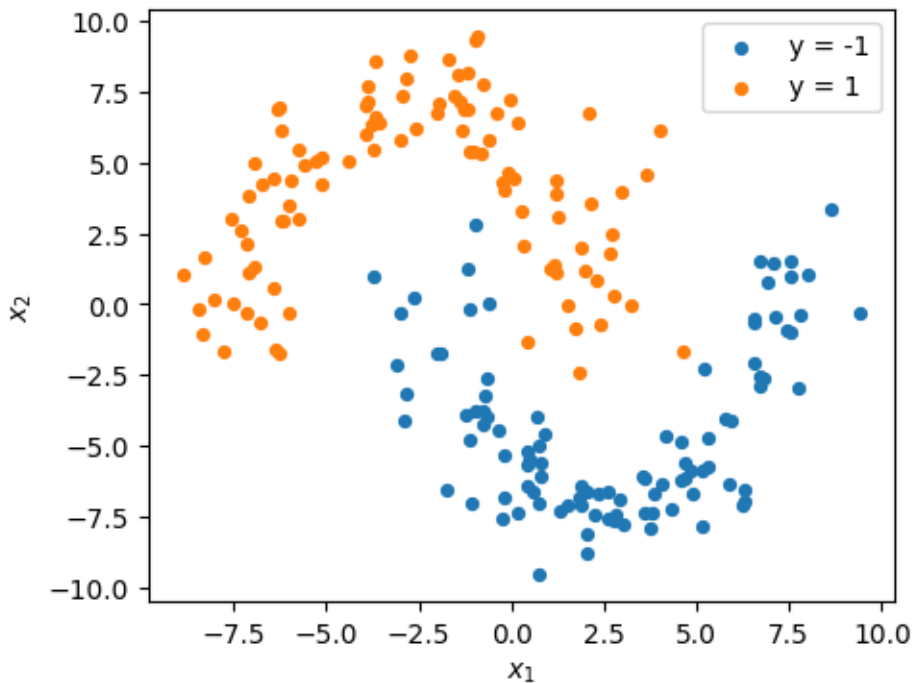


Python

```
import numpy as np
import matplotlib.pyplot as plt
import cvxpy as cp

# 200 nonlinearly separable observations from a moons distribution.
data = np.genfromtxt("data/moons.csv", delimiter=",", names=True)
X = np.column_stack([data["x1"], data["x2"]])
y = data["y"].astype(float)
n = X.shape[0]

fig, ax = plt.subplots(figsize=(5, 4))
for label in (-1, 1):
    mask = y == label
    ax.scatter(X[mask, 0], X[mask, 1], label=f"y = {label}", s=18)
ax.set_xlabel(r"$x_1$"); ax.set_ylabel(r"$x_2$")
ax.legend()
plt.show()
```



(a) Lagrangian

Write down the Lagrangian of the nonlinear SVM.

Solution.

$$\mathcal{L} = 0.5\|\theta\|^2 + C \sum_{i=1}^n \zeta^{(i)} - \sum_{i=1}^n \alpha^{(i)} (\mathbf{y}^{(i)} (\phi(\mathbf{x}^{(i)})^\top \theta + \theta_0) - 1 + \zeta^{(i)}) - \sum_{i=1}^n \mu^{(i)} \zeta^{(i)}$$

(b) Solve the primal SVM

Solve the primal nonlinear SVM with $C = 1$ and a third-order polynomial transformation (without intercept)

$$\phi(\mathbf{x}) = (x_1, x_2, x_1^2, x_2^2, x_1 x_2, x_1^2 x_2, x_1 x_2^2, x_1^3, x_2^3)^\top \in \mathbb{R}^9$$

using CVXR (R) or cvxpy (Python).

Solution.

R

```
X <- as.matrix(moon_data[, c("x1", "x2")])

# Cubic polynomial transformation (without intercept), 9 features
cubic <- function(x) cbind(x[, 1], x[, 2],
                           x[, 1]^2, x[, 2]^2, x[, 1] * x[, 2],
                           x[, 1]^2 * x[, 2], x[, 1] * x[, 2]^2,
                           x[, 1]^3, x[, 2]^3)

# Diagonal scaling so that <D phi(x), D phi(z)> = (x^T z + 1)^3 - 1
# (matches the dual kernel in (e)).
D_diag <- c(sqrt(3), sqrt(3), sqrt(3), sqrt(3), sqrt(6), sqrt(3), sqrt(3), 1, 1)
D <- diag(D_diag)

Z <- cubic(X)           # raw phi(X); used in (e) when recovering theta
Z_D <- Z %*% D          # D*phi(X); used in the primal constraint below
C_val <- 1.0

theta0 <- Variable()
theta <- Variable(9)
```

```

slack <- Variable(n)

obj <- (1/2) * sum_squares(theta) + C_val * sum(slack)
constr <- list(
  moon_data$y * (Z_D %%% theta + theta0) >= 1 - slack,
  slack >= 0
)
prob <- Problem(Minimize(obj), constr)
solution <- solve(prob)

theta0_p <- as.numeric(solution$getValue(theta0))
theta_p <- as.numeric(solution$getValue(theta)) # theta in D*phi basis
cat(sprintf("primal: status = %s, obj = %.4f, theta_0 = %+.4f\n",
            solution$status, solution$value, theta0_p))
cat("theta = (", paste(sprintf("%.3f", theta_p), collapse = ", "), ")\n")

# Decision boundary on a grid spanning the data (range computed dynamically).
x_lim <- range(moon_data$x1, moon_data$x2)
pad <- 0.05 * diff(x_lim)
x_grid <- seq(x_lim[1] - pad, x_lim[2] + pad, length.out = 120)
g <- expand.grid(x1 = x_grid, x2 = x_grid)
y_g <- as.numeric(cubic(as.matrix(g)) %%% D %%% theta_p) + theta0_p
df_g <- data.frame(g, val = y_g)

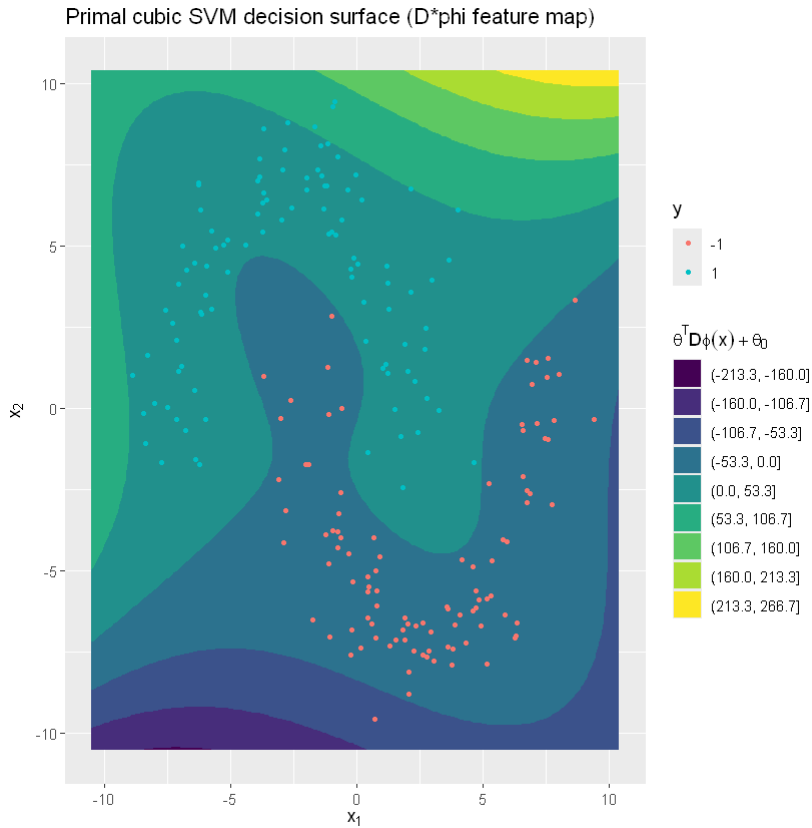
ggplot() +
  geom_contour_filled(data = df_g, aes(x1, x2, z = val), bins = 10) +
  geom_point(data = moon_data, aes(x1, x2, color = y_dec), size = 1.2) +
  xlab(expression(x[1])) + ylab(expression(x[2])) +
  labs(color = expression(y),
       fill = expression(theta^T * D * phi(x) + theta[0])) +
  ggtitle("Primal cubic SVM decision surface (D*phi feature map)")

```

```

primal: status = optimal, obj = 0.4163, theta_0 = -0.1928
theta = ( +0.839, -0.295, +0.074, +0.062, +0.102, +0.013, +0.061, -0.076, +0.111 )

```



Python

```
def cubic(X):
    """Cubic polynomial features without intercept (9 components)."""
    x1, x2 = X[:, 0], X[:, 1]
    return np.column_stack([x1, x2,
                            x1**2, x2**2, x1 * x2,
                            x1**2 * x2, x1 * x2**2,
                            x1**3, x2**3])

# Diagonal scaling so that <D phi(x), D phi(z)> = (x^T z + 1)^3 - 1
# (matches the dual kernel in (e)).
D_diag = np.array([np.sqrt(3), np.sqrt(3), np.sqrt(3), np.sqrt(3), np.sqrt(6),
                    np.sqrt(3), np.sqrt(3), 1.0, 1.0])
D = np.diag(D_diag)
```

```

Z    = cubic(X)                # raw phi(X); used in (e) when recovering theta
Z_D = Z @ D                    # D*phi(X); used in the primal constraint below
C_val = 1.0

theta0 = cp.Variable()
theta  = cp.Variable(9)
slack  = cp.Variable(n, nonneg=True)

obj = cp.Minimize(0.5 * cp.sum_squares(theta) + C_val * cp.sum(slack))
constraints = [cp.multiply(y, Z_D @ theta + theta0) >= 1 - slack]
prob = cp.Problem(obj, constraints)
prob.solve()

theta0_p = float(theta0.value)
theta_p = theta.value          # theta in D*phi basis
print(f"primal: status = {prob.status}, obj = {prob.value:.4f}, "
      f"theta_0 = {theta0_p:.4f}")
print("theta = (" + ", ".join(f"{v:.3f}" for v in theta_p) + ")")

# Decision boundary on a grid
lo, hi = X.min(), X.max()
pad = 0.05 * (hi - lo)
g_grid = np.linspace(lo - pad, hi + pad, 120)
G1, G2 = np.meshgrid(g_grid, g_grid)
G_flat = np.column_stack([G1.ravel(), G2.ravel()])
vals = (cubic(G_flat) @ D @ theta_p + theta0_p).reshape(G1.shape)

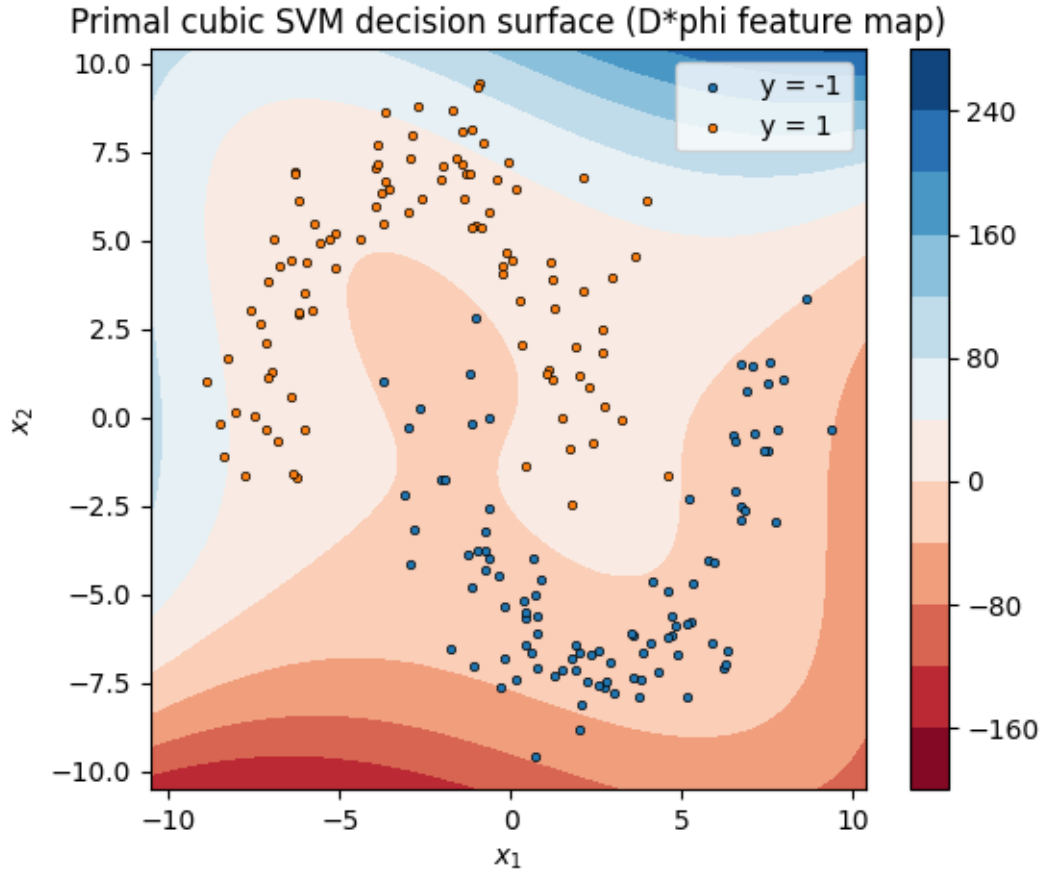
fig, ax = plt.subplots(figsize=(6, 5))
cs = ax.contourf(G1, G2, vals, levels=10, cmap="RdBu")
for label in (-1, 1):
    mask = y == label
    ax.scatter(X[mask, 0], X[mask, 1], s=10, edgecolor="k", linewidth=0.5,
              label=f"y = {label}")
ax.set_xlabel(r"$x_1$"); ax.set_ylabel(r"$x_2$"); ax.legend()
ax.set_title("Primal cubic SVM decision surface (D*phi feature map)")
fig.colorbar(cs)
plt.show()

```

```

primal: status = optimal, obj = 0.4163, theta_0 = -0.1928
theta = (+0.839, -0.295, +0.074, +0.062, +0.102, +0.013, +0.061, -0.076, +0.111)

```



(c) KKT conditions

State the KKT conditions of the general nonlinear SVM.

Solution.

Stationarity

$$\nabla_{\theta} \mathcal{L} = \theta - \sum_{i=1}^n \alpha^{(i)} \mathbf{y}^{(i)} \phi(\mathbf{x}^{(i)}) = 0$$

$$\nabla_{\theta_0} \mathcal{L} = - \sum_{i=1}^n \alpha^{(i)} \mathbf{y}^{(i)} = 0$$

$$\nabla_{\zeta} \mathcal{L} = C \cdot \mathbf{1}_n - \alpha - \mu = 0$$

Primal feasibility

$$\begin{aligned}
-(\mathbf{y}^{(i)}(\phi(\mathbf{x}^{(i)})^\top \theta + \theta_0) - 1 + \zeta^{(i)}) &\leq 0 \\
-\zeta^{(i)} &\leq 0
\end{aligned}$$

Dual feasibility

$$\begin{aligned}
\mu &\geq 0 \\
\alpha &\geq 0
\end{aligned}$$

Complementary slackness

$$\begin{aligned}
-\mu^{(i)}\zeta^{(i)} &= 0 \quad i = 1, \dots, n \\
-\alpha^{(i)}(\mathbf{y}^{(i)}(\phi(\mathbf{x}^{(i)})^\top \theta + \theta_0) - 1 + \zeta^{(i)}) &= 0 \quad i = 1, \dots, n
\end{aligned}$$

(d) Dual form

Derive the dual form of the nonlinear SVM. State an advantage of the dual over the primal.

Hint: use the KKT conditions to eliminate the primal variables from the Lagrangian.

Solution.

From the KKT conditions it follows that

$$\begin{aligned}
\theta &= \sum_{i=1}^n \alpha^{(i)} \mathbf{y}^{(i)} \phi(\mathbf{x}^{(i)}), \\
\sum_{i=1}^n \alpha^{(i)} \mathbf{y}^{(i)} &= 0, \\
C - \underbrace{\mu^{(i)}}_{\geq 0} &= \alpha^{(i)} \Rightarrow C \geq \alpha^{(i)} \quad i = 1, \dots, n.
\end{aligned}$$

Plugging these into the Lagrangian gives

$$\begin{aligned}
&0.5 \left\| \sum_{i=1}^n \alpha^{(i)} \mathbf{y}^{(i)} \phi(\mathbf{x}^{(i)}) \right\|^2 + \sum_{i=1}^n \mu^{(i)} \zeta^{(i)} + \sum_{i=1}^n \alpha^{(i)} \zeta^{(i)} - \left\| \sum_{i=1}^n \alpha^{(i)} \mathbf{y}^{(i)} \phi(\mathbf{x}^{(i)}) \right\|^2 + \sum_{i=1}^n \alpha^{(i)} - \sum_{i=1}^n \alpha^{(i)} \zeta^{(i)} - \sum_{i=1}^n \mu^{(i)} \zeta^{(i)} \\
&= -0.5 \left\| \sum_{i=1}^n \alpha^{(i)} \mathbf{y}^{(i)} \phi(\mathbf{x}^{(i)}) \right\|^2 + \sum_{i=1}^n \alpha^{(i)} = -0.5 \sum_{i=1}^n \sum_{j=1}^n \alpha^{(i)} \alpha^{(j)} \mathbf{y}^{(i)} \mathbf{y}^{(j)} \langle \phi(\mathbf{x}^{(i)}), \phi(\mathbf{x}^{(j)}) \rangle + \sum_{i=1}^n \alpha^{(i)}
\end{aligned}$$

Hence, the dual form of the nonlinear SVM is

$$\max_{\alpha} -0.5 \sum_{i=1}^n \sum_{j=1}^n \alpha^{(i)} \alpha^{(j)} \mathbf{y}^{(i)} \mathbf{y}^{(j)} \langle \phi(\mathbf{x}^{(i)}), \phi(\mathbf{x}^{(j)}) \rangle + \sum_{i=1}^n \alpha^{(i)}$$

s.t.

$$\sum_{i=1}^n \alpha^{(i)} \mathbf{y}^{(i)} = 0,$$

$$0 \leq \alpha \leq C.$$

Here, we can use the kernel trick to evaluate $\langle \phi(\mathbf{x}^{(i)}), \phi(\mathbf{x}^{(j)}) \rangle$ without explicitly computing the projections of each observation. (We only need to compute $\langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle$)

(e) Solve the dual SVM

Solve the dual form from (b) via CVXR/cvxpy.

Hint 1: For a polynomial transformation ϕ of order l (without intercept) there exists an invertible diagonal $\mathbf{D} \in \mathbb{R}^{l \times l}$ such that $\langle \mathbf{D}\phi(\mathbf{x}), \mathbf{D}\phi(\mathbf{z}) \rangle = (\mathbf{x}^\top \mathbf{z} + 1)^l - 1$.

Hint 2: Add $10^{-7}\mathbf{I}$ to the kernel matrix to ensure invertibility / positive-definiteness.

Solution.

R

```
# Kernel matrix: K = (X X^T + 1)^3 - 1.
# Hint 1: <D phi(x), D phi(z)> = (x^T z + 1)^l - 1
K <- (X %*% t(X) + 1)^3 - 1
P <- diag(moon_data$y) %*% K %*% diag(moon_data$y)
P <- P + diag(n) * 1e-7 # Hint 2: ensure PD

alpha <- Variable(n)
obj_d <- sum(alpha) - (1/2) * quad_form(alpha, P)
constr_d <- list(t(alpha) %*% moon_data$y == 0,
               alpha >= 0,
               alpha <= C_val)
prob_d <- Problem(Maximize(obj_d), constr_d)
solution_d <- solve(prob_d)
alpha_p <- as.numeric(solution_d$getValue(alpha))
```

```

cat(sprintf("dual:   status = %s, obj = %.4f\n",
           solution_d$status, solution_d$value))

# Recover theta via the diagonal scaling D so that <D phi, D phi> = K
D <- diag(c(rep(sqrt(3), 4), sqrt(6), rep(sqrt(3), 2), 1, 1))
theta_d <- as.numeric(t(alpha_p * moon_data$y) %*% Z %*% D)

# theta_0: midpoint between max negative and min positive class scores
scores <- as.numeric(Z %*% D %*% theta_d)
theta0_d <- -0.5 * (max(scores[moon_data$y == -1]) +
                  min(scores[moon_data$y == 1]))
cat(sprintf("recovered: theta_0 = %.4f, primal/dual obj diff = %.4e\n",
           theta0_d, solution$value - solution_d$value))

# Support vectors: alpha_i > eps
# (active margin constraint, by complementary slackness).
sv_eps <- 1e-4
sv_mask <- alpha_p > sv_eps
cat(sprintf("support vectors: %d / %d (alpha_i > %.0e)\n",
           sum(sv_mask), n, sv_eps))

y_g_d <- as.numeric(cubic(as.matrix(g)) %*% D %*% theta_d + theta0_d)
df_g_d <- data.frame(g, val = y_g_d)
moon_data$is_sv <- sv_mask

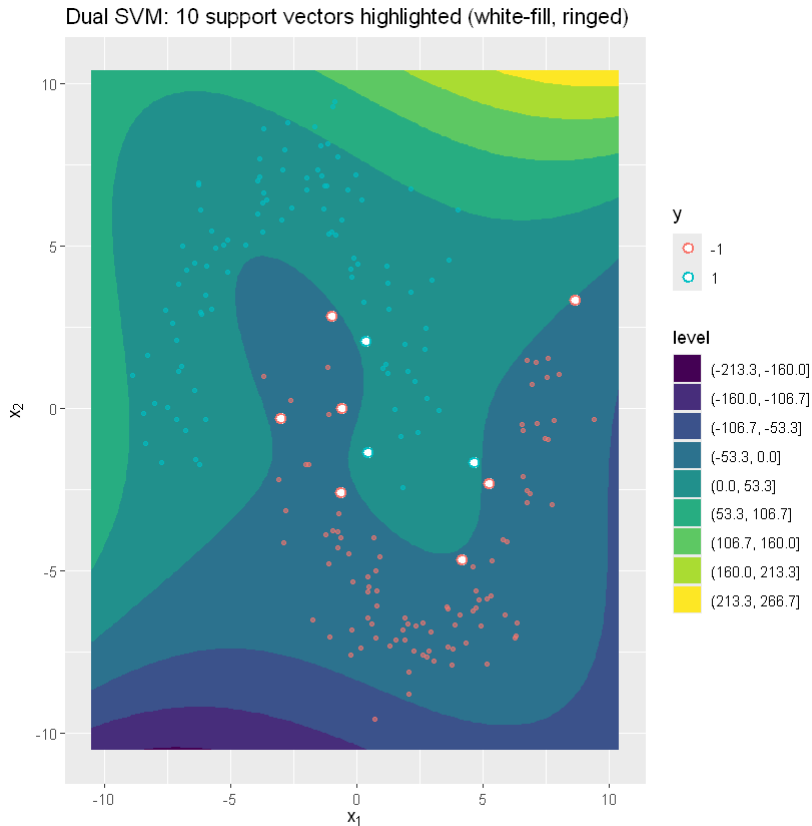
ggplot() +
  geom_contour_filled(data = df_g_d, aes(x1, x2, z = val), bins = 10) +
  geom_point(data = moon_data[!sv_mask, ], aes(x1, x2, color = y_dec),
            size = 1.0, alpha = 0.6) +
  geom_point(data = moon_data[sv_mask, ], aes(x1, x2, color = y_dec),
            size = 2.5, shape = 21, fill = "white", stroke = 1.0) +
  xlab(expression(x[1])) + ylab(expression(x[2])) +
  labs(color = expression(y)) +
  ggtitle(sprintf(paste0("Dual SVM: %d support vectors highlighted ",
                        "(white-fill, ringed)"), sum(sv_mask)))

```

```

dual:   status = optimal, obj = 0.4163
recovered: theta_0 = -0.1928, primal/dual obj diff = 7.0105e-09
support vectors: 10 / 200 (alpha_i > 1e-04)

```



Python

```
# Kernel:  $K = (X X^T + 1)^3 - 1$ .
# Hint 1:  $\langle D \phi(x), D \phi(z) \rangle = (x^T z + 1)^1 - 1$ 
K = (X @ X.T + 1) ** 3 - 1
P = np.diag(y) @ K @ np.diag(y) + 1e-7 * np.eye(n) # Hint 2: ensure PD

alpha = cp.Variable(n, nonneg=True)
obj_d = cp.Maximize(cp.sum(alpha) - 0.5 * cp.quad_form(alpha, cp.psd_wrap(P)))
constraints_d = [alpha @ y == 0, alpha <= C_val]
prob_d = cp.Problem(obj_d, constraints_d)
prob_d.solve()

alpha_p = alpha.value
print(f"dual: status = {prob_d.status}, obj = {prob_d.value:.4f}")

# Recover theta via the diagonal scaling D matching  $\langle D \phi, D \phi \rangle = K$ 
```

```

D = np.diag([np.sqrt(3), np.sqrt(3), np.sqrt(3), np.sqrt(3), np.sqrt(6),
             np.sqrt(3), np.sqrt(3), 1.0, 1.0])
theta_d = (alpha_p * y) @ Z @ D

scores = Z @ D @ theta_d
theta0_d = -0.5 * (scores[y == -1].max() + scores[y == 1].min())
print(f"recovered: theta_0 = {theta0_d:+.4f}, "
      f"primal/dual obj diff = {prob.value - prob_d.value:.4e}")

# Support vectors: alpha_i > eps
# (active margin constraint, by complementary slackness).
sv_eps = 1e-4
sv_mask = alpha_p > sv_eps
print(f"support vectors: {int(sv_mask.sum())} / {n} (alpha_i > {sv_eps:.0e})")

vals_d = (cubic(G_flat) @ D @ theta_d + theta0_d).reshape(G1.shape)
fig, ax = plt.subplots(figsize=(6, 5))
cs = ax.contourf(G1, G2, vals_d, levels=10, cmap="RdBu")
for label in (-1, 1):
    mask_class = y == label
    # Non-support-vector points: faded
    mask_non_sv = mask_class & ~sv_mask
    ax.scatter(X[mask_non_sv, 0], X[mask_non_sv, 1], s=10,
              edgecolor="k", linewidth=0.3, alpha=0.5, label=f"y = {label}")
    # Support vectors: ringed
    mask_sv = mask_class & sv_mask
    ax.scatter(X[mask_sv, 0], X[mask_sv, 1], s=70,
              facecolor="white", edgecolor="k", linewidth=1.2)
ax.set_xlabel(r"$x_1$"); ax.set_ylabel(r"$x_2$"); ax.legend()
ax.set_title(f"Dual SVM: {int(sv_mask.sum())} support vectors "
            f"highlighted (white-fill, ringed)")
fig.colorbar(cs)
plt.show()

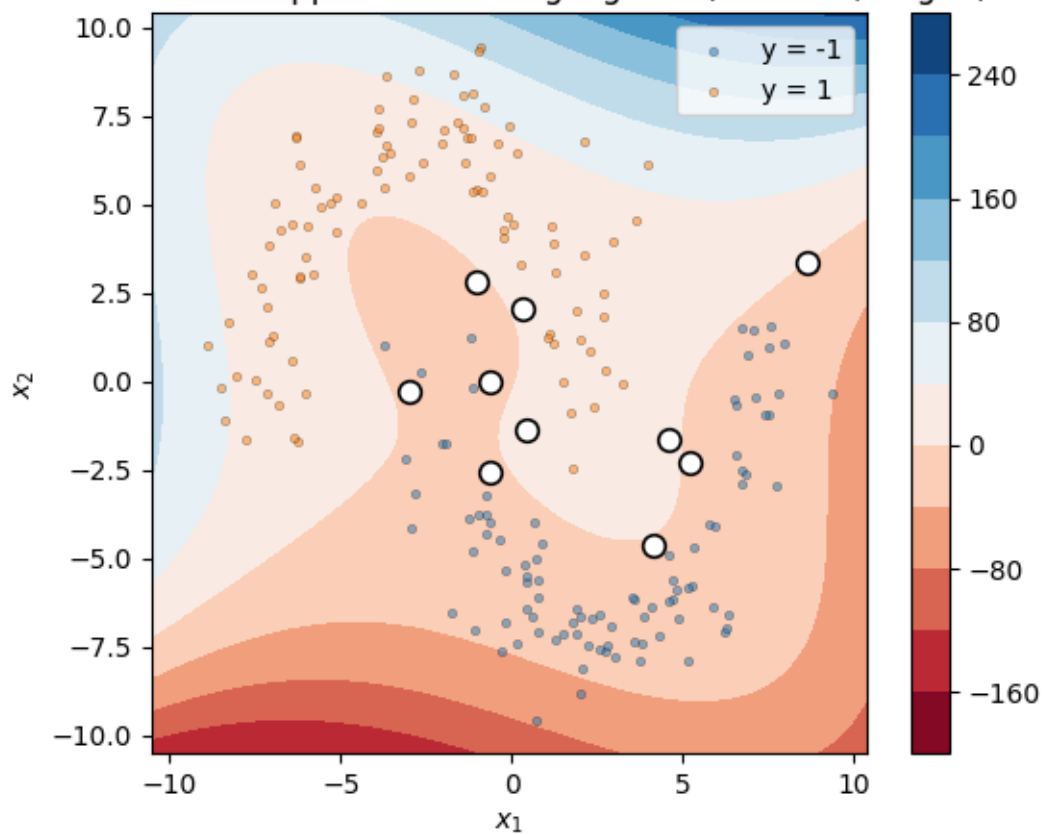
```

```

dual:    status = optimal, obj = 0.4163
recovered: theta_0 = -0.1928, primal/dual obj diff = 7.0105e-09
support vectors: 10 / 200 (alpha_i > 1e-04)

```

Dual SVM: 10 support vectors highlighted (white-fill, ringed)



Exercise 13: Bayesian Optimization

Optimization in Machine Learning

Reproducibility

Produced with Python 3.10 and R 4.4.1. To reproduce, install the pinned package versions below.

Python

```
pip install numpy==2.0.2 matplotlib==3.10.8 scipy==1.15.3 scikit-learn==1.7.2
```

R

```
install.packages("remotes")
remotes::install_version("ggplot2", "3.5.1")
remotes::install_version("DiceKriging", "1.6.1")
remotes::install_version("patchwork", "1.3.0")
```

Problem 1: Closed-form Expected Improvement

Derive the closed form expression of the **Expected Improvement** acquisition function:

$$a_{\text{EI}}(\mathbf{x}) = (f_{\min} - \hat{f}(\mathbf{x})) \Phi\left(\frac{f_{\min} - \hat{f}(\mathbf{x})}{\hat{s}(\mathbf{x})}\right) + \hat{s}(\mathbf{x}) \phi\left(\frac{f_{\min} - \hat{f}(\mathbf{x})}{\hat{s}(\mathbf{x})}\right).$$

Assume $Y(\mathbf{x}) \sim \mathcal{N}(\hat{f}(\mathbf{x}), \hat{s}^2(\mathbf{x}))$.

Hints:

- Start from $a_{\text{EI}}(\mathbf{x}) = \mathbb{E}_y[\max\{f_{\min} - y, 0\}] = \int \max\{f_{\min} - y, 0\} p(y) dy$.
- Decompose the integral by whether $y < f_{\min}$ or $y \geq f_{\min}$.

- Substitute $u = (y - \hat{f}(\mathbf{x}))/\hat{s}(\mathbf{x})$.
- Use $\int_{-\infty}^z u \phi(u) du = -\phi(z)$.

Solution.

We start with

$$a_{\text{EI}}(\mathbf{x}) = \mathbb{E}_y(\max\{f_{\min} - y, 0\}) = \int_{-\infty}^{\infty} \max\{f_{\min} - y, 0\} p(y) dy.$$

Observe that

$$\max\{f_{\min} - y, 0\} = \begin{cases} f_{\min} - y, & \text{if } y < f_{\min}, \\ 0, & \text{otherwise.} \end{cases}$$

All contributions for $y \geq f_{\min}$ are zero. Therefore, we can additively decompose the integral and it simplifies to

$$a_{\text{EI}}(\mathbf{x}) = \int_{-\infty}^{f_{\min}} (f_{\min} - y) p(y) dy.$$

$$\begin{aligned} \alpha_{\text{EI}}(\mathbf{x}) &= \int_{-\infty}^{f_{\min}} (f_{\min} - y) p(y) dy \\ &= \int_{-\infty}^{f_{\min}} (f_{\min} - y) \frac{1}{\sqrt{2\pi\hat{s}(\mathbf{x})^2}} \exp\left(-\frac{(y - \hat{f}(\mathbf{x}))^2}{2\hat{s}(\mathbf{x})^2}\right) dy \\ &= \int_{-\infty}^z (f_{\min} - \hat{f}(\mathbf{x}) - u\hat{s}(\mathbf{x})) \frac{1}{\sqrt{2\pi\hat{s}(\mathbf{x})^2}} \exp\left(-\frac{u^2}{2}\right) \hat{s}(\mathbf{x}) du \quad \left(\text{Def. } u := \frac{y - \hat{f}(\mathbf{x})}{\hat{s}(\mathbf{x})}, \frac{du}{dy} = \frac{1}{\hat{s}(\mathbf{x})}, z := \frac{f_{\min} - \hat{f}(\mathbf{x})}{\hat{s}(\mathbf{x})}\right) \\ &= \int_{-\infty}^z (f_{\min} - \hat{f}(\mathbf{x}) - u\hat{s}(\mathbf{x})) \phi(u) du \\ &= \int_{-\infty}^z (f_{\min} - \hat{f}(\mathbf{x})) \phi(u) du - \int_{-\infty}^z (u\hat{s}(\mathbf{x})) \phi(u) du \end{aligned}$$

Note that

$$\Phi(z) = \int_{-\infty}^z \phi(u) du$$

by definition.

Therefore, regarding the first integral:

$$\int_{-\infty}^z (f_{\min} - \hat{f}(\mathbf{x})) \phi(u) du = (f_{\min} - \hat{f}(\mathbf{x})) \Phi(z) = z \hat{s}(\mathbf{x}) \Phi(z).$$

Regarding the second integral we use the identity

$$\int_{-\infty}^z u \phi(u) du = -\phi(z).$$

Putting both together we obtain:

$$\begin{aligned} \alpha_{\text{EI}}(\mathbf{x}) &= z \hat{s}(\mathbf{x}) \Phi(z) - \hat{s}(\mathbf{x}) (-\phi(z)) \\ &= z \hat{s}(\mathbf{x}) \Phi(z) + \hat{s}(\mathbf{x}) \phi(z) \\ &= (f_{\min} - \hat{f}(\mathbf{x})) \Phi \left(\frac{f_{\min} - \hat{f}(\mathbf{x})}{\hat{s}(\mathbf{x})} \right) + \hat{s}(\mathbf{x}) \phi \left(\frac{f_{\min} - \hat{f}(\mathbf{x})}{\hat{s}(\mathbf{x})} \right). \end{aligned}$$

Problem 2: Implement Bayesian Optimization

Implement BO with a Gaussian-process surrogate and Expected Improvement acquisition.
Goal: minimize

$$f : [0, 1] \rightarrow \mathbb{R}, \quad x \mapsto 2x \sin(14x).$$

Start with 4 points sampled uniformly at random and terminate after 10 total evaluations.

1. Write the BO loop in pseudocode.
2. Implement it. Optimize the EI via a univariate method (Brent's / `optimize`).

Solution.

Let $\mathcal{D}$ be the initial design consisting of $(x^{[1]}, y^{[1]}), \dots, (x^{[4]}, y^{[4]})$. Set t to 4.

While $t < 10$:

1. Fit surrogate model on $\mathcal{D}$.
2. Optimize the Expected Improvement $a_{\text{EI}}(x)$ to obtain a new point $x^{[t+1]} := \arg \max_{x \in [0, 1]} a_{\text{EI}}(x)$.
3. Evaluate $x^{[t+1]}$ and update design data $\mathcal{D} = \mathcal{D} \cup \{(x^{[t+1]}, f(x^{[t+1]}))\}$.
4. Set t to $t + 1$.

Return x that minimizes $f(x)$ in $\mathcal{D}$: $\arg \min_{(x, y) \in \mathcal{D}} y$.

i Note

R vs. Python. Both use a Gaussian-kernel GP surrogate with the same closed-form EI, but they draw the 4 initial points from different RNGs (R's `runif` vs numpy's), so the specific evaluations - and hence the exact GP posterior and its uncertainty bands - differ between the panels. The method and where BO converges are the same.

R

```
library(DiceKriging)
library(ggplot2)
library(patchwork)

set.seed(308)
f <- function(x) 2 * x * sin(14 * x) # objective

# Initial design: 4 random points in [0, 1]
initial_x <- runif(4)
design <- data.frame(x = initial_x, y = f(initial_x))
t <- 4

# Expected improvement using the closed form derived in problem 1.
ei <- function(x, current_fmin, current_gp) {
  pr <- predict(current_gp, newdata = data.frame(x = x), type = "SK")
  diff <- current_fmin - pr$mean
  z <- diff / pr$sd
  diff * pnorm(z) + pr$sd * dnorm(z)
}

while (t < 10) {
  gp <- km(design = design[, 1L, drop = FALSE], response = design[, 2L],
          covtype = "gauss", nugget = 1e-8, control = list(trace = FALSE))
  fmin <- min(design$y)
  x_new <- optimize(ei, interval = c(0, 1), maximum = TRUE,
                  current_fmin = fmin, current_gp = gp)$maximum
  design <- rbind(design, data.frame(x = x_new, y = f(x_new)))
  t <- t + 1
}

bo_best <- design[which.min(design$y), ]
```

```

cat(sprintf("BO best after 10 evals: x = %.4f, f = %+.4f\n",
           bo_best$x, bo_best$y))

# Objective with BO's evaluations (light -> dark = evaluation order)
x_grid <- seq(0, 1, length.out = 401)
df_curve <- data.frame(x = x_grid, y = f(x_grid))
df_design <- design
df_design$step <- seq_len(nrow(df_design))

ggplot() +
  geom_line(data = df_curve, aes(x, y), color = "#0033A0", linewidth = 0.5) +
  geom_point(data = df_design, aes(x, y, color = step), size = 2.4) +
  geom_point(data = bo_best, aes(x, y), color = "#D90012",
            size = 3.5, shape = 4, stroke = 1.5) +
  scale_color_gradient(low = "grey80", high = "#F2A800") +
  ylab("f(x)") +
  ggtitle("BO evaluations on the objective (light->dark = order, red X = best)")

# GP posterior at the final BO iteration (mean +/- 2*sd over the grid)
final_gp <- km(design = design[, 1L, drop = FALSE], response = design[, 2L],
              covtype = "gauss", nugget = 1e-8, control = list(trace = FALSE))
pred <- predict(final_gp, newdata = data.frame(x = x_grid), type = "SK")
df_post <- data.frame(x = x_grid, mean = pred$mean,
                    lo = pred$mean - 2 * pred$sd,
                    hi = pred$mean + 2 * pred$sd)

# Expected improvement induced by that surrogate;
# BO would query its argmax next.
ei_grid <- ei(x_grid, min(design$y), final_gp)
x_ei_next <- x_grid[which.max(ei_grid)]
df_ei <- data.frame(x = x_grid, ei = ei_grid)

p_post <- ggplot() +
  geom_ribbon(data = df_post, aes(x, ymin = lo, ymax = hi),
            fill = "#F2A800", alpha = 0.35) +
  geom_line(data = df_post, aes(x, mean), color = "#F2A800", linewidth = 0.7) +
  geom_line(data = df_curve, aes(x, y), color = "#0033A0", linewidth = 0.5) +
  geom_point(data = design, aes(x, y), color = "black", size = 1.8) +
  geom_vline(xintercept = x_ei_next, linetype = "dashed",
            color = "#D90012", linewidth = 0.5) +
  ylab("f(x)") +
  ggtitle("GP posterior + EI acquisition (dashed = argmax EI)") +

```

```

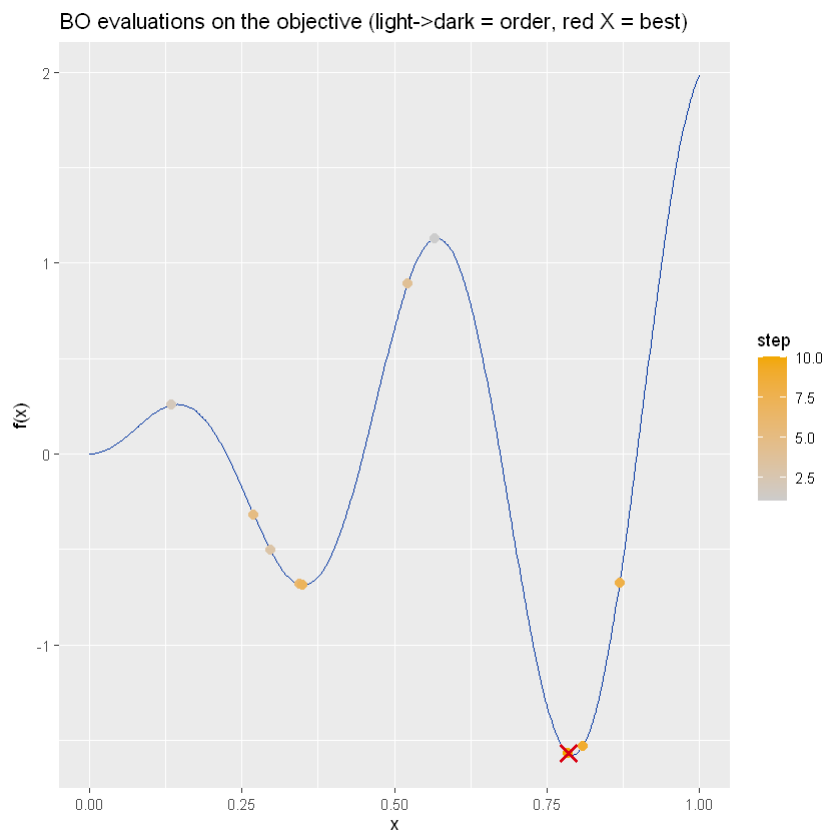
theme(axis.title.x = element_blank(), axis.text.x = element_blank())

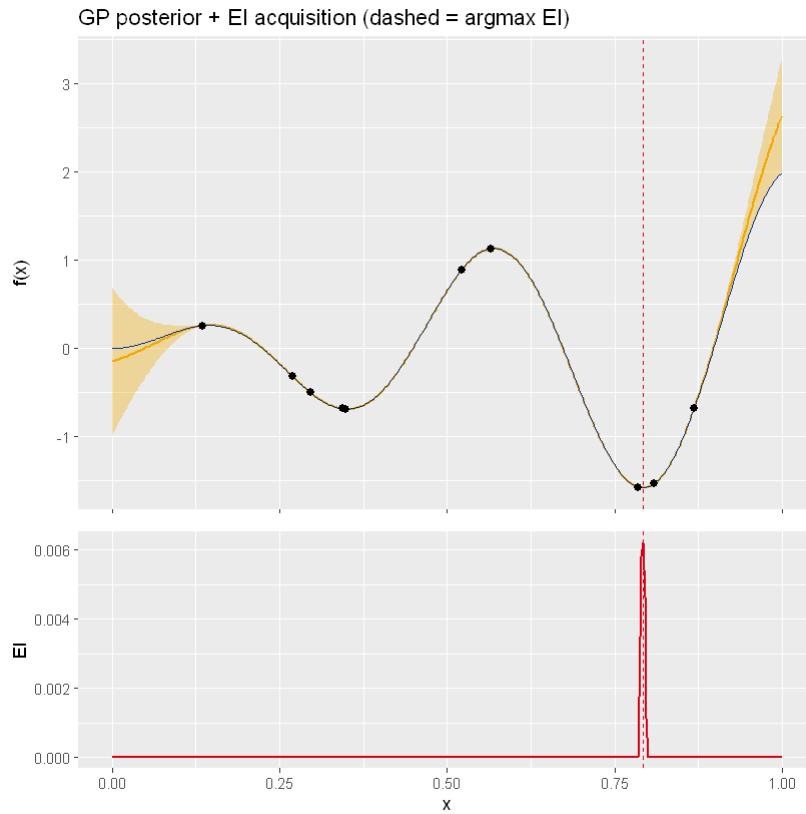
p_ei <- ggplot(df_ei, aes(x, ei)) +
  geom_line(color = "#D90012", linewidth = 0.6) +
  geom_vline(xintercept = x_ei_next, linetype = "dashed",
            color = "#D90012", linewidth = 0.5) +
  ylab("EI") + xlab("x")

print((p_post / p_ei) + plot_layout(heights = c(2, 1)))

```

BO best after 10 evals: $x = 0.7854$, $f = -1.5708$





Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
from scipy.optimize import minimize_scalar
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import ConstantKernel, RBF

def f(x):
    """Objective: 2x sin(14x) on [0, 1]."""
    return 2 * x * np.sin(14 * x)

# Gaussian-kernel GP surrogate, matching R's DiceKriging covtype = "gauss".
```

```

kernel = ConstantKernel(1.0) * RBF(length_scale=0.1)

def fit_gp(xd, yd):
    gp = GaussianProcessRegressor(kernel=kernel, alpha=1e-8, normalize_y=True)
    gp.fit(xd[:, None], yd)
    return gp

# Expected improvement using the closed form derived in problem 1.
def ei(x, current_fmin, gp):
    mu, sd = gp.predict(np.array([[x]]), return_std=True)
    mu, sd = mu[0], max(sd[0], 1e-9)
    diff = current_fmin - mu
    z = diff / sd
    return diff * norm.cdf(z) + sd * norm.pdf(z)

# Initial design: 4 random points in [0, 1]
np.random.seed(308)
x_design = np.random.uniform(size=4)
y_design = f(x_design)
t = 4

while t < 10:
    gp = fit_gp(x_design, y_design)
    fmin = y_design.min()
    res = minimize_scalar(lambda x: -ei(x, fmin, gp),
                        bounds=(0, 1), method="bounded")
    x_design = np.append(x_design, res.x)
    y_design = np.append(y_design, f(res.x))
    t += 1

best = int(np.argmaxmin(y_design))
print(f"BO best after 10 evals: x = {x_design[best]:.4f}, "
      f"f = {y_design[best]:+.4f}")

# Objective with BO's evaluations (light -> dark = evaluation order)
x_grid = np.linspace(0, 1, 401)
fig, ax = plt.subplots(figsize=(7, 4.5))
ax.plot(x_grid, f(x_grid), color="#0033A0", linewidth=0.6)
sc = ax.scatter(x_design, y_design, c=np.arange(len(x_design)), cmap="YlOrBr",

```

```

        s=45, edgecolor="k", linewidth=0.4)
ax.scatter(x_design[best], y_design[best], color="#D90012", marker="x",
           s=140, linewidths=2)
ax.set_xlabel("x"); ax.set_ylabel("f(x)")
fig.colorbar(sc, ax=ax, label="call index")
ax.set_title("B0 evaluations (light->dark = order, red X = best)")
plt.show()

# GP posterior at the final iteration (mean +/- 2*sd over the grid)
final_gp = fit_gp(x_design, y_design)
mu, sd = final_gp.predict(x_grid[:, None], return_std=True)

# EI induced by that surrogate; B0 would query its argmax next.
fmin = y_design.min()
sd_safe = np.maximum(sd, 1e-9)
z = (fmin - mu) / sd_safe
ei_grid = (fmin - mu) * norm.cdf(z) + sd_safe * norm.pdf(z)
x_ei_next = x_grid[int(np.argmax(ei_grid))]

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7, 6), sharex=True,
                               gridspec_kw={"height_ratios": [2, 1]})
ax1.fill_between(x_grid, mu - 2 * sd, mu + 2 * sd, color="#F2A800", alpha=0.35,
                 label="GP mean +/- 2 sd")
ax1.plot(x_grid, mu, color="#F2A800", linewidth=0.8, label="GP mean")
ax1.plot(x_grid, f(x_grid), color="#0033A0", linewidth=0.6, label="true f")
ax1.scatter(x_design, y_design, color="black", s=22, label="evaluations")
ax1.axvline(x_ei_next, linestyle="--", color="#D90012", linewidth=0.8)
ax1.set_ylabel("f(x)"); ax1.legend(loc="upper left", fontsize=8)
ax1.set_title("GP posterior + EI acquisition (Gaussian kernel)")
ax2.plot(x_grid, ei_grid, color="#D90012", linewidth=0.9)
ax2.axvline(x_ei_next, linestyle="--", color="#D90012", linewidth=0.8,
            label="argmax EI = next query")
ax2.set_xlabel("x"); ax2.set_ylabel(r"$a_{\mathrm{EI}}(x)$")
ax2.legend(loc="upper right", fontsize=8)
plt.show()

```

B0 best after 10 evals: x = 0.7898, f = -1.5766

